

Affinity SDK – Complete Scripting Tutorial

Affinity SDK — Complete Scripting Tutorial

Copyright © 2026

All rights reserved.

Fonts: IBM Plex Serif, IBM Plex Sans, IBM Plex Mono

Source Code: <https://github.com/peterfrieese/eightbyten>

ISBN: 978-X-XXXX-XXXX-X

Printed in the Cloud.

For everyone learning to automate Affinity by Canva.

Contents

Chapter 1: Affinity SDK — Complete Learning Tutorial	3	PART I. PHASE 1 — FOUNDATIONS	9
1.1 What this tutorial is	3	Chapter 2: Phase 1 — Foundations	11
1.2 What is the Affinity SDK	3	2.1 Lessons	11
1.3 Who this tutorial is for	4	2.2 What to expect	12
1.4 What you will be able to build	4	Chapter 3: Lesson 01 — Environment Setup & Execution Model	13
1.5 How the tutorial is structured	4	3.1 What this lesson covers	13
1.6 Phase overview	5	3.2 The execution model	13
1.7 How to use this tutorial	5	3.3 The module system	14
1.8 The three rules everything else builds on	6	3.4 console.log() — your only output	14
1.9 Running a script	6	3.5 File system access	14
1.10 SDK module reference	6	3.6 Application settings	14
1.11 Supporting files	7	3.7 Exercises	15
		3.8 SDK modules used in this lesson	15
		3.9 See also	15
		3.10 Examples	15
		Chapter 4: Lesson 02 — Application & Document Model	17
		4.1 What this lesson covers	17
		4.2 The document hierarchy — JS API vs node tree	17
		4.3 Getting the current document	18
		4.4 All open documents	19
		4.5 Spreads vs pages	19
		4.6 The selection / currentSpread ordering rule	19
		4.7 Document state flags	19
		4.8 Creating a new document	20
		4.9 Async document operations	20

4.10 Document properties	20
4.11 Key gotchas	24
4.12 Exercises	24
4.13 SDK modules used in this lesson	24
4.14 See also	24
4.15 Examples	24

Chapter 5: Lesson 03 — Node Model & Tree Traversal 27

5.1 What this lesson covers	27
5.2 The node tree	27
5.3 Traversing layers	28
5.4 Multi-spread traversal	28
5.5 Node type guards	28
5.6 Working with the selection	29
5.7 Node interfaces reference	29
5.8 NodeChildType	30
5.9 ContainerNode — named layer groups	30
5.10 Node convenience methods	30
5.11 Per-node tag metadata	31
5.12 Exercises	32
5.13 SDK modules used in this lesson	33
5.14 See also	33
5.15 Examples	33

PART II. PHASE 2 — CORE API 35

Chapter 6: Phase 2 — Core API 37

6.1 The core loop	37
6.2 Lessons	38
6.3 What to expect	38

Chapter 7: Lesson 04 — The Command Pattern 39

7.1 What this lesson covers	39
7.2 The golden rule	39
7.3 Basic command execution	40
7.4 CompoundCommandBuilder — one undo step for many changes ..	40
7.5 AddChildNodesCommandBuilder — creating new nodes ..	40
7.6 Reading command results	41
7.7 Preview mode	41
7.8 Async command execution	41
7.9 Undo and redo	42
7.10 Common DocumentCommand factories ..	42
7.11 Key gotchas	42
7.12 Exercises	43
7.13 SDK modules used in this lesson	43
7.14 See also	43
7.15 Examples	43

Chapter 8: Lesson 05 — Fills & Strokes 45

8.1 What this lesson covers	45
8.2 Colours	45
8.3 FillDescriptor	46
8.4 Gradient fills	46
8.5 Applying fills to selected nodes	47
8.6 Three fill channels	48
8.7 Line styles	48
8.8 Line caps and joins	49
8.9 Dash patterns	49
8.10 LineStyleMask — partial updates	50
8.11 Stroke alignment	50
8.12 Opacity and blend mode	51
8.13 Hatch fills	51
8.14 Bitmap fills	51
8.15 Doc convenience fill methods	52
8.16 VectorNodeDefinition — multi-fill and multi-stroke	52
8.17 Fluent node-definition builders	53
8.18 Colour accepted where FillDescriptor expected	53
8.19 Exercises	54
8.20 SDK modules used in this lesson	54
8.21 See also	54
8.22 Examples	54

Chapter 9: Lesson 06 — Geometry, Transforms & Curves 57

9.1 What this lesson covers	57
-----------------------------------	----

9.2 Coordinate system	57
9.3 Transforms — moving and resizing nodes ..	58
9.4 Adding basic shapes	59
9.5 Shape parameters	59
9.6 Building curves with CurveBuilder	60
9.7 Bezier curves	60
9.8 Grid duplicator	61
9.9 Converting a shape to curves	61
9.10 Rasterising objects	61
9.11 Adding artboards	62
9.12 Knife cut and scissor cut	62
9.13 Key gotchas	62
9.14 Exercises	62
9.15 Combining bounding boxes	63
9.16 SDK modules used in this lesson	63
9.17 See also	63
9.18 Examples	64

PART III. PHASE 3 — TOOLS 67

Chapter 10: Phase 3 — Tools 69

10.1 Lessons	70
10.2 What to expect	70

Chapter 11: Lesson 07 — Text & Story Model 71

11.1 What this lesson covers	72
11.2 How text works in Affinity SDK	72
11.3 Reading text from a node	72
11.4 Replacing all text in a frame	72
11.5 Appending text to a frame	73
11.6 The selection setup order matters	74
11.7 Applying character formatting (StoryDelta)	74
11.8 StoryDelta convenience factories	74
11.9 StoryBuilder — building text from scratch	75
11.10 TableTextNode	75
11.11 GlyphAtts and ParagraphAtts — attribute wrappers	76
11.12 Attribute runs on Story	77
11.13 Batch text update — rename all text nodes	77
11.14 Key gotchas	78
11.15 Exercises	78
11.16 SDK modules used in this lesson	78
11.17 See also	79
11.18 Examples	79

Chapter 12: Lesson 08 — Dialog UI 83

12.1 What this lesson covers	84
12.2 The Dialog hierarchy	84
12.3 Minimal dialog example	84
12.4 All control types	84
12.5 Dialog-level properties	89
12.6 Multi-column dialogs	90
12.7 Reactive controls	90
12.8 Full practical example — export dialog	91
12.9 Reactive dialog example — shape creator	91
12.10 Live preview dialogs	92
12.11 Attaching controls to the dialog object ..	93
12.12 alert()	93
12.13 The canonical example — addGuides.js .	93
12.14 Key gotchas	93
12.15 Exercises	94
12.16 SDK modules used in this lesson	94
12.17 See also	95
12.18 Examples	95

Chapter 13: Lesson 09 — Export & File I/O 103

13.1 What this lesson covers	104
13.2 Export overview	104
13.3 Getting available export presets	104
13.4 Synchronous export	104

13.5 Export areas	105
13.6 Export every artboard	105
13.7 Async export with doc.promises	106
13.8 Saving the document	106
13.9 File system — reading and writing text ..	107
13.10 JSON sidecar pattern	107
13.11 Network requests	108
13.12 Key gotchas	108
13.13 200	108
13.14 Exercises	108
13.15 SDK modules used in this lesson	109
13.16 See also	109
13.17 Examples	109

PART IV. PHASE 4 — ADVANCED 113

Chapter 14: Phase 4 — Advanced 115

14.1 Lessons	116
14.2 What to expect	116

Chapter 15: Lesson 10 — AI Commands 117

15.1 What this lesson covers	118
15.2 Why use the SDK AI commands?	118
15.3 Generate an image from a prompt	118
15.4 Generative edit (inpainting)	118
15.5 Remove background	118
15.6 Select subject	119
15.7 Detect depth	119
15.8 Colourise	119
15.9 Image trace (raster to vector)	119
15.10 Batch image generation	120
15.11 Using AI commands with doc.promises (async)	120
15.12 Dialog-driven image generator	120
15.13 Key gotchas	121
15.14 Exercises	121
15.15 SDK modules used in this lesson	121
15.16 See also	121
15.17 Examples	122

Chapter 16: Lesson 11 — Layer Effects & Adjustments 123

16.1 What this lesson covers	124
16.2 Effect categories	124
16.3 Outer shadow	124
16.4 The `enableIfDisabled` parameter	125
16.5 The index parameter	125
16.6 Outline (stroke effect)	125
16.7 Inner glow	125
16.8 Gaussian blur (layer effect)	126
16.9 Outer glow	126
16.10 Inner shadow	127
16.11 Colour overlay	127
16.12 Bevel and emboss	127
16.13 Remove all effects	128
16.14 Gradient overlay	128
16.15 Phong bevel	129
16.16 Destructive filters on raster nodes	130
16.17 Drawing scale & margins	131
16.18 Raster brush properties	131
16.19 Key gotchas	131
16.20 Exercises	132
16.21 SDK modules used in this lesson	132
16.22 See also	132
16.23 Examples	133

Chapter 17: Lesson 12 — Pro Patterns: Async, Hints & Network 135

17.1 What this lesson covers	136
17.2 The doc.promises API	136

17.3 Processing and exporting the current document	137	19.9 Read-only analysis (no write-back needed)	157
17.4 Full async pipeline	137	19.10 Creating a bitmap from scratch	157
17.5 Network requests	138	19.11 Loading a bitmap from a file	158
17.6 Remote data → shapes pattern	138	19.12 Rendering a node to a bitmap with NodeRenderingEngine	158
17.7 Timers	139	19.13 The Buffer class	159
17.8 search_sdk_hints — community knowledge base	139	19.14 Performance considerations	160
17.9 add_sdk_hint — contributing back	139	19.15 Key gotchas	160
17.10 save_script_to_library	139	19.16 Raster pixel selection	160
17.11 Production script checklist	140	19.17 SDK modules used in this lesson	162
17.12 Key gotchas	140	19.18 See also	162
17.13 Exercises	140	19.19 Examples	162
17.14 SDK modules used in this lesson	141		
17.15 See also	141	Chapter 20: Lesson 15 — Fonts & Typography	171
17.16 Examples	141	20.1 What this lesson covers	172
Chapter 18: Lesson 13 — Special Modules: Pixel Access, Brushes & More	143	20.2 The fonts module	172
18.1 What this lesson covers	144	20.3 Three levels of font data	172
18.2 Modules at a glance	144	20.4 Enumerating font families	172
18.3 <code>`/pixelaccessor`</code> — Direct pixel read/write	144	20.5 Inspecting fonts within a family	172
18.4 <code>`/taginterface`</code> — Per-node custom metadata	146	20.6 Getting a Font object	173
18.5 <code>`/documentproperties`</code> — Document metadata	146	20.7 Font classification via PANOSE	173
18.6 <code>`/drawingscale`</code> — Drawing/map scale	147	20.8 Font distance (finding the closest match)	174
18.7 <code>`/marginsinterface`</code> — Spread margins	147	20.9 Font collections	175
18.8 <code>`/rasterbrush`</code> — Brush property access	147	20.10 Practical: check which fonts a document uses	175
18.9 <code>`DocumentCommand`</code> — full method reference	148	20.11 Practical: font picker dialog	175
18.10 <code>`createSetEditable`</code> — argument order	149	20.12 Font properties reference	176
18.11 <code>`AddChildNodesCommandBuilder`</code> — insertion options	149	20.13 FontFamily properties reference	177
18.12 SDK native binding modules	149	20.14 Key gotchas	177
18.13 Exercises	149	20.15 Exercises	178
18.14 New node types	150	20.16 SDK modules used in this lesson	178
18.15 SDK modules used in this lesson	150	20.17 See also	178
18.16 See also	150	20.18 Examples	178
18.17 Examples	150		
Chapter 19: Lesson 14 — Pixel Processing ..	153	Chapter 21: Lesson 16 — Raster Adjustments	181
19.1 What this lesson covers	154	21.1 What this lesson covers	181
19.2 What a pixel is in this context	154	21.2 Creating an adjustment layer	181
19.3 Why pixel processing rather than filters or adjustments	154	21.3 Reading parameters from an existing adjustment node	182
19.4 Why pixel processing is slow — and what to do about it	155	21.4 Brightness / Contrast	182
19.5 Why pixel processing matters	155	21.5 Levels	183
19.6 The pipeline	155	21.6 Curves	183
19.7 Choosing the right format	155	21.7 HSL Shift	184
19.8 Core pixel read/write pattern	156	21.8 White Balance	184
		21.9 Vibrance	184
		21.10 Exposure	184
		21.11 Colour Balance	185
		21.12 Posterise	185
		21.13 Threshold	185
		21.14 Invert	185
		21.15 Recolour	185

21.16 Split Toning	185	23.20 TableTextNode (SDK 3.2)	208
21.17 Black & White	185	23.21 GlyphAtts / ParagraphAtts (SDK 3.2) ...	208
21.18 Tone Compression and Tone Stretch ...	186	23.22 Attribute runs on Story (SDK 3.2)	208
21.19 Normals (normal map rotation)	186	23.23 Fluent node-definition builders (SDK 3.2)	208
21.20 All adjustment types — quick reference	186	23.24 Colour → FillDescriptor auto-convert (SDK 3.2)	208
21.21 Key gotchas	187	23.25 `node.spreadBaseBox` (SDK 3.2)	208
21.22 Exercises	188	23.26 Spread / artboard document properties (SDK 3.2)	208
21.23 HalftoneFilter	188	23.27 Critical rules	208
21.24 SDK modules used in this lesson	189		
21.25 See also	189		
21.26 Examples	189		
Chapter 22: Lesson 17 — Align & Distribute	193		
22.1 What this lesson covers	193		
22.2 How it works	194		
22.3 GroupTransformType — what to do	194		
22.4 GroupTransformAnchor — reference frame	195		
22.5 Common operations	195		
22.6 Full example — align & distribute dialog .	196		
22.7 Combining align and distribute	197		
22.8 Using DocumentCommand directly	197		
22.9 Key gotchas	197		
22.10 Exercises	198		
22.11 SDK modules used in this lesson	198		
22.12 See also	198		
22.13 Examples	198		
Chapter 23: Affinity SDK — Quick Reference Cheat Sheet	201		
23.1 Boilerplate — every script starts here	201		
23.2 Imports by task	201		
23.3 The command loop	202		
23.4 Colours	202		
23.5 Apply fill / stroke / opacity	202		
23.6 Text replacement	203		
23.7 Dialog	203		
23.8 Export	203		
23.9 Layer effects (shadow example)	203		
23.10 Async pipeline	204		
23.11 Additional APIs	204		
23.12 `createSetEditable` — argument order	206		
23.13 Document convenience shorthands (SDK 3.2)	206		
23.14 Document line style shorthands	206		
23.15 StoryDelta convenience factories	206		
23.16 Geometry utilities	206		
23.17 HalftoneFilter (SDK 3.2)	207		
23.18 DialogButton and DialogColumnStack (SDK 3.2)	207		
23.19 Fills — options object pattern (SDK 3.2 breaking change)	207		

Affinity SDK — Complete Learning Tutorial

1

1.0.1 Beta

1.1 What this tutorial is

This is a structured tutorial for learning the Affinity by Canva JavaScript SDK — the scripting system that lets you automate Affinity application using JavaScript running directly inside the application.

The tutorial covers the SDK from the ground up: the execution model, the document structure, the command system, visual properties, text, dialogs, export, AI operations, layer effects, async patterns, pixel processing, and typography. It is built from direct inspection of the SDK source code and the official test suite, and every example script has been verified against a running Affinity installation.

1.2 What is the Affinity SDK

The Affinity SDK is a JavaScript scripting environment embedded inside Affinity by Canva. Scripts run in a sandboxed JS engine that has direct access to the open document — its layers, nodes, text, fills, geometry, and pixel data — through a set of native modules.

It is not a browser API. It is not Node.js. It is a purpose-built environment with its own rules:

- Modules are loaded with `require('/module-name')` — not npm, not ES imports
- Scripts run synchronously inside the application's main thread unless explicitly made async

- All document modifications go through a command system that integrates with Affinity's undo stack
- The only output mechanism available to scripts is `console.log()`, viewed in the script console
- File access is restricted to the Desktop

Within those constraints, scripts can do a remarkable amount: read and modify any property of any layer, apply fills and effects, manipulate text, export assets in any format, call external APIs, drive Affinity's AI features, and process images pixel by pixel.

1.3 Who this tutorial is for

This tutorial assumes you can read and write JavaScript — loops, functions, objects, and basic async patterns. You do not need experience with Node.js, browser APIs, or any specific JavaScript framework. SDK scripting is different from all of them, and this tutorial treats it on its own terms.

You do not need any prior experience with Affinity's scripting panel. Lesson 01 covers everything you need to get your first script running.

1.4 What you will be able to build

By the end of this tutorial you will be able to write scripts that:

- Iterate every layer in a document, identify node types, and read or modify their properties
- Apply fills, strokes, gradients, and effects to any node or set of nodes in a single undoable command
- Read, replace, and style text in any text frame using the Story API
- Present native dialogs that collect input from the user and drive the rest of the script
- Export every artboard as named files, write JSON manifests, and POST results to an external API
- Generate images from text prompts, remove backgrounds, and chain AI operations with export pipelines
- Process every pixel of a raster layer to implement custom image filters and analysis tools
- Enumerate installed fonts, detect missing typefaces, and substitute them programmatically
- Write async pipelines that chain execute, export, and save operations with proper error handling

1.5 How the tutorial is structured

The tutorial is organised into four phases, each building on the last. Within each phase, lessons are ordered so that earlier lessons are prerequisites for later ones. The phase and lesson READMEs tell you exactly what they depend on.

```
affinity-sdk-course/
├── phase-1-foundations/           Lessons 01-03
│   ├── 01-environment-setup/
│   ├── 02-app-and-document/
│   └── 03-node-model/
├── phase-2-core-api/             Lessons 04-06
│   ├── 04-command-pattern/
│   ├── 05-fills-and-strokes/
│   └── 06-geometry/
├── phase-3-tools/                Lessons 07-09
│   ├── 07-text-and-story/
│   ├── 08-dialog-ui/
│   └── 09-export-and-files/
└── phase-4-advanced/             Lessons 10-17
```

```

├── 10-ai-commands/
├── 11-layer-effects/
├── 12-pro-patterns/
├── 13-special-modules/
├── 14-pixel-processing/
├── 15-fonts-and-typography/
├── 16-raster-adjustments/
└── 17-align-distribute/

```

Each lesson folder contains:

- `README.md` — a full explanation of the concepts, the correct API patterns, the common failure modes, and a key gotchas section that documents the non-obvious behaviours that cost time to discover independently
- `examples/` — runnable `.js` scripts that you paste directly into Affinity's script panel. Every example is self-contained and includes comments explaining the non-obvious parts

1.6 Phase overview

Phase	What it covers	Lessons
1 – Foundations	The execution model, document entry point, and node tree. Everything else depends on this.	1–3
2 – Core API	The command system, fills, strokes, geometry, and transforms. This is where scripts start making changes.	4–6
3 – Tools	Text automation, dialog UI, file export, and HTTP networking. This is where scripts become useful to other people.	7–9
4 – Advanced	AI commands, layer effects, raster adjustments, async pipelines, pixel processing, typography, and align/distribute. The full capability of the SDK.	10–17

1.7 How to use this tutorial

Read the README before running the examples. Each lesson README explains the concepts, the gotchas, and the argument orders before you encounter them in code. The examples will make more sense — and fail less often — if you understand the model first.

Keep CHEAT-SHEET.md open while coding. It is a single-page reference covering the patterns, argument orders, and critical rules that come up in almost every session. The critical rules table at the bottom is particularly worth memorising — it documents the SDK behaviours that produce silent failures rather than helpful error messages.

Run the examples on a real document. Most examples require an open document and a selection. Start with a simple document — a few shapes on a single page — until you understand how a script interacts with it. Move to more complex documents once the patterns are clear. I would encourage you to type all the examples yourself. Don't just copy-paste. The muscle memory you will gain will serve you well in the future.

Work through phases in order, but not necessarily every lesson. Phases 1 and 2 are prerequisites for everything. Within Phase 3 and 4, individual lessons are largely independent — if you need text automation now, go to Lesson 07. If you need pixel processing, go to Lesson 14. Come back to the others when you need them.

Use SDK-EXPOSURE-MAP.md as a reference. It documents the full API surface — every module, every class, every method — in one place. When you need something the lessons don't cover, start there.

1.8 The three rules everything else builds on

1. `require('/module')` — not `./module` The SDK has its own module registry. Paths start with `/`. There is no npm, no ES modules, no `node_modules`. If a `require` call silently returns `undefined`, the module path is wrong.

2. `console.log()` is your only output Scripts don't return values to the UI. Everything you want to see must be logged. View the output in Affinity's script console panel.

3. All document changes go through `doc.executeCommand()` Never mutate nodes directly. Build a command object and execute it. This is what gives you undo/redo integration, and it is also what gives the command system its ability to batch multiple changes into a single undoable step via `CompoundCommandBuilder`.

1.9 Running a script

1. Open a document in Affinity by Canva
2. Open the script panel: **Edit → Script Editor...**
3. Paste/type a script into the editor
4. Click **Run** — output appears in the console below the editor

For scripts that require a selection, select the relevant objects before clicking Run.

1.10 SDK module reference

Module	Key exports
<code>/application</code>	<code>app</code> — version, platform, Desktop path
<code>/document</code>	<code>Document</code> , <code>DocumentPromises</code> , <code>DocumentPreset</code> , <code>DocumentHistory</code> , <code>DocumentSnapshot</code> , <code>FileExportOptions</code> , <code>FileExportArea</code> , <code>NewDocumentOptions</code> , <code>LoadDocumentOptions</code>
<code>/commands</code>	<code>DocumentCommand</code> , <code>CompoundCommandBuilder</code> , <code>AddChildNodesCommandBuilder</code>
<code>/nodes</code>	All node classes — <code>ShapeNode</code> , <code>TextNode</code> , <code>RasterNode</code> , etc.

Module	Key exports
/selections	Selection, TextSelection
/colours	RGBA8, Colour, ColourProfile
/fills	FillDescriptor, SolidFill, GradientFill, HatchFill, BitmapFill, No Fill
/linestyle	LineStyle, LineStyleMask, Line Type
/geometry	Transform, CurveBuilder, Poly Curve, Bezier
/shapes	Shape definitions and parameters
/dialog	Dialog, all control types, DialogResult
/storybuilder	StoryBuilder — construct text content programmatically
/storydelta	StoryDelta — describe text formatting changes
/glyphatts	Character-level formatting attributes
/paragraphatts	Paragraph-level formatting attributes
/pixelaccessor	PixelReaderWriterRGBA8 and all other pixel accessor classes
/rasterobject	Bitmap, PixelBuffer, NodeRenderingEngine, RasterFormat
/fonts	Font, FontFamily, FontWeight, Font Width
/fs	File — Desktop read/write only
/network	HttpRequest, RequestMethod
/timers	setTimeout(delay, callback), setInterval(delay, callback)
/buffer	Buffer — raw binary data
/units	UnitType, UnitValueConverter

For the complete API surface including all methods and argument signatures, see SDK-EXPOSURE-MAP.md.

1.11 Supporting files

File	Purpose
SDK-PRIMER.md	AI project starter — correct patterns, broken APIs, non-obvious behaviours. Read this before writing any code in a new session.

File	Purpose
CHEAT-SHEET.md	One-page quick reference — keep this open while coding
SDK-EXPOSURE-MAP.md	Full API surface inventory — modules, classes, methods

Phase 1 — Foundations

PART



Phase 1 — Foundations

2

Every Affinity script, no matter how sophisticated, is built on three pieces of knowledge: how the script runtime works, how a document is structured, and how the objects inside it are organised. Phase 1 covers all three.

This is not a phase you should rush. The patterns established here — getting the current document, guarding against null, iterating the node tree — appear in virtually every script you will ever write. Developers who skip ahead and try to write something complex before understanding these foundations consistently hit the same walls: confusing errors about undefined properties, scripts that work on one document and fail silently on another, loops that miss nodes or visit them twice. The three lessons in this phase exist specifically to prevent those problems.

The Affinity SDK is not a browser API and not a Node.js API. It is a JavaScript environment embedded inside a running design application, with its own rules about what is allowed, what modules exist, and how the document model is exposed. Lesson 01 orients you in that environment so that none of its constraints catch you off guard. Lesson 02 introduces the document itself — the entry point for every script — and explains the spread and page model that underpins every Affinity file. Lesson 03 maps the node tree: the structure that holds every layer, group, shape, and text frame in the document.

These three lessons have no dependencies on each other — but every later phase depends on them.

2.1 Lessons

Lesson	Topic	Key skill you'll gain
01 — Environment setup	Execution model, console, module system	Running your first script

Lesson	Topic	Key skill you'll gain
02 — App & document	Entry point, current doc, spreads	Accessing any open document
03 — Node model	Tree traversal, node types, selection	Iterating layers and identifying objects

2.2 What to expect

By the end of Phase 1 you should be able to write a script that:

- Runs without throwing errors when pasted into Affinity's script panel
- Guards against no document being open (handles the case where `Document.current` returns `null`)
- Walks every layer in the document
- Logs the name, type, and visibility of each node
- Works correctly on multi-spread (multi-page) documents

You will also understand why every SDK modification requires a command, why the selection must be read before `setCurrentSpread` is called, and why `doc.layers` gives you top-level nodes across all spreads while `doc.layers.all` descends recursively into every group. These are not edge cases — they are core SDK behaviours that every script author encounters within their first hour.

Lesson 01 — Environment Setup & Execution Model

3

Affinity SDK scripts are JavaScript files that run directly inside the Affinity application — not in a browser, not in Node.js, but embedded in the same process as the program itself. This lesson explains exactly how that works: how scripts are loaded and executed, how you access SDK modules with `require()`, and what constraints the environment places on you.

Understanding the execution model before writing a single line of code will save you hours of confusion. The runtime has no `fetch`, no `window`, no `process`, and no file access outside the Desktop. Knowing these boundaries upfront means you will never waste time debugging something that simply cannot work in this environment.

3.1 What this lesson covers

- How Affinity SDK scripts execute
- The `require()` module system
- `console.log()` as your output channel
- File system access constraints
- How to read SDK documentation from within a session

3.2 The execution model

Affinity SDK scripts are **synchronous JavaScript** running inside the Affinity application process.

Key differences from Node.js or browser JS:

Feature	Affinity SDK	Node.js
Module imports	<code>require('/module')</code>	<code>require('./module')</code> or ESM
Package manager	None — built-in modules only	npm
Output	<code>console.log()</code> → Script console	stdout
Return values	Scripts return nothing	Functions return values
File access	Desktop only	Any path
Network	<code>require('/network')</code>	HttpRequest, Request Method

3.3 The module system

All imports use a leading `/`. This is the SDK's internal registry — not the file system.

```
'use strict';
// Correct
const { Document } = require('/document');
const { app } = require('/application');
const { RGBA8 } = require('/colours');

// Wrong — will throw
const { Document } = require('./document'); // no relative paths
const { Document } = require('document'); // no bare specifiers
```

The `.js` extension is optional but harmless:

```
'use strict';
require('/document') // works
require('/document.js') // also works
```

3.4 console.log() — your only output

Scripts produce no visible return value. Everything you want to inspect must be logged.

```
// Snippet
console.log('simple string');
console.log('label:', someValue);
console.log('multiple:', a, b, c);
console.assert(someCondition, 'message if false');
```

View output in the **Script console** panel in Affinity, below the editor.

3.5 File system access

The SDK can only read/write files on the **Desktop**. Any other path returns `PERMISSION_DENIED`.

```
'use strict';
const { app } = require('/application');

// Always use this to get the Desktop path — don't hardcode it
const desktop = app.userDesktopPath;
console.log('Desktop:', desktop);
// macOS: /Users/yourname/Desktop
// Windows: C:\Users\yourname\Desktop
```

3.6 Application settings

`app.settings` exposes application-level preferences that scripts can read or write:

```
'use strict';
const { app } = require('/application');

// Read the undo history limit (read-only)
console.log('Undo limit:', app.settings.undoLimit);

// Read/write whether PSD files load with editable text
console.log('PSD editable text:', app.settings.loadPSDWithEditableText);
app.settings.loadPSDWithEditableText = true;
```

No npm, no node_modules. You cannot install external libraries. Only the built-in SDK modules are available.

3.7 Exercises

3.7.1 Exercise 1.1 — Hello Affinity

Write a script that logs the application name, version, and platform.

Expected output:

```
Product: Affinity Designer
Version: 3.x.x
Platform: macOS
```

Hint: Look at `app.productShortName`, `app.shortVersion`, `app.platformName`.

3.7.2 Exercise 1.2 — Desktop path

Write a script that logs the Desktop path and confirms it is a non-empty string.

3.7.3 Exercise 1.3 — Module explorer

require several different modules and log what each exports (use `console.log(Object.keys(...))`). Get familiar with what's available before you need it.

3.8 SDK modules used in this lesson

- `/application` — app singleton

3.9 See also

- Lesson 02 — App & Document
- `examples/` — runnable scripts for this lesson

3.10 Examples

3.10.1 Application info

```
'use strict';

// Example 01a — Application info
// Lesson 01: Environment Setup
//
// Run this first. If it logs cleanly, your environment is working.

const { app } = require('/application');

console.log('=== Affinity Application Info ===');
console.log('Product: ', app.productShortName);
console.log('Full name: ', app.productFullName);
console.log('Version: ', app.shortVersion);
console.log('Platform: ', app.platformName);
console.log('Desktop: ', app.userDesktopPath);
console.log('Build kind: ', app.buildKind);
console.log('Suite: ', app.suiteFullName);
```

Note
console.log() is the only output. Scripts don't return values to the UI. If nothing appears in the console, the script either errored silently or has no log statements.

Note
Script storage. Library scripts are stored in a single binary file (`scripts.propcol`), not as individual `.js` files. You cannot read them back with `fs` at runtime. There is no Save to Library button in the Scripts panel — library saving is only available via the MCP server. This will change in the future.

3.10.2 Module explorer

```
'use strict';

// Example 01b — Module explorer
// Lesson 01: Environment Setup
//
// Inspect what each core SDK module exports.
// A useful reference when you can't remember where something lives.

const modules = [
  '/application',
  '/document',
  '/commands',
  '/colours',
  '/fills',
  '/linestyle',
  '/geometry',
  '/shapes',
  '/dialog',
  '/units',
];

for (const m of modules) {
  try {
    const exports = require(m);
    const keys = Object.keys(exports);
    console.log(`\n${m} (${keys.length} exports):`);
    console.log(' ', keys.join(', '));
  } catch (e) {
    console.log(`\n${m} — failed to load:`, e.message);
  }
}
```

Lesson 02 — Application & Document Model

4

Every useful Affinity script begins the same way: get the current document, inspect it, and decide what to do with it. This lesson covers the full document model — from the app singleton at the top, down through spreads, pages, and properties like DPI, format, and title.

The document is the root of everything. Before you can move a node, change a colour, or export a file, you need a valid document reference. New SDK developers often encounter a few key pitfalls: the difference between `Document.current` and `Document.all`, the spread/page distinction and the crucial selection-vs-spread ordering rule. Mastering these fundamentals ensures your scripts behave predictably across all documents.

4.1 What this lesson covers

- The app singleton as entry point
- `Document.current` vs `Document.all`
- Document properties: title, path, format, DPI, dimensions
- The spread/page distinction
- Sync vs async document operations
- The critical selection-vs-spread ordering rule

4.2 The document hierarchy — JS API vs node tree

It is important to distinguish between the JavaScript API objects and the actual node tree that lives inside Affinity.

JS API objects:

```
app
|
Document (doc)
|— rootNode
```

```

├─ currentSpread
├─ selection
└─ history

```

```

Node tree (inside Affinity):
  DocumentNode (parent: null)
    └─ SpreadNode(s)
      └─ Layers / children

```

```

Mapping:
  doc.rootNode      → DocumentNode
  doc.currentSpread → SpreadNode (child of DocumentNode)

```

(The Document object itself is not a node — `doc.isDocumentNode` is undefined.)

The Document object (`doc`) is the **JS API entry point** — it holds properties like `currentSpread`, `selection`, `history`, and `rootNode`, but it is **not itself a node** (`doc.isDocumentNode` returns undefined).

The **node tree** starts at `rootNode`, a `DocumentNode` (`rootNode.isDocumentNode === true`). It has `parent: null` — it is the true root. Every node in the document, no matter how deeply nested, is a descendant of this `DocumentNode`. It has exactly one type of direct child: `SpreadNode` objects — one per spread in the document.

That is essentially all it exposes. It has no fill, no transform, no visibility — it is purely a container.

The root node exists to give tree traversal a consistent entry point. Every node, including spreads themselves, is a child of something. The `DocumentNode` is the answer to “what is the parent of a spread?” — without it, the tree would have no single root and recursive walkers would need special-case handling for the top level.

```

// Snippet
const rootNode = doc.rootNode;
rootNode.spreadCount // same as doc.spreadCount
rootNode.pageCount   // same as doc.pageCount
rootNode.isDocumentNode // true

```

In practice you almost never use it directly. The SDK gives you more convenient access points that you will use instead. The only time you need `doc.rootNode` directly is when you want to write a recursive walker that starts from the absolute top of the tree and descends through every spread and every layer in one pass — the kind of thing lesson 03 covers.

4.3 Getting the current document

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;

if (!doc) {
  console.log('No document open.');
```

```

} else {
  console.log('Title:      ', doc.title);
  console.log('Path:       ', doc.path);
  console.log('Format:      ', doc.format);
  console.log('DPI:         ', doc.dpi);
  console.log('Width (px):   ', doc.widthPixels);
  console.log('Height (px):  ', doc.heightPixels);
  console.log('Spreads:     ', doc.spreadCount);
  console.log('Pages:       ', doc.pageCount);
  console.log('Needs save:   ', doc.needsSaving);

```

```
console.log('Read only: ', doc.isReadOnly);
}
```

4.4 All open documents

Use `Document.all` to get an array of every document currently open. This is useful for batch operations or multi-document workflows.

```
'use strict';
const { Document } = require('/document');

const docs = Document.all;
console.log(`${docs.length} document(s) open`);

for (const d of docs) {
  console.log(d.title, '-', d.path);
}
```

Note

Use `.title` not `.name` — `.name` is undefined on Document objects. The SDK test suite uses `.title`.

4.5 Spreads vs pages

- A **spread** is a physical canvas unit. In most documents, one spread = one page.
- In a facing-pages publication, one spread = two pages side by side.
- `doc.spreads` — iterable collection of `SpreadNode` objects
- `doc.pageCount` — total number of pages across all spreads
- `doc.spreadCount` — number of spreads

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  for (const spread of doc.spreads) {
    console.log('Spread:', spread);
  }

  // Shortcuts
  const firstSpread = doc.spreads.first;
  const currentSpread = doc.currentSpread;
}
```

4.6 The selection / currentSpread ordering rule

This is the **most important gotcha in the entire SDK**.

```
// Snippet
// WRONG — selection is lost before you read it
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));
const sel = doc.selection; // always empty!

// CORRECT — capture selection BEFORE changing spread
const sel = doc.selection;
const nodes = sel.nodes; // extract everything you need
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));
```

`setCurrentSpread` **clears the selection**. Always read `doc.selection` and extract all needed data before executing that command.

4.7 Document state flags

Property	Meaning
<code>doc.needsSaving</code>	Has unsaved changes
<code>doc.isDirty</code>	Same as <code>needsSaving</code>
<code>doc.isReadOnly</code>	Cannot be edited

Property	Meaning
doc.mustSaveAs	New doc — must be saved to a path first
doc.isEmbedded	Placed inside another document

4.8 Creating a new document

```
'use strict';
const { Document, NewDocumentOptions } = require('/document');

const opts = NewDocumentOptions.createDefault();
opts.width = 1920;
opts.height = 1080;
opts.dpi = 96;

const newDoc = Document.create(opts);
console.log('Created:', newDoc.title);
```

4.9 Async document operations

For long-running work, use doc.promises:

```
'use strict';
const { DocumentPromises } = require('/document');
const { app } = require('/application');

async function main() {
  const doc = await DocumentPromises.getCurrent();
  if (!doc) return;

  // save the file on the desktop
  const desktop = app.userDesktopPath;
  await doc.saveAs(`${desktop}/output.af`);
  console.log('Saved.');
```

}

main();

4.10 Document properties

The DocumentPreset class lists all built-in templates. NewDocumentOptions lets you customise every detail of a new document, including its drawing scale.

```
'use strict';
const { Document, DocumentPreset, NewDocumentOptions } = require('/document');
const { DrawingScale } = require('/drawingscale');

// Enumerate all built-in document presets
const presets = DocumentPreset.all;
console.log(`${presets.length} presets available`);
for (const p of presets.slice(0, 5)) {
  console.log(` ${p.name} - ${p.width}x${p.height} @ ${p.dpi}dpi`);
}

// Create a document from a preset
const a4Preset = presets.find(p => p.name === 'A4');
if (a4Preset) {
  const doc = Document.createFromPreset(a4Preset, false); // false = portrait
  console.log('Created:', doc.title);
}

// Create with full options including drawing scale
const opts = NewDocumentOptions.createDefault();
opts.width = 297; // mm equivalent in document units
opts.height = 210; // A4 landscape
```

```

opts.dpi      = 300;
opts.useDrawingScale = true;
opts.drawingScale = DrawingScale.createFromIntegers(1, 50, true); // 1:50
scale

'use strict';
const { Document } = require('/document');

// doc.getFontNames() - all fonts referenced in the document
const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  const names = doc.getFontNames();
  console.log('Document uses ${names.length} font(s):');
  for (const name of names) console.log(' ', name);
}

```

4.10.1 Drawing scale

Affinity supports architectural and engineering drawing scales (e.g. 1:50, 1:100). Create them with `createFromIntegers`, or fetch the built-in defaults for a given unit system.

```

'use strict';
const { DrawingScale } = require('/drawingscale');

// Common scales - createFromIntegers(left, right, simplifyFraction)
const scale = DrawingScale.createFromIntegers(1, 100, true); // 1:100
console.log(scale.toString(false, true, false)); // "1 : 100"
console.log('Left:', scale.leftValue, 'Right:', scale.rightValue);

// Identity (no scale, 1:1)
const identity = DrawingScale.identity;

// Available defaults for a unit - pass a raw integer, NOT UnitType.*
// UnitType.Metres etc. return undefined from JS and cause INVALID_ARGS.
// Use these verified integer constants instead:
const DrawingScaleUnit = {
  Pixels:      5, // screen/pixel defaults
  Mixed:       6, // pixels + imperial
  Points:      7, // typographic points
  Millimetres: 8, // mm (includes km map scales)
  Centimetres: 9, // cm (includes km map scales)
  Picas:       10, // picas
  Inches:      11, // inches (includes ft)
  Yards:       12, // yards
  Feet:        13, // feet
  Kilometres:  14, // kilometres
  Metres:      15, // metres
};

const mmDefaults = DrawingScale.getDefaults(DrawingScaleUnit.Millimetres);
console.log(`${mmDefaults.length} mm presets:');
for (const s of mmDefaults) {
  console.log(' ', s.toString(false, true, false));
}

// Output:
// 1 : 10
// 1 : 25
// 1 : 40
// 1 : 50
// 1 : 100
// 1 : 200
// 1 : 500
// 1 mm : 1 cm
// 1 mm : 1 m
// 1 : 25000 km
// 1 : 50000 km

```

Note
Do not pass `UnitType.Millimetres` or any `UnitType.*` value to `getDefaults().UnitType`

is a C++ constructor — its named properties return undefined from JavaScript, which causes

`INVALID_ARGS`. Use the integer constants from the `DrawingScaleUnit` table above instead.

4.10.2 Document snapshots

Snapshots let you bookmark the document state — useful for before/after comparisons or checkpointing during long scripts. Create, list, and restore them via `DocumentCommand`.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Add a named snapshot
  doc.executeCommand(DocumentCommand.createAddDocumentSnapshot('Before
changes'));

  // List snapshots
  for (const snap of doc.snapshots) {
    console.log(snap.description, snap.format);
  }

  // Restore the first snapshot
  if (doc.snapshots.length > 0) {
doc.executeCommand(DocumentCommand.createRestoreDocumentSnapshot(doc.snapshots[0]));
  }
}
```

4.10.3 Spread & artboard document properties

Use `SpreadDocumentProperties` and `ArtboardDocumentProperties` to read and write per-artboard settings like margins, and to resize spreads and artboards relative to an anchor point.

```
'use strict';
const { Document } = require('/document');
const { ArtboardDocumentProperties, SpreadDocumentProperties } =
require('/documentproperties');

const doc = Document.current;
const spread = doc.currentSpread;
const artboard = spread.artboardInterface;

// Read / write artboard properties
const props = ArtboardDocumentProperties.create()
  .setUseMargin(true)
  .setMargin({ top: 20, left: 20, bottom: 20, right: 20 });

doc.setArtboardDocumentProperties(artboard, props);

// Resize spread about an anchor
doc.setSpreadSizeWithAnchor(spread, 800, 600, SpatialAnchor.TopLeft);

// Resize artboard about an anchor
doc.setArtboardSizeWithAnchor(artboard, 400, 300, SpatialAnchor.Centre);
```

4.10.4 Guide management

Guides are non-printing alignment lines — horizontal (true) or vertical (false) — positioned at a pixel coordinate. Add, move, remove, and recolour them through convenience methods on `doc`.

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (doc) {
  // Add guides
  doc.addGuide(true, 200); // horizontal guide at y=200px
}
```

```

doc.addGuide(false, 400); // vertical guide at x=400px

// Convenience methods
// doc.addHorizontalGuide(pixels) / doc.addVerticalGuide(pixels)
// doc.moveGuide(horizontal, index, newPixels)
// doc.removeGuide(horizontal, index)
// doc.setGuidesColour(colour)

console.log('Guides added.');
```

4.10.5 Document size and page count

Quick-access properties for common document dimensions. `sessionUuid` uniquely identifies this document instance within the Affinity session.

```

// Snippet
const doc = Document.current;
console.log('Width:', doc.widthPixels);
console.log('Height:', doc.heightPixels);
console.log('Page count:', doc.pageCount);
console.log('Spread count:', doc.spreadCount);
console.log('Session UUID:', doc.sessionUuid);
```

4.10.6 Document history

The `doc.history` object exposes undo/redo state, and `doc.undo()` / `doc.redo()` let you step through it without building a command.

```

// Snippet
const doc = Document.current;
const h = doc.history;

console.log('Can undo:', h.canUndo, '-', h.undoDescription);
console.log('Can redo:', h.canRedo, '-', h.redoDescription);
console.log('History size:', h.size);

// Undo/redo via document convenience methods
doc.undo();
doc.redo();
```

4.10.7 Undo and selection convenience methods

The Document class exposes many common operations directly, without needing to build a `DocumentCommand` manually:

```

// Snippet
const doc = Document.current;
const sel = doc.selection;

doc.undo(); // undo last action
doc.redo(); // redo
doc.selectAll(); // select all
doc.deleteSelection(sel); // delete selected nodes
doc.showSelection(sel); // make visible
doc.hideSelection(sel); // hide
doc.lockSelection(sel); // lock (non-editable)
doc.unlockSelection(sel); // unlock
doc.setLayerDescription('My layer', sel); // rename layer
doc.convertToCurves(sel); // convert shape to curves
doc.rasteriseObjects(sel); // rasterise vector to pixels
doc.flatten(); // flatten all layers
doc.mergeVisible(); // merge visible layers

// Additional convenience methods
doc.setText('Hello', sel); // replace text
doc.formatText(delta, sel); // apply text formatting
doc.insertGlyph(0x2665, sel); // insert ♥ glyph
doc.setDashPhase(0, sel); // dash phase offset
doc.setBalancedDashes(true, sel); // balanced dashes at corners
```

```
// Saving and closing
doc.save(); // save in place (fails if
mustSaveAs)
doc.saveAs('/path/to/file.af'); // save to new path
doc.saveAsPackage('/path/to/pkg',
  PackageResourcesPolicy.CopyLinks); // package with linked assets
doc.close(); // close the document
```

PackageResourcesPolicy controls how linked resources are handled when packaging. Import it from /document:

```
// Snippet
const { PackageResourcesPolicy } = require('/document');
// Values: CopyLinks (copies linked assets into the package)
```

4.11 Key gotchas

mustSaveAs on new documents — a freshly created document has no path. Calling `doc.save()` on it will fail. Save to the Desktop with `doc.saveAs(desktop + '/file.af')`.

doc.close() and doc.closeAsync() may not be implemented in some SDK builds. Prefer DocumentPromises for async save/close workflows.

① **Note**
setCurrentSpread clears selection — always read selection data before calling this command.

4.12 Exercises

4.12.1 Exercise 2.1 — Document dashboard

Write a script that logs a full summary of the current document: title, path, format, DPI, pixel dimensions, spread count, page count, and whether it needs saving.

4.12.2 Exercise 2.2 — Multi-doc lister

Write a script that lists all open documents and, for each, logs the title and whether it's dirty.

4.12.3 Exercise 2.3 — Spread walker

Iterate every spread in the document. For each spread, log its index and artboard count. (Hint: check `spread.artboardCount`.)

4.12.4 Exercise 2.4 — New document

Create a new document programmatically (1920×1080, 96 DPI), log its title, then close it.

4.13 SDK modules used in this lesson

- /document — Document, NewDocumentOptions, DocumentPreset, DocumentPromises, PackageResourcesPolicy
- /application — app
- /drawingscale — DrawingScale
- /commands — DocumentCommand

4.14 See also

- Lesson 01 — Environment Setup
- Lesson 03 — Node Model
- examples/

4.15 Examples

4.15.1 Document dashboard

```
'use strict';

// Example 02a — Document dashboard
// Lesson 02: App & Document
//
```

```
// Logs a full summary of the current open document.

const { Document } = require('/document');
const { app } = require('/application');

console.log('=== Application ===');
console.log('Product:', app.productShortName, app.shortVersion);
console.log('Desktop:', app.userDesktopPath);

const doc = Document.current;

if (!doc) {
  console.log('\nNo document is open. Open a file and run again.');
```

```
} else {
  console.log('\n=== Document ===');
  console.log('Title:      ', doc.title);
  console.log('Path:        ', doc.path || '(unsaved)');
  console.log('Format:       ', doc.format);
  console.log('DPI:          ', doc.dpi);
  console.log('View DPI:     ', doc.viewdpi);
  console.log('Width (px):   ', doc.widthPixels);
  console.log('Height (px):  ', doc.heightPixels);
  console.log('Spreads:      ', doc.spreadCount);
  console.log('Pages:        ', doc.pageCount);
  console.log('Snapshots:    ', doc.snapshotCount);
  console.log('Needs save:   ', doc.needsSaving);
  console.log('Read only:    ', doc.isReadOnly);
  console.log('Must save as:', doc.mustSaveAs);
  console.log('Embedded:     ', doc.isEmbedded);
  console.log('UUID:         ', doc.persistentUuid);
}
```

4.15.2 Spread walker

```
'use strict';

// Example 02b – Spread walker
// Lesson 02: App & Document
//
// Iterates every spread and logs its properties.

const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```
} else {
  console.log(`Document: ${doc.title}`);
  console.log(`Total spreads: ${doc.spreadCount}, pages: ${doc.pageCount}`);
}

let i = 0;
for (const spread of doc.spreads) {
  i++;
  console.log(`\nSpread ${i}:`);
  console.log('  Artboards:', spread.artboardCount);
  console.log('  Has artboards:', spread.artboardCount > 0);
}

console.log(`\nCurrent spread:`, doc.currentSpread);
}
```


Lesson 03 — Node Model & Tree Traversal

5

In Affinity, every object you see in the canvas — every shape, text frame, image, group, and curve — is a node in a tree. This lesson teaches you how that tree is structured, how to walk it, and how to identify what kind of node you are looking at.

Understanding the node tree is the foundation of every non-trivial script. Whether you want to rename all layers, find every text frame, apply an effect to shapes only, or build a custom export pipeline, you need to be able to traverse the tree and identify nodes by type. Without this lesson, most of the rest of the tutorial would be inaccessible.

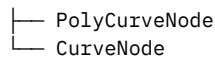
5.1 What this lesson covers

- The node tree structure
- Node type guards (`isShapeNode`, `isTextNode`, etc.)
- Traversing layers recursively
- Working with selections
- `NodeChildType` — Main vs Mask child lists

5.2 The node tree

Every object in an Affinity document is a node. The tree looks like this:

```
DocumentNode (rootNode)
├── SpreadNode
│   ├── GroupNode
│   │   ├── ShapeNode
│   │   └── TextNode
│   ├── RasterNode
│   └── ImageNode
```



Nodes expose typed **interfaces** as properties:

- `node.visibilityInterface` — show/hide
- `node.baseBoxInterface` — bounding box (x, y, width, height)
- `node.transformInterface` — transform matrix (NOT bounding box)
- `node.descriptionInterface` — layer name
- `node.editabilityInterface` — lock/unlock
- `node.blendModeInterface` — blend mode, opacity

5.3 Traversing layers

`doc.layers` iterates top-level nodes across every spread. To descend into group children, use `doc.layers.all`.

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No doc.'); }
else {
  // Top-level layers across all spreads (does not descend into groups)
  for (const node of doc.layers) {
    console.log(node.descriptionInterface?.description ?? '(unnamed)');
  }

  // ALL nodes recursively across all spreads
  for (const node of doc.layers.all) {
    console.log(node);
  }
}

```

① **Note**
`doc.layers` covers all spreads but not group children. It returns top-level nodes across every spread. Use `doc.layers.all` to descend into groups. Use `doc.spreads` when you need to know which spread a node belongs to.

5.4 Multi-spread traversal

When you need per-spread context, iterate `doc.spreads` directly and use `getNodeChildren` to access nodes within a specific spread.

```

'use strict';
const { Document } = require('/document');
const Nodes = require('/nodes');

const doc = Document.current;
if (!doc) return;

let spreadIndex = 0;
for (const spread of doc.spreads) {
  spreadIndex++;
  console.log(`\n— Spread ${spreadIndex} —`);

  for (const node of Nodes.getNodeChildren(spread.handle,
Nodes.NodeChildType.Main, false)) {
    const name = node.descriptionInterface?.description ?? '(unnamed)';
    console.log(' ', name);
  }
}

```

5.5 Node type guards

Use these boolean properties to identify node types before accessing type-specific APIs:

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {

```

```

for (const node of doc.layers.all) {
  if (node.isShapeNode) console.log('Shape: ',
node.descriptionInterface?.description);
  if (node.isTextNode) console.log('Text: ',
node.descriptionInterface?.description);
  if (node.isRasterNode) console.log('Raster: ',
node.descriptionInterface?.description);
  if (node.isImageNode) console.log('Image: ',
node.descriptionInterface?.description);
  if (node.isPolyCurveNode) console.log('PolyCurve:',
node.descriptionInterface?.description);
  if (node.isGroupNode) console.log('Group: ',
node.descriptionInterface?.description);
  if (node.isContainerNode) console.log('Container:',
node.descriptionInterface?.description);
  if (node.isCurveNode) console.log('Curve: ',
node.descriptionInterface?.description);
}
}

```

5.6 Working with the selection

`doc.selection.nodes` gives you the current selection as an iterable. Access each node's interfaces to read its name, bounding box, etc.

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) return;

const sel = doc.selection;
const nodes = sel.nodes;

if (nodes.isEmpty) {
  console.log('Nothing selected.');
```

```

} else {
  console.log(`${nodes.length} node(s) selected: `);
  for (const node of nodes) {
    const name = node.descriptionInterface?.description ?? '(unnamed)';
    const bbox = node.baseBoxInterface?.baseBox; // NOT
transformInterface.baseBox
    console.log(` ${name} — x:${bbox?.x} y:${bbox?.y} w:${bbox?.width} h:
${bbox?.height}`);
  }
}

```

5.7 Node interfaces reference

Interface	Property on node	What it gives you
Visibility	<code>visibilityInterface</code>	<code>.isVisible</code> , <code>.visibility</code>
Transform	<code>transformInterface</code>	<code>.transform</code> (matrix), <code>.unconstrainedTransform</code>
Bounding box	<code>baseBoxInterface</code>	<code>.baseBox</code> (x, y, width, height) — NOT on <code>transformInterface</code>
Spread bounding box	<code>spreadBaseBox</code> (convenience)	<code>.spreadBaseBox</code> — equivalent to <code>getSpreadBaseBox(true)</code>
Description	<code>descriptionInterface</code>	<code>.description</code> (layer name)

Interface	Property on node	What it gives you
Editability	editabilityInterface	.isEditable
Blend mode	blendModeInterface	.blendMode, .opacity
Tag	tagInterface	.tagColour
Artboard	artboardInterface	artboard-specific properties

5.8 NodeChildType

Nodes can have two child lists:

Type	Used for
NodeChildType.Main	Normal layer children
NodeChildType.Mask	Mask layers attached to a node

Most traversal only needs Main.

5.9 ContainerNode — named layer groups

ContainerNode is a named group type distinct from the anonymous Group Node. It appears in the Layers panel as a coloured folder and supports a fill, stroke, and blend mode of its own. The key difference is that ContainerNode Definition.create(name) accepts a display name upfront, whereas Group Node has no programmatic factory.

```
'use strict';
const { Document } = require('/document');
const { ContainerNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Create a named container (coloured layer group)
  const def = ContainerNodeDefinition.create('My Group');
  const builder = AddChildNodesCommandBuilder.create();
  builder.addContainerNode(def);
  const cmd = builder.createCommand(true);
  doc.executeCommand(cmd);

  const container = cmd.newNodes[0];
  console.log('Container created:',
    container.descriptionInterface.description);
  console.log('Is container:', container.isContainerNode);
}
```

Use ContainerNodeDefinition.createDefault() when you want an unnamed container and will set def.userDescription separately.

5.10 Node convenience methods

Every Node exposes shorthand methods that wrap the underlying commands:

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.first;
  if (!node) { console.log('Select a node first.')}
  else {
```

```

// Rename
node.userDescription = 'Renamed layer';

// Lock / unlock (sets editability)
node.lock(); // equivalent to
DocumentCommand.createSetEditable(sel, false)
node.unlock();

// Duplicate - returns the new node
const copy = node.duplicate();
console.log('Duplicate created:',
copy?.descriptionInterface?.description);

// Delete
// node.delete(); // removes the node (wraps doc.deleteSelection)
}
}

```

`node.duplicate()` applies no transform by default, so the copy lands on top of the original. Pass a Transform as the first argument to offset it.

`doc.addNode(nodeDefinition, targetNode)` is a shorthand that wraps `AddChildNodesCommandBuilder` for single-node additions without the full builder setup:

```

'use strict';
const { Document } = require('/document');
const { ContainerNodeDefinition } = require('/nodes');

const doc = Document.current;
if (doc) {
  const def = ContainerNodeDefinition.create('Quick group');
  doc.addNode(def); // adds to the current spread, top of stack
}

```

Null-safe interface access — not every node implements every interface. Always use optional chaining (`node.descriptionInterface?.description`) or check for null first.

Read `doc.selection` before `setCurrentSpread` — switching spreads clears the selection.

5.11 Per-node tag metadata

The SDK lets you store arbitrary string key-value pairs on any node via `DocumentCommand`. These persist inside the `.af` file, making them useful for metadata workflows, asset management systems, and script-to-script communication.

`node.tagInterface` exposes methods for reading tags back at runtime: `hasKey(key)`, `getValueForKey(key)`, `hasPredefinedKey(key)`, and `getValueForPredefinedKey(key)`.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { PredefinedTagKey } = require('/taginterface');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select a node first.')}
else {
  const sel = doc.selection;
  const node = sel.nodes.first;
}

```

Note
`doc.layers` = all spreads, top-level only. It covers every spread but does not descend into groups. Use `doc.spreads` when you need to know which spread a node belongs to.

Note
`doc.layers.all` = all spreads, recursive. Descends into groups and returns every node at any depth across all spreads.

```
// Write a custom tag (persists in the file, adds to undo stack)
doc.executeCommand(
  DocumentCommand.createSetTagValueForKey(sel, 'asset-id', 'IMG-0042')
);
doc.executeCommand(
  DocumentCommand.createSetTagValueForKey(sel, 'approved', 'true')
);

// Write a predefined tag (ScriptLabel, AltTextSource, AltTextTag, etc.)
doc.executeCommand(
  DocumentCommand.createSetTagValueForPredefinedKey(
    sel, PredefinedTagKey.ScriptLabel, 'hero-image')
);

// Read tags back via tagInterface
const tagIface = node.tagInterface;
if (tagIface) {
  console.log('Has asset-id:', tagIface.hasKey('asset-id'));
  console.log('asset-id value:', tagIface.getValueForKey('asset-id'));
  console.log('ScriptLabel:',
tagIface.getValueForPredefinedKey(PredefinedTagKey.ScriptLabel));
  console.log('Is decoration:', tagIface.isMarkAsDecoration);
}
}
```

5.11.1 Use cases

- **Asset management:** tag layers with IDs, status, or project codes that survive export/reimport.
- **Script handoff:** script A tags nodes, external tools read the .af XML to process them.
- **Conditional logic:** write tags that future script runs can detect via other means.
- **Audit trail:** write timestamps or version numbers when a layer was last processed.

5.12 Exercises

5.12.1 Exercise 3.1 — Layer tree printer

Write a script that prints a tree of all layer names, indented by depth, with the node type shown in brackets.

Expected output:

```
Background (ShapeNode)
  Sky (RasterNode)
  Mountains (GroupNode)
    Peak left (PolyCurveNode)
    Peak right (PolyCurveNode)
Logo (GroupNode)
  Text (TextNode)
```

5.12.2 Exercise 3.2 — Selection inspector

Write a script that logs the name, type, bounding box, and visibility of every currently selected node.

5.12.3 Exercise 3.3 — Type counter

Write a script that counts how many of each node type exist in the document and logs a summary:

```
ShapeNode:    12
TextNode:     3
RasterNode:   1
GroupNode:    5
```

5.12.4 Exercise 3.4 — Multi-spread walker

For a multi-page document, walk every spread and log how many top-level nodes each spread contains.

5.13 SDK modules used in this lesson

- /document — Document
- /nodes — NodeChildType, ContainerNodeDefinition, getNodeChildren, getNodeChildrenRecursive
- /commands — AddChildNodesCommandBuilder
- /selections — Selection

5.14 See also

- Lesson 02 — App & Document
- Lesson 04 — Command Pattern
- examples/

5.15 Examples**5.15.1 Layer tree printer**

```
'use strict';

// Example 03a — Layer tree printer
// Lesson 03: Node Model
//
// Prints every layer in the document as an indented tree.
// Works across all spreads.

const { Document } = require('/document');
const Nodes = require('/nodes');

const nodeType = (node) => {
  if (node.isShapeNode)      return 'Shape';
  if (node.isTextNode)      return 'Text';
  if (node.isRasterNode)    return 'Raster';
  if (node.isImageNode)     return 'Image';
  if (node.isPolyCurveNode) return 'PolyCurve';
  if (node.isGroupNode)     return 'Group';
  if (node.isCurveNode)     return 'Curve';
  return 'Node';
};

const printChildren = (parentHandle, depth) => {
  const indent = ' '.repeat(depth);
  for (const node of Nodes.getNodeChildren(parentHandle,
Nodes.NodeChildType.Main, false)) {
    const name      = node.descriptionInterface?.description ?? '(unnamed)';
    const type      = nodeType(node);
    const visible   = node.visibilityInterface?.isVisible !== false ? '' : '[hidden]';
    console.log(`${indent}${name} (${type})${visible}`);
    printChildren(node.handle, depth + 1);
  }
};

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  console.log(`Layer tree: ${doc.title}\n`);

  let i = 0;
  for (const spread of doc.spreads) {
    i++;
    console.log(`— Spread ${i} —`);
    printChildren(spread.handle, 1);
    console.log('');
  }
}
```

```

    }
  }

```

5.15.2 Selection inspector

```

'use strict';

// Example 03b - Selection inspector
// Lesson 03: Node Model
//
// Logs full details of every selected node.

const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const sel = doc.selection;
  const nodes = sel.nodes;

  if (nodes.isEmpty) {
    console.log('Nothing selected. Select some objects and run again.')}
  else {
    console.log(`${nodes.length} node(s) selected:\n`);

    let i = 0;
    for (const node of nodes) {
      i++;
      const name = node.descriptionInterface?.description ??
        '(unnamed)';
      const visible = node.visibilityInterface?.isVisible;
      const bbox = node.baseBoxInterface?.baseBox; // NOT
        transformInterface.baseBox

      console.log(`[${i}] ${name}`);
      if (node.isShapeNode) console.log('    Type: Shape');
      else if (node.isTextNode) console.log('    Type: Text');
      else if (node.isRasterNode) console.log('    Type: Raster');
      else if (node.isImageNode) console.log('    Type: Image');
      else if (node.isGroupNode) console.log('    Type: Group');
      else if (node.isPolyCurveNode) console.log('    Type: PolyCurve');
      else console.log('    Type: (other)');

      console.log('    Visible:', visible);
      if (bbox) {
        console.log(`    Bounds:  x=${Math.round(bbox.x)}
y=${Math.round(bbox.y)} w=${Math.round(bbox.width)}
h=${Math.round(bbox.height)}`);
      }
    }
  }
}

```

Phase 2 — Core API

PART



Phase 2 – Core API

Phase 1 taught you how to read a document — how to navigate its structure, identify nodes, and understand what is there. Phase 2 teaches you how to change it. This is where scripts stop being inspectors and start being tools.

The four lessons in this phase cover the operations you will reach for in almost every practical script: executing commands, applying visual properties, moving and transforming objects, and creating new vector content. They are ordered deliberately — commands come first because everything else depends on them. You cannot apply a fill, move a node, or create a shape without going through the command system.

What makes this phase demanding is that the SDK rarely lets you do the obvious thing. You cannot set a fill by assigning a property. You cannot move a node by changing its coordinates. You cannot make a line dashed without also changing its type. Each lesson in this phase documents not just the correct patterns but the specific reasons the intuitive approaches fail — because those are the mistakes every developer makes, and knowing them in advance saves significant debugging time.

By the time you complete this phase you will have a working mental model of how Affinity separates reading state from writing state, and why that separation exists. That model will make everything in Phases 3 and 4 considerably easier to reason about.

6.1 The core loop

Every useful script follows this pattern:

1. Get doc → `Document.current`
2. Get nodes → `doc.selection` or `doc.layers.all`
3. Build commands → `DocumentCommand.create*()`
4. Batch into compound → `CompoundCommandBuilder`
5. Execute → `doc.executeCommand()`

Internalize this loop and every later API just becomes a matter of knowing which `DocumentCommand.create*()` factory to call.

6.2 Lessons

Lesson	Topic	Key skill you'll gain
04 — Command pattern	<code>DocumentCommand</code> , <code>CompoundCommandBuilder</code> , <code>undo</code>	Making any change to the document safely
05 — Fills & strokes	<code>FillDescriptor</code> , gradients, <code>LineStyle</code> , dash patterns	Controlling all visual properties of any node
06 — Geometry	<code>Transform</code> , <code>CurveBuilder</code> , bounding boxes, <code>PolyCurve</code>	Moving nodes and drawing vector paths in code

6.3 What to expect

By the end of Phase 2 you should be able to write a script that:

- Applies any fill — solid colour, gradient, or none — to any selected node
- Sets stroke weight, colour, cap style, join style, and dash pattern correctly
- Moves, scales, and rotates nodes using `Transform`
- Draws arbitrary vector paths using `CurveBuilder` and adds them to the document
- Reads a node's bounding box and uses it to compute positions
- Batches multiple changes into a single undoable compound command

You will also understand three of the most counterintuitive SDK behaviours: why `LineStyle.createDefault()` produces a style with zero weight, why setting `style.weight` directly has no effect, and why dash patterns require `LineStyle.Dash` to be set explicitly before they render. These are not bugs — they reflect the SDK's design — but knowing them before you encounter them will save you considerable frustration.

Lesson 04 – The Command Pattern

The most important rule in the Affinity SDK is this: you cannot change anything by setting a property directly. Every modification — moving a node, changing a colour, replacing text — must go through a command that you build and execute. This lesson explains why that is, and how to use the command pattern effectively.

This design gives you undo/redo for free. Every command you execute appears in the undo stack exactly as if the user had made the change manually. It also means scripts are safe — you can build a compound command, inspect it, and only commit it when you are ready. Getting fluent with `DocumentCommand`, `CompoundCommandBuilder`, and `executeCommand` is what separates scripts that feel like first-class Affinity operations from ones that feel like hacks.

7.1 What this lesson covers

- Why every change goes through `executeCommand()`
- `DocumentCommand` static factories
- `CompoundCommandBuilder` — batching changes into one undo step
- `AddChildNodesCommandBuilder` — creating new nodes
- Preview mode
- Async command execution

7.2 The golden rule

This is how Affinity gives you undo/redo for free. Every user action in the UI is a command. Your scripts are no different.

```
// Snippet
// WRONG — direct mutation (won't work, and has no undo)
```

Note
Never mutate nodes directly.
Always build a command
and execute it.

```
node.fill = someColour;

// CORRECT — command pattern (options object first)
const cmd = DocumentCommand.createSetBrushFill({ fillDescriptor },
selection);
doc.executeCommand(cmd);
```

7.3 Basic command execution

Build a command with a DocumentCommand factory, then pass it to doc.executeCommand().

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { FillDescriptor } = require('/fills');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No doc.');
```

```
}
else {
  const sel = doc.selection;
  if (sel.nodes.isEmpty) { console.log('Select something first.');
```

```
}
else {
  const colour = RGBA8(220, 50, 50, 255); // red
  const fill = FillDescriptor.createSolid(colour);
  const cmd = DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel);
  doc.executeCommand(cmd);
  console.log('Fill applied.');
```

```
}
}
```

7.4 CompoundCommandBuilder — one undo step for many changes

When your script makes several changes, wrap them in a compound command so the user can undo everything in one step.

```
'use strict';
const { Document } = require('/document');
const { CompoundCommandBuilder, DocumentCommand } = require('/commands');
const { FillDescriptor } = require('/fills');
const { RGBA8 } = require('/colours');

const doc = Document.current;
const nodes = doc.layers.all.filter(n => n.isShapeNode);

const builder = CompoundCommandBuilder.create();

for (const node of nodes) {
  const sel = doc.selection; // or build a selection from node
  const colour = RGBA8(0, 120, 200, 255);
  const fill = FillDescriptor.createSolid(colour);
  const cmd = DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel);
  builder.addCommand(cmd);
}

// Execute all changes as a single undoable action
doc.executeCommand(builder.createCommand());
console.log('All shapes recoloured. One undo step.');
```

7.5 AddChildNodesCommandBuilder — creating new nodes

To add objects to the document, use AddChildNodesCommandBuilder:

```
'use strict';
const { Document } = require('/document');
const { AddChildNodesCommandBuilder } = require('/commands');
const Nodes = require('/nodes');
```

```

const doc = Document.current;

// Get a node definition from the SDK (shapes, text, etc.)
// See Lesson 06 (Geometry) — basic shapes section — for ShapeRectangle,
ShapeEllipse, etc.
const builder = AddChildNodesCommandBuilder.create();
builder.addNode(someNodeDefinition);

const cmd = builder.createCommand(
  true, // andSelect — select the new node after
  adding
  Nodes.NodeChildType.Main // which child list to add to
);
doc.executeCommand(cmd);

// Access the newly created node
const newNode = cmd.newNodes[0];
console.log('Created:', newNode.descriptionInterface?.description);

```

7.6 Reading command results

After executing an add-node command, you can access the newly created nodes:

```

// Snippet — assumes cmd has already been executed via
doc.executeCommand(cmd)
doc.executeCommand(cmd);
const newNodes = cmd.newNodes;
for (const n of newNodes) {
  console.log('New node:', n.descriptionInterface?.description);
}

```

7.7 Preview mode

Preview mode lets you see the effect of a command without committing it to the undo stack. Useful for interactive tools.

```

// Snippet — assumes cmd and doc are already defined
// Apply as preview (not committed to history)
doc.executeCommand(cmd, true);

// Clear all previews
doc.executeCommand(DocumentCommand.createClearPreviews());

```

7.8 Async command execution

For long-running scripts or sequences:

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { FillDescriptor } = require('/fills');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.'); }
else {
  const sel = doc.selection;
  const fill = FillDescriptor.createSolid(RGBA8(80, 140, 220, 255));
  const cmd = DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel);

  doc.executeCommandAsync(cmd, (errorCode) => {
    if (errorCode) {
      console.log('Command failed:', errorCode);
    } else {
      console.log('Done.'); }
  })
}

```

```
    });
}
```

Or with `doc.promises` for promise-based workflows:

```
// Snippet – assumes doc is already defined (e.g. const doc =
Document.current)
async function applyAndSave() {
  const docP = doc.promises;
  await docP.executeCommand(cmd);
  // Check mustSaveAs before saving – new documents need saveAs(path)
  first
  if (doc.mustSaveAs) {
    const { app } = require('/application');
    await docP.saveAs(app.userDesktopPath + '/' + doc.title + '.af');
  } else {
    await docP.save();
  }
  console.log('Saved.');
```

7.9 Undo and redo

`doc.undo()` and `doc.redo()` step through the history. `doc.canUndo` and `doc.undoDescription` let you inspect the undo stack first.

```
// Snippet – assumes doc is already defined
// Programmatic undo/redo
doc.undo();
doc.redo();

// Check what's available
console.log('Can undo:', doc.canUndo);
console.log('Can redo:', doc.canRedo);
console.log('Next undo:', doc.undoDescription);
```

7.10 Common DocumentCommand factories

Factory	What it does
<code>createSetBrushFill({ fillDescriptor }, sel)</code>	Set interior fill
<code>createSetPenFill({ fillDescriptor }, sel)</code>	Set stroke fill
<code>createSetLineStyle(sel, style)</code>	Set stroke style
<code>createSetVisibility(sel, bool)</code>	Show/hide
<code>createSetEditable(sel, bool)</code>	Lock/unlock – selection first
<code>createDeleteSelection(sel)</code>	Delete nodes
<code>createSelectAll(layerOnly)</code>	Select all
<code>createSetDescription(sel, str)</code>	Rename layer
<code>createSetBlendMode(sel, mode)</code>	Blend mode
<code>createTransform(sel, xf)</code>	Move/scale/rotate
<code>createSetText(sel, text)</code>	Set text content
<code>createSetCurrentSpread(spread)</code>	Change active spread

7.11 Key gotchas

Compound commands = one undo step. If you call `executeCommand()` in a loop, each call creates a separate undo entry. Use `CompoundCommandBuilder` to group them.

cmd.newNodes only works after execution. Access the result after `doc.executeCommand(cmd)` returns.

7.12 Exercises

7.12.1 Exercise 4.1 – Batch recolour

Select several shapes. Write a script that sets each one to a different colour using a compound command (one undo step for all changes).

7.12.2 Exercise 4.2 – Undo explorer

Write a script that logs the current undo stack size, makes a visible change, then logs the undo description before calling `doc.undo()`.

7.12.3 Exercise 4.3 – Visibility toggle

Write a script that hides all currently visible nodes in the selection, or shows all hidden ones – toggling their state – in a single compound command.

7.13 SDK modules used in this lesson

- /commands — DocumentCommand, CompoundCommandBuilder, AddChildNodesCommandBuilder
- /document — Document
- /nodes — NodeChildType

7.14 See also

- Lesson 03 — Node Model
- Lesson 05 — Fills & Strokes
- examples/

7.15 Examples

7.15.1 Batch recolour (compound command)

```
'use strict';

// Example 04a – Batch recolour (compound command)
// Lesson 04: Command Pattern
//
// Recolours all selected shape nodes to a palette of colours.
// All changes are grouped into one undo step.

const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Selection } = require('/selections');
const { FillDescriptor } = require('/fills');
const { RGBA8 } = require('/colours');

const PALETTE = [
  RGBA8(220, 60, 60, 255), // red
  RGBA8(60, 160, 220, 255), // blue
  RGBA8(80, 200, 120, 255), // green
  RGBA8(240, 160, 40, 255), // amber
  RGBA8(160, 80, 220, 255), // purple
  RGBA8(40, 190, 190, 255), // teal
];

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const shapes = doc.selection.nodes.filter(n => n.isShapeNode);

  if (shapes.isEmpty) {
    console.log('Select some shape nodes and run again.')}
  else {
    const compound = CompoundCommandBuilder.create();
```

Note
`setCurrentSpread` clears selection. See Lesson 02.

```
    let i = 0;

    for (const node of shapes) {
      const colour = PALETTE[i % PALETTE.length];
      const fill = FillDescriptor.createSolid(colour);
      const sel = Selection.create(doc, node);
      const cmd = DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel);
      compound.addCommand(cmd);
      i++;
    }

    doc.executeCommand(compound.createCommand());
    console.log(`Recoloured ${i} shape(s) — one undo step.`);
  }
}
```

Lesson 05 — Fills & Strokes

Fills and strokes are the visual language of vector design. This lesson covers how the SDK represents colour, how to construct any fill type — solid, gradient, or none — and how to apply stroke styles including weight, caps, joins, and dash patterns to selected nodes.

Almost every script that does something visible will need to set a fill or a stroke. The SDK's approach is more structured than you might expect: fills are wrapped in a `FillDescriptor`, strokes are described by a `LineStyle`, and partial updates require a `LineStyleMask`. Understanding this structure means you can apply exactly the visual properties you want without accidentally overwriting ones you meant to keep.

8.1 What this lesson covers

- Creating colours with `RGBA8()`
- `FillDescriptor` — the wrapper for any fill type
- Solid, gradient, hatch, and bitmap fills
- Line styles — weight, cap, join, dash patterns
- `LineStyleMask` — partial style updates
- Brush fill (interior) vs pen fill (stroke) vs transparency fill
- Setting opacity and blend mode
- Convenience fill methods on doc directly

8.2 Colours

Create colours with the `RGBA8(r, g, b, a)` convenience function — values are 0–255 integers.

```
'use strict';  
const { RGBA8, Colour } = require('/colours');
```

```
// RGBA8(red, green, blue, alpha) – values 0–255
const red    = RGBA8(220, 50, 50, 255);
const blue   = RGBA8(40, 120, 220, 255);
const semiBlack = RGBA8(0, 0, 0, 128);    // 50% opacity
const clear   = RGBA8(0, 0, 0, 0);        // fully transparent

// Alternative – same result
const red2 = Colour.createRGBA8({ r: 220, g: 50, b: 50, alpha: 255 });
```

Note
Never use `Colour.fromRGBA()` – it does not exist in the SDK. Use `RGBA8()` instead.

8.3 FillDescriptor

Wrap a fill type in `FillDescriptor` so it can be passed to command factories like `createSetBrushFill`.

```
'use strict';
const { FillDescriptor, SolidFill } = require('/fills');
const { RGBA8 } = require('/colours');

// Solid colour fill
const colour = RGBA8(200, 80, 40, 255);
const fill = FillDescriptor.createSolid(colour);

// No fill (transparent)
const none = FillDescriptor.createNone();
```

8.4 Gradient fills

Create a `GradientFill` from its default black→white state, then switch gradient shape with `cloneWithNewGradientFillType(type)`. Affinity supports four gradient types:

Integer	Type	Description
0	Linear	Straight line from stop to stop
1	Radial	Circular radiation from centre
2	Conical	Cone sweeping around a point
3	Elliptical	Ellipse radiation from centre

```
'use strict';
const { GradientFill, FillDescriptor } = require('/fills');
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;

  // Start from the default gradient (black → white, linear)
  const baseGf = GradientFill.createDefault();

  // Apply a linear gradient (type 0)
  // Options object { fillDescriptor } replaces flat first arg
  const linearFd = FillDescriptor.create(baseGf);
  doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor: linearFd }, sel));
  console.log('Linear gradient applied.');
```

```
// Switch to radial (type 1) – same gradient, different shape
const radialGf = baseGf.cloneWithNewGradientFillType(1);
const radialFd = FillDescriptor.create(radialGf);
doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
radialFd }, sel));
console.log('Radial gradient applied.');
```

```
// Conical (type 2)
const conicalGf = baseGf.cloneWithNewGradientFillType(2);
const conicalFd = FillDescriptor.create(conicalGf);
doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
conicalFd }, sel));
console.log('Conical gradient applied.');
```

8.4.1 Reading gradient stop data

Inspect the gradient stops — their positions, midpoints, and colours — via `GradientFill.gradient`.

```
'use strict';
const { GradientFill } = require('/fills');
```

```
const gf = GradientFill.createDefault();
const grad = gf.gradient;
```

```
console.log('Stop count:', grad.stopCount);
```

```
for (let i = 0; i < grad.stopCount; i++) {
  const stop = grad.getStop(i);
  console.log('Stop ${i}: position=${stop.position}
midpoint=${stop.midpoint}');
```

The SDK exposes `GradientFill.createDefault()` for a basic black-to-white gradient, and `cloneWithNewGradientFillType(type)` to switch the shape. Fine-grained stop colour modification via `Gradient.getStop(i).colour` is readable but setting stop colours programmatically requires passing a pre-built Gradient object to `GradientFill.create(gradient, type)`, which in turn requires the `colours.js` Gradient class stop-building API.

8.5 Applying fills to selected nodes

Use `createSetBrushFill` for interior fill and `createSetPenFill` for stroke fill — both take an options object as the first argument.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { FillDescriptor } = require('/fills');
const { RGBA8 } = require('/colours');
```

```
const doc = Document.current;
if (!doc || doc.selection.nodes.isEmpty) {
  console.log('Select something first.');
```

```
const sel = doc.selection;
const colour = RGBA8(255, 140, 0, 255); // orange
const fill = FillDescriptor.createSolid(colour);
```

```
// Brush fill = interior fill (options object first)
doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel));
```

```
// Pen fill = stroke fill (use a different colour)
const strokeColour = RGBA8(40, 40, 40, 255);
const strokeFill = FillDescriptor.createSolid(strokeColour);
```

Note
Gradient colours are controlled interactively in the Affinity UI gradient editor.

```
doc.executeCommand(DocumentCommand.createSetPenFill({ fillDescriptor:
strokeFill }, sel));
}
```

8.6 Three fill channels

Channel	Command	What it controls
Brush	createSetBrushFill	Interior / body fill
Pen	createSetPenFill	Stroke / outline fill
Transparency	createSetTrans- parencyFill	Transparency mask fill

8.7 Line styles

Always clone the node's current line style via `lineStyleInterface` rather than using `LineStyle.createDefault()`.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { LineStyleMask } = require('/linestyle');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something
first.')}
else {
  const sel = doc.selection;
  const node = sel.nodes.first;

  // Clone the node's current line style – do NOT use
  LineStyle.createDefault()
  // because its weight is 0 and the setter is silently ignored
  const currentStyle =
node.lineStyleInterface?.lineStyleDescriptor?.lineStyle;
  if (currentStyle) {
    const style = currentStyle.clone();
    style.weight = 3 * (doc.dpi / 72); // 3pt in document pixels
    doc.executeCommand(DocumentCommand.createSetLineStyle(sel, style,
      { lineStyleMask: LineStyleMask.Weight }));
    console.log('Line weight set to 3pt.')}
  }
}
```

`createDefault()` returns a style with `weight = 0`, and the `.weight` setter is silently ignored — setting it has no effect. Cloning the existing style from `node.lineStyleInterface.lineStyleDescriptor.lineStyle` preserves the current weight and allows you to modify it.

ⓘ **Note**
Always clone the node's current line style, never use `LineStyle.createDefault()` for weight.

8.7.1 Converting points to pixels

Affinity works in 96dpi pixels internally. If you want a weight in points (like in the UI):

```
// Snippet – assumes doc is already defined (e.g. const doc =
Document.current)
const weightInPts = 2;
const weightInPx = weightInPts * (doc.dpi / 72);
style.weight = weightInPx;
```

Or use the `doc` shorthand which handles the pts-to-pixels conversion automatically:

```
'use strict';
const { Document } = require('/document');
```

```
const { LineStyleMask } = require('/linestyle');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  // doc.setLineWeightPts converts points to document pixels automatically
  doc.setLineWeightPts(2, doc.selection);
  console.log('Weight set to 2pt.')}
}
```

8.8 Line caps and joins

Clone the current style, set .cap and .join to the desired integer value, and apply with LineStyleMask.Cap / LineStyleMask.Join.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { LineStyle, LineStyleMask } = require('/linestyle');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;
  const compound = CompoundCommandBuilder.create();

  // Set round caps
  const capStyle = LineStyle.createDefault();
  // Cap values: 0 = Butt, 1 = Square, 2 = Round
  capStyle.cap = 2;
  compound.addCommand(DocumentCommand.createSetLineStyle(sel, capStyle,
    { lineStyleMask: LineStyleMask.Cap }));

  // Set round joins
  const joinStyle = LineStyle.createDefault();
  // Join values: 0/1 = Miter, 2 = Round, 3 = Bevel
  joinStyle.join = 2;
  compound.addCommand(DocumentCommand.createSetLineStyle(sel, joinStyle,
    { lineStyleMask: LineStyleMask.Join }));

  doc.executeCommand(compound.createCommand());
  console.log('Round caps and joins applied.')}
}
```

8.9 Dash patterns

For dashes to be visible you must set **three things**: the line type to Dash, a non-zero weight, and the dash pattern. Setting only DashPattern stores the values but Affinity continues to render a solid line. Apply all three in one command using LineStyleMask.All.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { LineStyleMask, LineType } = require('/linestyle');
const { FillDescriptor } = require('/fills');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;
  const compound = CompoundCommandBuilder.create();
```

```

// Set a visible stroke colour
compound.addCommand(DocumentCommand.createSetPenFill(
  { fillDescriptor: FillDescriptor.createSolid(RGBA8(30, 30, 30,
255)) }, sel));

// Clone current style – set weight, type AND dash together, apply with
All mask
const node = sel.nodes.first;
const currentStyle =
node.lineStyleInterface?.lineStyleDescriptor?.lineStyle;
if (currentStyle) {
  const dashStyle = currentStyle.clone();
  dashStyle.weight = 2 * (doc.dpi / 72); // 2pt in document
pixels
dashStyle.type = LineType.Dash; // REQUIRED –
renders as solid without this
dashStyle.dashPattern = [8, 4]; // 8px dash, 4px
gap
dashStyle.hasBalancedDashes = true;

  compound.addCommand(DocumentCommand.createSetLineStyle(sel, dashStyle,
    { lineStyleMask: LineStyleMask.All }));
}

doc.executeCommand(compound.createCommand());
console.log('Dashed stroke applied.');
```

ⓘ **Note**
LineStyleMask.Dash is required for dashes to appear.

Without it the pattern is stored but Affinity renders solid. Always use LineStyleMask.All when setting type, weight, and pattern together.

8.10 LineStyleMask — partial updates

Apply only specific parts of a line style, leaving the rest unchanged:

```

// Snippet – assumes doc, sel and style are already defined
// Only change the weight – cap, join, dash stay as they are
doc.setLineStyle(style, sel, { lineStyleMask: LineStyleMask.Weight });

// Available masks:
// LineStyleMask.Weight, .Cap, .Join, .DashPattern, .DashPhase,
// .BalancedDashes, .Type, .All
```

8.11 Stroke alignment

Control whether the stroke sits inside, outside, or centred on the path using `doc.setStrokeAlignment()`.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { StrokeAlignment, LineStyleMask } = require('/linestyle');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;
  const style = require('/linestyle').LineStyle.createDefault();

  // StrokeAlignment is in /linestyle – not in affinity:dom
  // Values: check StrokeAlignment enum entries
  console.log('StrokeAlignment values:', Object.keys(StrokeAlignment)
    .filter(k => ![ 'entries', 'values', 'keys' ].includes(k)));

  doc.setStrokeAlignment(StrokeAlignment.Centre, sel);
  console.log('Stroke alignment set to Centre.');
```

8.12 Opacity and blend mode

Set overall layer opacity or fill-specific opacity, and apply blend modes like Multiply or Screen.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
const sel = doc.selection;

// Layer opacity (0.0-1.0)
doc.executeCommand(DocumentCommand.createSetOpacity(sel, 0.5));

// Or use the doc convenience method:
doc.setOpacity(0.5, sel);

// Blend mode - BlendMode enum from affinity:blendmode
// doc.setBlendMode(blendMode, setPassthrough, selection)
// common values: Normal, Multiply, Screen, Overlay, SoftLight, HardLight,
// Darken, Lighten

// Fill-level opacity (separate from layer opacity)
doc.setBrushFillOpacity(0.7, sel); // interior fill opacity
doc.setLineFillOpacity(0.5, sel); // stroke fill opacity
```

8.13 Hatch fills

Hatch fills tile a repeating line pattern over the shape. Fetch a HatchPattern from HatchPattern.all, then build a HatchFill and wrap it in a FillDescriptor.

```
'use strict';
const { HatchFill, FillDescriptor } = require('/fills');
const { HatchPattern } = require('/hatch');
const { Colour } = require('/colours');
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
const sel = doc.selection;

// Get available hatch patterns
const patterns = HatchPattern.all;
console.log('Available patterns:', patterns.length);

if (patterns.length > 0) {
  const pattern = patterns[0];
  const penColor = new Colour(/* ... */);

  const hatch = HatchFill.create(
    pattern, // HatchPattern
    1, // units (1 = document units)
    penColor, // line colour
    null, // background colour (null = transparent)
    1.0 // line weight
  );

  const fill = FillDescriptor.create(hatch, false, null, null, false);
  doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel));
}
```

8.14 Bitmap fills

Load a raster image from disk with Bitmap.loadFromFile, then wrap it in a BitmapFill and apply as a fill pattern.

```

'use strict';
const { BitmapFill, FillDescriptor } = require('/fills');
const { Bitmap } = require('/rasterobject');
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { app } = require('/application');

const doc = Document.current;
const bitmap = Bitmap.loadFromFile(app.userDesktopPath + '/texture.png');

if (bitmap) {
  const bitmapFill = BitmapFill.create(bitmap, 0, 0, false);
  const fill = FillDescriptor.create(bitmapFill, true, null, null, false);
  doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor: fill }, doc.selection));
  console.log('Bitmap fill applied.');
```

8.15 Doc convenience fill methods

Document exposes shorthand methods that accept a Colour, SolidFill, or FillDescriptor directly — no need to call DocumentCommand.createSetBrushFill explicitly:

```

// Snippet
const doc = Document.current;
const sel = doc.selection;

// All three accept Colour, SolidFill, or FillDescriptor
doc.setBrushFillDescriptor(myColour, sel); // interior fill
doc.setPenFillDescriptor(myFillDescriptor, sel); // stroke fill
doc.setTransparencyFillDescriptor(myFill, sel); // transparency

// Line style convenience — no LineStyle object needed
doc.setLineWeight(4, sel); // weight in pixels
doc.setLineWeightPts(2, sel); // weight in points (auto-converts to pixels)
doc.setLineCap(LineCap.Round, sel);
doc.setLineJoin(LineJoin.Round, sel);
doc.setStrokeAlignment(StrokeAlignment.Inside, sel);

// Dash enhancements
doc.setDashPhase(0, sel); // dash phase offset
doc.setBalancedDashes(true, sel); // balanced dashes at corners
```

8.16 VectorNodeDefinition — multi-fill and multi-stroke

When creating vector nodes programmatically, VectorNodeDefinition supports stacking multiple fill descriptors and line descriptor pairs. This is how Affinity represents nodes that have more than one fill (e.g. a shape with both a solid fill and a gradient fill layered).

```

'use strict';
const { Document } = require('/document');
const { PolyCurveNodeDefinition, VectorNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder } = require('/commands');
const { FillDescriptor, SolidFill, GradientFill, NoFill } = require('/fills');
const { LineStyle, LineStyleDescriptor } = require('/linestyle');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (doc) {
  const def = PolyCurveNodeDefinition.createDefault();

  // --- Multiple brush fills ---
  const solid = FillDescriptor.createSolid(rgba8(200, 80, 40, 255));
```

```

    const gradient = FillDescriptor.create(GradientFill.createDefault(),
true);
    def.addBrushFillDescriptor(solid);    // fill slot 0
    def.addBrushFillDescriptor(gradient); // fill slot 1

    console.log('Fill count:', def.brushFillDescriptorCount); // 2
    def.currentBrushFillIndex = 0; // which fill is active when rendering

    // Replace or remove a fill slot
    // def.setBrushFillDescriptor(0, newFillDescriptor);
    // def.removeBrushFillDescriptor(1);

    // --- Multiple line (stroke) descriptor pairs ---
    const ls = LineStyle.createDefault();
    ls.weight = 2;
    const lsd = LineStyleDescriptor.createDefault(ls);
    const lineFill = FillDescriptor.createSolid(RGBA8(40, 40, 40, 255));
    def.addLineDescriptors(lineFill, lsd); // stroke slot 0

    console.log('Stroke count:', def.lineDescriptorsCount); // 1
}

```

For most scripts you will only use one fill and one stroke — `addBrushFillDescriptor` and `addLineDescriptors` with a single call each. The multi-fill API is primarily useful when reproducing a node's visual state exactly from an existing document.

8.17 Fluent node-definition builders

Node-definition property setters return the definition, enabling fluent chains:

```

// Snippet
const def = PolyCurveNodeDefinition.createDefault()
    .setUserDescription('My Shape')
    .setTransform(someTransform);

const adjDef =
    BrightnessContrastAdjustmentRasterNodeDefinition.create(params)
        .setParameters(params)
        .setUserDescription('Brightness fix')
        .setTransform(xf);

```

8.18 Colour accepted where FillDescriptor expected

`VectorNodeDefinition` fill/line methods accept a `Colour` directly — a solid fill is created automatically:

```

// Snippet
const def = PolyCurveNodeDefinition.createDefault();
const { RGBA8 } = require('/colours');

// Old: had to create FillDescriptor manually
// def.addBrushFillDescriptor(FillDescriptor.createSolid(RGBA8(200, 80,
40, 255)));

// Pass Colour directly
def.addBrushFillDescriptor(RGBA8(200, 80, 40, 255));
def.setBrushFillDescriptor(0, RGBA8(100, 180, 60, 255));

```

LineStyle.Dash is required for dashes to render. Setting `dashPattern` alone stores the values but Affinity continues to render a solid line. Always set `style.type = LineStyle.Dash` and apply with `LineStyleMask.All`.

createSetBrushFill and createSetPenFill use options objects. The first argument is `{ fillDescriptor, copyTransform, ... }`, not a flat colour/`FillDescriptor`. Use `createSetBrushFill({ fillDescriptor: myFill }, sel)`.

copyTransform option in createSetBrushFill — when true (default), gradient fill transforms are copied from the selection's bounding box. Set to false if you want the gradient anchored independently.

① **Note**
LineStyle.createDefault()
 has `weight = 0` and the setter is silently ignored.

8.19 Exercises

8.19.1 Exercise 5.1 — Solid fill batch

Select several shapes. Apply a different solid colour from a predefined palette to each one.

8.19.2 Exercise 5.2 — Stroke styler

Write a script that gives every selected node a 2pt round-capped dashed stroke, without changing their fill colours. Use `LineStyleMask` to target only the relevant properties.

8.19.3 Exercise 5.3 — Opacity fade

Apply decreasing opacity to each node in the selection (100%, 80%, 60%, 40%, 20%) using `setTransparencyFillDescriptor`.

8.20 SDK modules used in this lesson

- `/colours` — `Colour`, `RGBA8`
- `/fills` — `FillDescriptor`, `SolidFill`, `GradientFill`, `HatchFill`, `Bitmap Fill`, `NoFill`
- `/hatch` — `HatchPattern` (for hatch fills)
- `/linestyle` — `LineStyle`, `LineStyleMask`, `LineType`, `StrokeAlignment`
- `/commands` — `DocumentCommand`, `CompoundCommandBuilder`
- `/document` — `Document`

8.21 See also

- Lesson 04 — Command Pattern
- Lesson 06 — Geometry
- `examples/`

8.22 Examples

8.22.1 Fills and strokes

```
'use strict';

// Example 05a — Fills and strokes
// Lesson 05: Fills & Strokes
//
// Applies a solid fill and a visible dashed stroke to all selected nodes.
//
// KEY INSIGHTS:
// 1. Clone the node's CURRENT line style — LineStyle.createDefault() has
//    weight=0
//    and the weight setter is silently ignored.
// 2. Set style.type = LineType.Dash — without this the dash pattern is
//    stored
//    but Affinity renders a solid line.
// 3. Apply weight + type + dash in one command using LineStyleMask.All.

const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Selection } = require('/selections');
const { FillDescriptor } = require('/fills');
const { LineStyle, LineStyleMask, LineType } = require('/linestyle');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) {
```

```

    console.log('Select some objects and run again.');
```

```

} else {
    const compound = CompoundCommandBuilder.create();

    for (const node of doc.selection.nodes) {
        const nodeSel = Selection.create(doc, node);

        // 1. Solid orange fill (options object first)
        compound.addCommand(DocumentCommand.createSetBrushFill(
            { fillDescriptor: FillDescriptor.createSolid(rgba8(255, 140, 0,
255)) }, nodeSel));

        // 2. Dark stroke colour (options object first)
        compound.addCommand(DocumentCommand.createSetPenFill(
            { fillDescriptor: FillDescriptor.createSolid(rgba8(30, 30, 30,
255)) }, nodeSel));

        // 3. Stroke: clone current style, set weight + type + dash, apply all
        at once
        const currentStyle =
node.lineStyleInterface?.lineStyleDescriptor?.lineStyle;
        if (currentStyle) {
            const strokeStyle = currentStyle.clone();
            strokeStyle.weight          = 2 * (doc.dpi / 72); // 2pt in document
pixels
            strokeStyle.type            = LineType.Dash;      // REQUIRED for
dashes to show
            strokeStyle.dashPattern     = [6, 3];             // 6px dash, 3px
gap
            strokeStyle.hasBalancedDashes = true;

            compound.addCommand(DocumentCommand.createSetLineStyle(nodeSel,
strokeStyle,
                { lineStyleMask: LineStyleMask.All }));
        }
    }

    doc.executeCommand(compound.createCommand());
    console.log('Fill and dashed stroke applied.');
```

```

}
```


Lesson 06 — Geometry, Transforms & Curves

This lesson covers the geometric side of the SDK — how to move, scale, and rotate nodes using transforms, how to draw vector paths programmatically with `CurveBuilder`, and how to read a node's bounding box. It also covers the coordinate system all SDK measurements use.

Geometry operations unlock a whole class of scripts that go beyond simple property changes: grid layouts, pattern generators, custom shape creation, alignment tools, and anything that needs to know where things are on the canvas. The `Transform` class and `CurveBuilder` are the two workhorses here — once you are comfortable with them you can build almost any spatial manipulation imaginable.

9.1 What this lesson covers

- Adding basic shapes — rectangles, ellipses, polygons, stars, triangles
- Shape parameters — corner radius, number of sides, star inner radius
- `Transform` — moving, scaling, rotating nodes
- `CurveBuilder` — programmatically drawing vector paths
- `PolyCurve` — multi-subpath curves
- Unit conversions (document pixels vs points)
- Duplicating nodes in a grid layout

9.2 Coordinate system

The SDK works in **document pixels** at 96dpi. All coordinates and sizes are in this unit.

To convert from points (as shown in the UI):

```
// Snippet — assumes doc is already defined (e.g. const doc =  
Document.current)
```

```
const PTS_T0_PX = doc.dpi / 72;
const tenPtInPx = 10 * PTS_T0_PX;
```

9.3 Transforms — moving and resizing nodes

Build a Transform via static factories like `createTranslate`, then pass it to `DocumentCommand.createTransform()`.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Transform } = require('/geometry');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;

  // Pattern 1: instance method — createIdentity, then mutate
  const xf = Transform.createIdentity();
  xf.translate(100, 50); // translate(x, y) — mutates in place
  doc.executeCommand(DocumentCommand.createTransform(sel, xf));

  // Pattern 2: static constructor (most concise)
  const xf2 = Transform.createTranslate(100, 50);
  doc.executeCommand(DocumentCommand.createTransform(sel, xf2));

  console.log('Nodes translated.');
```

9.3.1 Transform properties and chainable methods

Access matrix components via `.xAxis`, `.yAxis`, `.origin`, and chain mutating calls like `.translate()`, `.rotate()`, `.scale()`.

```
// Snippet — assumes xf is a Transform
xf.xAxis; // Vector — first column of the matrix
xf.yAxis; // Vector — second column
xf.origin; // Point — translation component
xf.inverted; // Transform — non-mutating inverse

// Chainable mutating methods
xf.translate(50, 30)
  .rotate(Math.PI / 4)
  .scale(2, 1.5);

// Non-mutating variants return a new Transform
const scaled = xf.scaled(2); // new Transform, xf unchanged
const rotated = xf.rotated(Math.PI / 4);
const combined = xf.translated(50, 0);

// around(originX, originY) — transform about a point, then translate back.
// Replaces deprecated about().
xf.around(100, 100).rotate(Math.PI / 4);

// Compose with another transform
const a = Transform.createTranslate(100, 0);
const b = Transform.createRotate(Math.PI / 4);
const c = a.multiply(b); // a * b — new Transform
a.postmultiplyBy(b); // a = a * b — mutate a in place
a.premultiplyBy(b); // a = b * a — mutate a in place

// Static multiply
const d = Transform.multiply(a, b); // same as a.multiply(b)

// Decompose into components
```

```
const data = xf.decompose();
console.log(data.translation, data.rotation, data.scaling, data.shear);
```

9.4 Adding basic shapes

Shapes are created using static `.create()` factory methods from the `/shapes` module — never with `new`. Each shape type has its own class: `ShapeRectangle`, `ShapeEllipse`, `ShapePolygon`, `ShapeTriangle`, `ShapeStar`, and more.

The pattern is always the same: create the shape, wrap it in a `ShapeNodeDefinition`, set its bounding rectangle, then add it to the document.

```
'use strict';
const { Document } = require('/document');
const { ShapeNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder } = require('/commands');
const { ShapeRectangle, ShapeEllipse, ShapePolygon, ShapeTriangle,
ShapeStar } = require('/shapes');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  function addShape(shape, x, y, w, h) {
    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.setBoundingRectangle({ x, y, width: w, height: h });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    doc.executeCommand(builder.createCommand(true));
  }

  addShape(ShapeRectangle.create(), 100, 100, 200, 150); // rectangle
  addShape(ShapeEllipse.create(), 350, 100, 200, 150); // ellipse
  addShape(ShapePolygon.create(), 600, 100, 200, 200); // hexagon
  (default 6 sides)
  addShape(ShapeTriangle.create(), 100, 300, 200, 200); // triangle
  addShape(ShapeStar.create(), 350, 300, 200, 200); // star

  console.log('Shapes added.')}
}
```

9.5 Shape parameters

Once a shape is on the canvas, its parameters can be modified using `Document Command`. Select the shape first or use a per-node selection:

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else if (doc.selection.nodes.isEmpty) { console.log('Select a shape
first.')} }
else {
  const sel = doc.selection;

  // Corner radius on a rectangle (0 = sharp corners)
  doc.executeCommand(DocumentCommand.createSetShapeFloatParam(sel,
'CornerRadius', 20));

  // Number of sides on a polygon (integer param)
  doc.executeCommand(DocumentCommand.createSetShapeIntParam(sel,
'NumberOfPoints', 3));

  // Star inner radius — controls how pointy the star is (0.0–1.0)
  doc.executeCommand(DocumentCommand.createSetShapeFloatParam(sel,
'InnerRadius', 0.3));
}
```

Note
ShapeRectangle.create()
not new ShapeRectangle().
 All shape classes require a native handle — instantiating with `new` throws `Invalid handle`. Always use the static `.create()` factory method.

Common float params: CornerRadius, InnerRadius, SweepAngle, StartAngle
 Common int params: NumberOfPoints

9.6 Building curves with CurveBuilder

Use CurveBuilder to draw vector paths programmatically — beginXY(), lineToXY(), then createCurve().

```
'use strict';
const { Document } = require('/document');
const { CurveBuilder, PolyCurve } = require('/geometry');
const { AddChildNodesCommandBuilder } = require('/commands');
const { PolyCurveNodeDefinition } = require('/nodes');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Build a triangle
  const builder = CurveBuilder.create();
  builder.beginXY(300, 100); // NOT begin({x,y}) - use beginXY
  builder.lineToXY(500, 300); // NOT addLineXY - use lineToXY
  builder.lineToXY(100, 300);
  builder.close();

  const curve = builder.createCurve();
  const poly = PolyCurve.create();
  poly.addCurve(curve);

  // Use createDefault() not create() - create() requires arguments
  const nodeDef = PolyCurveNodeDefinition.createDefault();
  nodeDef.polyCurve = poly;

  const addBuilder = AddChildNodesCommandBuilder.create();
  addBuilder.addPolyCurveNode(nodeDef);
  doc.executeCommand(addBuilder.createCommand(true));
  console.log('Triangle added.')}
}
```

9.7 Bezier curves

For curved segments, use addBezierXY(cp1x, cp1y, cp2x, cp2y, endX, endY).

```
'use strict';
const { Document } = require('/document');
const { CurveBuilder, PolyCurve } = require('/geometry');
const { AddChildNodesCommandBuilder } = require('/commands');
const { PolyCurveNodeDefinition } = require('/nodes');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const builder = CurveBuilder.create();
  builder.beginXY(100, 200);
  builder.addBezierXY(
    150, 100, // control point 1
    250, 100, // control point 2
    300, 200 // end point
  );
  builder.close();

  const curve = builder.createCurve();
  const poly = PolyCurve.create();
  poly.addCurve(curve);

  const nodeDef = PolyCurveNodeDefinition.createDefault();
  nodeDef.polyCurve = poly;

  const addBuilder = AddChildNodesCommandBuilder.create();
  addBuilder.addPolyCurveNode(nodeDef);
```

```

doc.executeCommand(addBuilder.createCommand(true));
console.log('Bezier curve added.');
```

9.8 Grid duplicator

A common pattern — take a shape and tile it into a grid:

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Transform } = require('/geometry');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

9.9 Converting a shape to curves

Turn a parametric shape (rectangle, ellipse, etc.) into raw editable vector curves.

```

// Snippet
// Via DocumentCommand
doc.executeCommand(DocumentCommand.createConvertToCurves(sel));

// Or via doc convenience method
doc.convertToCurves(sel);
```

9.10 Rasterising objects

Convert vector layers into pixel rasters, optionally clipping to spread bounds.

```

// Snippet
// Convert vector to raster pixels
doc.rasteriseObjects(sel);

// rasteriseContentsOnly = true - rasterise contents but keep layer
structure
```

```
// clipToSpread = true - clip to spread bounds
doc.rasteriseObjects(sel, false, true);
```

9.11 Adding artboards

Use `doc.addRectangularArtboard(rect)` to add a new artboard at a given position and size.

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (doc) {
  // Add a rectangular artboard at a specific position
  doc.addRectangularArtboard({ x: 0, y: 0, width: 1920, height: 1080 });
  console.log('Artboard added.');
```

9.12 Knife cut and scissor cut

Split nodes along a curve (knife) or at a parameterised point (scissor).

```
// Snippet - assumes doc, sel, and cutCurve are already defined
// Knife cut along a polyCurve
doc.executeCommand(DocumentCommand.createKnifeCut(cutCurve, sel));

// Scissor - split a curve at a specific point
doc.executeCommand(DocumentCommand.createScissorCut(node, polyCurveIndex,
  curveIndex, true, 0.5));
```

9.13 Key gotchas

`node.baseBoxInterface?.baseBox` not `node.transformInterface?.baseBox`. `transformInterface.baseBox` returns undefined — the bounding box (x, y, width, height) lives on `baseBoxInterface.baseBox`. `transformInterface` provides the transform matrix, not the box.

`xf.translate(x, y)` not `xf.translateBy(x, y)`. `translateBy` does not exist — the correct instance method is `translate`. Prefer the static constructors: `Transform.createTranslate(x, y)`, `Transform.createRotate(angle)`, `Transform.createScale(x, y)`. Use `Transform.createIdentity()` to get a clean transform — `makeIdentity()` is safe but less idiomatic.

`builder.lineToXY` not `builder.addLineXY`. The method to draw a straight line segment is `lineToXY(x, y)`. Similarly, `beginXY(x, y)` not `begin({x, y})` — the XY variant takes two numbers, not an object.

`createSetCurves` replaces the entire polycurve. Build the full replacement, including any subpaths you want to keep.

Geometry types are proper wrappers. `Rectangle`, `Point`, `Vector`, `Size`, and similar geometry structs are returned as documented types — not anonymous objects. All expected properties (`.x`, `.y`, `.width`, `.height`, etc.) behave identically.

① **Note**
All coordinates are in document pixels at 96dpi. The UI shows points — convert with `pts * (doc.dpi / 72)`.

② **Note**
duplicateNodes: true in createTransform — without this, the transform moves the original; with it, it clones and moves the clone.

9.14 Exercises

9.14.1 Exercise 6.1 — Shape creator

Write a script that creates a regular hexagon at a given position using `Curve Builder`.

9.14.2 Exercise 6.2 — Grid tiler

Select one shape. Generate a 5×3 grid of copies with 10px gaps. Use a compound command.

9.14.3 Exercise 6.3 — Corner rounder

For every selected rectangle, set its corner radius to 12px using `setShapeFloatParam`.

9.14.4 Exercise 6.4 — Spiral

Use `CurveBuilder` and bezier segments to draw an approximate Archimedean spiral.

9.14.5 TransformBuilder

A fluent builder that accumulates transforms in reading order:

```
// Snippet
const { TransformBuilder } = require('/geometry');

const tb = new TransformBuilder();
tb.translateXY(100, 50) // move right 100, down 50
  .rotate(Math.PI / 4) // rotate 45°
  .scaleXY(2, 1.5);    // scale 2x horizontally, 1.5x vertically
const finalXf = tb.transform; // built transform
```

Each builder method appends to the accumulated transform. Use `.transform` to retrieve the result.

9.15 Combining bounding boxes

The `/geometry` module exports `unionRects` — combines two `Rectangle` objects into their union:

```
'use strict';
const { Document } = require('/document');
const { unionRects } = require('/geometry');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select at least one node.')}
else {
  let combined = null;
  for (const node of doc.selection.nodes) {
    const box = node.getSpreadBaseBox();
    combined = combined ? unionRects(combined, box) : box;
  }
  if (combined) {
    console.log('Combined bounds:',
      Math.round(combined.x), Math.round(combined.y),
      Math.round(combined.width), 'x', Math.round(combined.height));
  }
}
```

9.16 SDK modules used in this lesson

- `/shapes` — `ShapeRectangle`, `ShapeEllipse`, `ShapePolygon`, `ShapeTriangle`, `ShapeStar`, `ShapeType`, and all other shape classes
- `/geometry` — `Transform`, `TransformBuilder`, `CurveBuilder`, `PolyCurve`, `Rectangle`, `unionRects`, `CubicBezier`, `Vector`, `Point`, `Size`
- `/shapes` — shape definitions and parameter names
- `/commands` — `DocumentCommand`, `AddChildNodesCommandBuilder`, `CompoundCommandBuilder`
- `/nodes` — `PolyCurveNodeDefinition`

9.17 See also

- Lesson 05 — Fills & Strokes
- Lesson 07 — Text & Story
- examples/

9.18 Examples

9.18.1 Grid duplicator

```
'use strict';

// Example 06a – Grid duplicator
// Lesson 06: Geometry
//
// Takes the selected shape and creates a grid of copies.
// Select ONE shape before running.

const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Transform } = require('/geometry');

const COLS = 4;
const ROWS = 3;
const GAP = 16; // pixels between copies

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const sel = doc.selection;
  const node = sel.nodes.first;

  if (!node) {
    console.log('Select exactly one shape and run again.')}
  else {
    // baseBoxInterface.baseBox – NOT transformInterface.baseBox (that is
    // undefined)
    const bbox = node.baseBoxInterface?.baseBox;
    if (!bbox) { console.log('Could not get bounding box.')}
    else {
      const stepX = bbox.width + GAP;
      const stepY = bbox.height + GAP;

      const compound = CompoundCommandBuilder.create();
      let count = 0;

      for (let row = 0; row < ROWS; row++) {
        for (let col = 0; col < COLS; col++) {
          if (row === 0 && col === 0) continue; // skip original position

          const xf = Transform.createTranslate(col * stepX, row * stepY);

          compound.addCommand(
            DocumentCommand.createTransform(sel, xf, { duplicateNodes:
true })
          );
          count++;
        }
      }

      doc.executeCommand(compound.createCommand());
      console.log(`Grid created: ${COLS}x${ROWS} (${count} copies added).`);
    }
  }
}
```

9.18.2 Basic shapes

```
'use strict';

// Example 06b – Basic shapes
// Lesson 06: Geometry, Transforms & Curves
//
// Creates a rectangle, ellipse, polygon, triangle, and star on the
// current
// document using ShapeNodeDefinition and the shape factory methods.
```

```
//
// Key rule: always use ShapeRectangle.create() – never new
ShapeRectangle().
// The static factory provides the native handle required by the SDK.

const { Document } = require('/document');
const { ShapeNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder, DocumentCommand } = require('/
commands');
const {
  ShapeRectangle,
  ShapeEllipse,
  ShapePolygon,
  ShapeTriangle,
  ShapeStar
} = require('/shapes');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Helper – create and add a shape at a given position and size
  function addShape(shape, x, y, w, h, label) {
    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.setBoundingRectangle({ x, y, width: w, height: h });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    const cmd = builder.createCommand(true);
    doc.executeCommand(cmd);
    console.log(`${label} added at (${x}, ${y}) ${w}x${h}`);
    return cmd.newNodes?.[0];
  }

  const W = 180; // shape width
  const H = 150; // shape height
  const GAP = 20; // gap between shapes

  // Row 1 – basic shapes
  const rect = addShape(ShapeRectangle.create(), 100, 100,
W, H, 'Rectangle');
  const ellipse = addShape(ShapeEllipse.create(), 100 + W + GAP, 100,
W, H, 'Ellipse');
  const polygon = addShape(ShapePolygon.create(), 100 + (W+GAP)*2, 100,
W, H, 'Polygon');

  // Row 2
  const triangle = addShape(ShapeTriangle.create(), 100, 100 +
H + GAP, W, H, 'Triangle');
  const star = addShape(ShapeStar.create(), 100 + W + GAP, 100 + H
+ GAP, W, H, 'Star');

  // Customise the polygon to a triangle (3 sides) using shape params
  if (polygon) {
    const sel = doc.selection; // polygon is selected after being added
    doc.executeCommand(DocumentCommand.createSetShapeIntParam(sel,
'NumberOfPoints', 3));
    console.log('Polygon set to 3 sides');
  }

  // Add a rounded rectangle by setting corner radius on the rect
  if (rect) {
    // Re-add a fresh rectangle at a new position with rounded corners
    const rounded = addShape(ShapeRectangle.create(), 100 + (W+GAP)*2, 100
+ H + GAP, W, H, 'Rounded rectangle');
    if (rounded) {
      const sel = doc.selection;
      doc.executeCommand(DocumentCommand.createSetShapeFloatParam(sel,
'CornerRadius', 25));
    }
  }
}
```

```
        console.log('Corner radius set to 25px');  
    }  
}  
  
console.log('Done.');
```

Phase 3 — Tools

PART



Phase 3 — Tools

10

Phase 2 gave you the mechanical skills — commands, fills, geometry, curves. Phase 3 is where those skills become useful in real world scenarios. This phase covers the four areas that turn a working script into a finished tool: text automation, interactive dialogs, file export, and external communication.

These are the lessons most directly connected to the kinds of tasks designers actually need automating. Text operations — finding, replacing, styling, and generating content — are among the most requested scripting use cases in any design tool. Dialogs transform a hardcoded script into something configurable, where a colleague who has never opened a code editor can supply the inputs and get a result. Export pipelines replace the most time-consuming part of a production workflow with a single button press. And networking means scripts can connect to the wider world — pulling in data, pushing results to APIs, and integrating with the systems a team already uses.

Each lesson in this phase also introduces a subtlety that is easy to miss. The Story model is richer and more structured than a simple string API, and getting text operations wrong produces silent failures rather than errors. Dialogs require understanding the control lifecycle — controls are configured before the dialog is shown and read after it closes. Export has a matrix of area types, format presets, and result records that all need to be handled correctly. And networking requires both user permission and careful attention to the `req['do']()` syntax that trips up every developer who encounters it for the first time.

By the end of this phase your scripts will be capable of doing things that were previously only possible through manual work or complex external tooling.

10.1 Lessons

Lesson	Topic	Key skill you'll gain
07 — Text & story	Story model, storyRange, formatting, StoryBuilder	Reading, replacing, and styling text in any document
08 — Dialog UI	Dialog, controls, columns, groups, reactive layout	Building native dialogs that collect user input
09 — Export & files	FileExportOptions, FileExportArea, File, HttpRequest	Exporting assets, writing files, calling external APIs

10.2 What to expect

By the end of Phase 3 you should be able to write a script that:

- Reads and replaces text in any text frame using the Story API
- Applies character styles and formatting to a precise range of characters
- Presents a dialog with multiple controls and uses the values to drive the script
- Exports every artboard in a document as named PNG files with a JSON manifest
- Reads and writes text files on the Desktop
- Makes an HTTP GET or POST request to an external API

You will also understand why `si.storyRange` lives on the `storyInterface` directly and not on `si.story`, why dialog controls must be configured before `runModal()` is called and read after it returns, and why network requests require `req['do']()` rather than `req.do()`. These are the three most common sources of confusion in this phase — and knowing them in advance means you will recognise them immediately if you encounter them.

Lesson 07 — Text & Story Model

11

Text in Affinity is not a simple string. Every text node contains a Story — a structured sequence of characters with formatting attached, much like a miniature word processor embedded in the document. This lesson explains that model and shows you how to read text content, replace it, apply formatting, and build new text content from scratch.

Automating text is one of the most common scripting use cases: batch-renaming layers, applying consistent character styles across a document, generating text from data, or checking for missing fonts. All of that requires understanding the Story model. It is more complex than a plain string API, but that complexity is what makes it possible to apply formatting to a precise range of characters without affecting anything else.

The model has a structure that rewards learning upfront. A `TextNode` gives you access to a `storyInterface`, which in turn exposes a `storyRange` covering the entire text content. From that range you can read characters, extract sub-ranges, apply formatting, or build entirely new content using `StoryBuilder`. The boundary between these objects — particularly the fact that `storyRange` lives on the `storyInterface` directly and not on the `story` property — is the most common source of confusion in this lesson. Knowing that before you start means you will recognise the correct pattern immediately rather than discovering it through error messages.

There is also a subtlety in how replacement ranges are constructed. Text content in a Story is terminated by a hidden final character — a paragraph marker that is part of the story's structure and should not be included in a text selection. The `end - 1` convention exists to exclude it: when you construct a `TextSelection` that covers all the text in a frame, using `range.end - 1`

keeps the selection within the editable content and avoids including the story terminator. In practice the SDK may tolerate `range.end` without visible error, but the correct and safe form is always `range.end - 1`.

By the time you have worked through the examples here, you will be able to read text from any frame in a document, replace it with new content, apply character and paragraph formatting to a precise range, and generate structured text from scratch using `StoryBuilder`. These capabilities underpin a wide range of practical automation tasks — data-driven layout population, copy editing tools, consistency checks, and more.

11.1 What this lesson covers

- The Story model — how text is stored internally
- `StoryBuilder` — constructing text content
- `StoryDelta` — describing formatting changes
- `TextSelection` — targeting ranges of text
- Constructing text selection ranges correctly (the `end - 1` convention)
- Applying character styles by name
- Paragraph attributes

11.2 How text works in Affinity SDK

Text in Affinity is not a simple string. It is a **Story** — a structured sequence of glyphs with formatting attached. Think of it like a mini word processor document embedded inside each text frame.

```
TextNode
├─ storyInterface
│   └─ story
│       └─ storyRange { begin: 0, end: N }
│           └─ glyphs (characters + formatting)
```

To change text, you either:

1. **Replace the whole story** with `StoryBuilder`
2. **Apply formatting** to a range with `StoryDelta`

11.3 Reading text from a node

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No doc.'); }
else {
  for (const node of doc.layers.all) {
    if (!node.isTextNode) continue;

    const si = node.storyInterface;
    if (!si) continue;

    const range = si.storyRange;
    console.log('Text node:', node.descriptionInterface?.description);
    console.log('Range: begin=', range.begin, 'end=', range.end);
  }
}
```

11.4 Replacing all text in a frame

This is the most common operation — and has a critical gotcha.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');
const { StoryBuilder } = require('/storybuilder');

const doc = Document.current;
```

```

const sel = doc.selection;
const node = sel.nodes.filter(n => n.isTextNode).first;

if (!node) { console.log('Select a text frame.')}
else {
  // CRITICAL: set current spread before any text work
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  const si = node.storyInterface;
  const range = si.storyRange;

  // Build a text selection that covers all editable text.
  // end - 1 excludes the story terminator. To append instead of replace,
  // see the "Appending text" section below and example 07c.
  const textSel = TextSelection.create([
    { begin: range.begin,
      end: range.end - 1 // <-- CRITICAL: -1 excludes the story
terminator
    }]);

  const selection = Selection.create(doc, node);
  selection.addSubSelectionForNode(node, textSel);

  const cmd = DocumentCommand.createSetText(selection, 'New text
content');
  doc.executeCommand(cmd);
  console.log('Text replaced.');
```

11.5 Appending text to a frame

To append without overwriting existing content, use a zero-length Text Selection — one where begin and end are the same value. This is an insertion point rather than a range. Text passed to `createSetText` is inserted at that position, leaving everything before it untouched.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');

const doc = Document.current;
const node = doc.selection.nodes.filter(n => n.isTextNode).first;

if (!node) { console.log('Select a text frame.')}
else {
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  const si = node.storyInterface;
  const range = si.storyRange;

  // Insert at end - 1: just before the story terminator.
  // begin == end makes this an insertion point, not a selection.
  const insertAt = range.end - 1;
  const textSel = TextSelection.create([
    { begin: insertAt, end: insertAt }]);

  const sel = Selection.create(doc, node);
  sel.addSubSelectionForNode(node, textSel);

  doc.executeCommand(DocumentCommand.createSetText(sel, ' (draft)'));
  console.log('Suffix appended.');
```

The contrast with replacement is just in the TextSelection:

Note

The end - 1 convention.

The story's final character is a hidden paragraph marker that is part of the story's internal structure. Including it in your selection (`range.end`) may work in practice, but the correct form is always `range.end - 1` — it keeps the selection within the editable content and avoids touching the terminator. Use it consistently.

Intent	begin	end
Replace all	range.begin	range.end - 1
Append	range.end - 1	range.end - 1

11.6 The selection setup order matters

`Selection.createEmpty(doc)` causes `addSubSelectionForNode` to silently fail — always use `Selection.create(doc, node)` instead.

```
// Snippet
// WRONG — using Selection.createEmpty causes addSubSelectionForNode to
// silently fail
const selection = Selection.createEmpty(doc);
selection.addSubSelectionForNode(node, textSel); // silently does nothing

// CORRECT — use Selection.create(doc, node) as the base
const selection = Selection.create(doc, node);
selection.addSubSelectionForNode(node, textSel); // works
```

11.7 Applying character formatting (StoryDelta)

Build a `StoryDelta` with a factory like `createGlyphString`, then pass it to `DocumentCommand.createFormatText()` to apply formatting to a text selection.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');
const { StoryDelta } = require('/storydelta');
const { GlyphAttStringType } = require('/glyphatts');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const textNode = doc.selection.nodes.filter(n => n.isTextNode).first;
  if (!textNode) { console.log('Select a text node first.')}
  else {
    doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

    const si = textNode.storyInterface;
    const range = si.storyRange;
    const textSel = TextSelection.create([ { begin: range.begin, end:
range.end - 1 } ]);
    const sel = Selection.create(doc, textNode);
    sel.addSubSelectionForNode(textNode, textSel);

    // Apply a named character style to the text selection
    const delta =
StoryDelta.createGlyphString(GlyphAttStringType.StyleName, 'Heading
Bold');
    const cmd = DocumentCommand.createFormatText(sel, delta);
    doc.executeCommand(cmd);
    console.log('Style applied.')}
  }
}
```

11.8 StoryDelta convenience factories

`StoryDelta` provides higher-level factories for common formatting operations — no need to build raw glyph attribute strings:

```
'use strict';
const { Document } = require('/document');
const { StoryDelta } = require('/storydelta');
const { FontWeight } = require('/fonts');

const doc = Document.current;
```

```

if (!doc) { console.log('No document open.')} }
else {
  // Apply bold and italic to current text selection in one composite
  delta
  const makeBold = StoryDelta.createWeight(FontWeight.Bold);
  const makeItalic = StoryDelta.createItalic(true);
  const composite = StoryDelta.createComposite([makeBold, makeItalic]);

  // doc.formatText() applies to current selection without needing an
  explicit Selection object
  doc.formatText(composite);
  console.log('Bold italic applied.')}
}

```

11.9 StoryBuilder — building text from scratch

Use `StoryBuilder.create()` to construct styled text programmatically, then set the default style for artistic or frame text.

```

'use strict';
const { Document } = require('/document');
const { StoryBuilder } = require('/storybuilder');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  // For artistic text (single-line, not in a frame)
  const sb = StoryBuilder.create();
  sb.setToArtisticTextDefaultStyle(doc.dpi, doc.format); // INSTANCE
  // method, not static
  sb.addText('Hello, Affinity!');
  console.log('StoryBuilder ready — use with AddChildNodesCommandBuilder
  to add to doc.')}

  // For frame text use:
  // sb.setToFrameTextDefaultStyle(doc.dpi, doc.format);
}

```

Note
`doc.formatText(delta, selection?)` is a shorthand that applies the delta to any selection without manually building a Selection and Text Selection. Use it for quick formatting operations. Similarly, `doc.setText(text, selection)` replaces text in one call.

11.10 TableTextNode

`TableTextNode` is a grid-based text node. Cells are separated by `HardBreakType`. `TableCell` glyphs in the node's story.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');
const { TableTextNodeDefinition } = require('/nodes');
const { StoryDelta } = require('/storydelta');
const { GlyphAtDoubleType } = require('/glyphatts');
const { FontWeight } = require('/fonts');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  // Create a 3x3 table
  const box = { x: 100, y: 100, width: 300, height: 200 };
  const def = TableTextNodeDefinition.create(box, { width: 3, height:
  3 });
  doc.addNode(def);
  const table = doc.layers.first; // newly added table

  // Walk cells via HardBreakType
  const story = table.storyInterface.story;
  const len = story.length;
  const starts = [0];
  for (let i = 0; i < len; i++) {
    if (story.getHardBreakType(i)?.value === 2) { //
    HardBreakType.TableCell

```

```

        starts.push(i + 1);
    }
}

// Fill cells in REVERSE order so positions stay stable as text grows
const data = [['A1', 'B1', 'C1'], ['A2', 'B2', 'C2'], ['A3', 'B3', 'C3']];
for (let k = data.length - 1; k >= 0; k--) {
    for (let j = data[k].length - 1; j >= 0; j--) {
        const begin = starts[k * 3 + j];
        const end = (k * 3 + j + 1 < starts.length) ? starts[k * 3 + j + 1]
- 1 : len;
        const textSel = TextSelection.create([begin, end]);
        const sel = Selection.create(doc, table);
        sel.addSubSelectionForNode(table, textSel);
        doc.executeCommand(DocumentCommand.createSetText(sel, data[k][j]));
    }
}

// Bold the header row
const headerSel = TextSelection.create([begin: 0, end: starts[3]]);
const sel = Selection.create(doc, table);
sel.addSubSelectionForNode(table, headerSel);
const boldDelta = StoryDelta.createComposite([
    StoryDelta.createWeight(FontWeight.Bold),
    StoryDelta.createGlyphDouble(GlyphAttDoubleType.Height, 14 *
(doc.dpi / 72))
]);
doc.executeCommand(DocumentCommand.createFormatText(sel, boldDelta));
console.log('Table created and populated.');
```

① **Note**
TableTextNode
Definition.create(box, {width, height}) creates a grid. Cells are indexed left-to-right, top-to-bottom. Always fill cells in reverse order so earlier cell positions stay stable as the story grows.

11.11 GlyphAtts and ParagraphAtts — attribute wrappers

GlyphAtts and ParagraphAtts provide proper wrapper types for reading and building text attributes:

```

'use strict';
const { GlyphAtts, GlyphAttStringType, GlyphAttDoubleType } = require('./glyphatts');
const { ParagraphAtts } = require('./paragraphatts');
const { FontWeight } = require('./fonts');
const { Document } = require('./document');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isTextNode).first;

if (node) {
    // Build glyph attributes
    const glyphAtts = GlyphAtts.create();
    glyphAtts.setString(GlyphAttStringType.StyleName, 'Heading');
    glyphAtts.setDouble(GlyphAttDoubleType.Height, 24);

    // Build paragraph attributes
    const paraAtts = ParagraphAtts.create();
    paraAtts.setBoolean(ParagraphAttBooleanType.Caps, true); // all caps
    paraAtts.setDouble(ParagraphAttDoubleType.Leading, 1.4); // leading
    paraAtts.setInt32(ParagraphAttInt32Type.Justification, 1); // centre alignment

    // Read existing attributes from a story
    const si = node.storyInterface;
    const story = si.story;
    const glyphEnd = story.length - 1;
    const existing = story.getGlyphAtts(0, glyphEnd);
    console.log('Font loaded:',
existing.getDouble(GlyphAttDoubleType.Height));
}
```

11.12 Attribute runs on Story

Story can enumerate contiguous runs of formatting, making it easy to walk the styling of a story without inspecting every glyph:

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isTextNode).first;

if (node) {
  const story = node.storyInterface.story;

  // Enumerate all glyph attribute runs
  for (const run of story.glyphAttRuns) {
    console.log(`Glyphs ${run.begin}-${run.end}`, run.glyphAtts);
  }

  // Enumerate all paragraph attribute runs
  for (const run of story.paragraphAttRuns) {
    console.log(`Paragraphs ${run.begin}-${run.end}`, run.paragraphAtts);
  }

  // Combined glyph + paragraph runs
  for (const run of story.attRuns) {
    console.log(`Combined run ${run.begin}-${run.end}`, run);
  }

  // Partial range queries
  const partialRuns = story.getGlyphAttRunsFrom(5, 20);
  for (const run of partialRuns) {
    console.log(`Sub-run ${run.begin}-${run.end}`);
  }
}
```

Each run yields { begin, end, glyphAtts, paragraphAtts } so you can process homogeneous spans in one step.

11.13 Batch text update — rename all text nodes

Iterate all text nodes in the document and replace their content using a CompoundCommandBuilder for a single undo step.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Selection, TextSelection } = require('/selections');

const doc = Document.current;
const compound = CompoundCommandBuilder.create();

// Capture selection BEFORE setCurrentSpread
const spreadCmd =
  DocumentCommand.createSetCurrentSpread(doc.currentSpread);

let count = 0;
for (const node of doc.layers.all) {
  if (!node.isTextNode) continue;

  const si = node.storyInterface;
  if (!si) continue;
  const range = si.storyRange;

  const textSel = TextSelection.create([
    { begin: range.begin, end: range.end - 1 }
  ]);
  const nodeSel = Selection.create(doc, node);
  nodeSel.addSubSelectionForNode(node, textSel);

  compound.addCommand(DocumentCommand.createSetText(nodeSel, 'Updated text
```

```

    ${++count}')));
  }

  if (count > 0) {
    doc.executeCommand(spreadCmd);
    doc.executeCommand(compound.createCommand());
    console.log('Updated ${count} text node(s).');
  } else {
    console.log('No text nodes found.');
```

11.14 Key gotchas

end - 1 to stay within editable content. The final character in any story is a hidden paragraph marker. Including it in a TextSelection (range.end) may work without error in practice, but the correct and safe form is range.end - 1. Use it consistently rather than testing where the boundary actually bites.

setToArtisticTextDefaultStyle() is an instance method. Call it as sb.setToArtisticTextDefaultStyle(dpi, format) on a StoryBuilder instance, not as a static.

createSetCurrentSpread before text editing. Always set the spread before any text commands — even if you think it's already the current spread.

Fill cells in reverse order. When populating a TableTextNode, fill cells from the last cell to the first. Inserting text at earlier positions shifts later positions; reverse order avoids position drift.

ⓘ **Note**
Capture doc.selection BEFORE setCurrentSpread. See Lesson 02.

11.15 Exercises

11.15.1 Exercise 7.1 — Text reader

Write a script that finds every text node in the document and logs its layer name and current text content.

11.15.2 Exercise 7.2 — Suffix appender

For every selected text node, append (draft) to its text content. Use a zero-length TextSelection at range.end - 1. See 07c-append-text.js for a working implementation.

11.15.3 Exercise 7.3 — Style applier

Build a script that applies a named character style (e.g. “Bold”) to the full text of each selected text node.

11.15.4 Exercise 7.4 — Text census

Count how many characters total are across all text nodes in the document (use range.end - range.begin).

11.16 SDK modules used in this lesson

- /story — Story, StoryRange
- /storybuilder — StoryBuilder
- /storydelta — StoryDelta
- /storyinterface — StoryInterface
- /glyphatts — GlyphAtts, GlyphAttStringType, GlyphAttDoubleType
- /paragraphatts — ParagraphAtts, ParagraphAttBooleanType, ParagraphAttDoubleType, ParagraphAttInt32Type
- /selections — Selection, TextSelection
- /commands — DocumentCommand
- /nodes — TableTextNodeDefinition, FrameTextNodeDefinition, ArtTextNodeDefinition

11.17 See also

- Lesson 06 — Geometry
- Lesson 08 — Dialog UI
- examples/

11.18 Examples**11.18.1 Text reader**

```
'use strict';

// Example 07a - Text reader
// Lesson 07: Text & Story
//
// Finds every text node in the document and logs its content.

const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  let count = 0;

  for (const node of doc.layers.all) {
    if (!node.isTextNode) continue;

    const name = node.descriptionInterface?.description ?? '(unnamed)';
    const si = node.storyInterface;

    if (!si) {
      console.log(`[${name}] - no storyInterface`);
      continue;
    }

    const range = si.storyRange;
    const charCount = range.end - range.begin;

    console.log(`\n[${name}]`);
    console.log('  Chars:', charCount);
    console.log('  Range: begin=' + range.begin + ' end=' + range.end);
    count++;
  }

  if (count === 0) {
    console.log('No text nodes found in document.')}
  else {
    console.log(`\nTotal text nodes: ${count}`);
  }
}
```

11.18.2 Replace text in selected frame

```
'use strict';

// Example 07b - Replace text in selected frame
// Lesson 07: Text & Story
//
// Replaces ALL text in the selected text node.
// Demonstrates the critical end - 1 rule.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');

const NEW_TEXT = 'This text was set by the Affinity SDK.';

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
```

```

// Capture the node BEFORE touching spreads
const textNodes = doc.selection.nodes.filter(n => n.isTextNode);

if (textNodes.isEmpty) {
  console.log('Select a text frame and run again.');
```

```

} else {
  const node = textNodes.first;
  const name = node.descriptionInterface?.description ?? '(unnamed)';

  // Always set current spread before text operations
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  const si = node.storyInterface;
  const range = si.storyRange;

  // CORRECT: end - 1 excludes the story's hidden paragraph terminator.
  // range.end may work in practice but end - 1 is the safe, correct
  form.
  const textSel = TextSelection.create([
    begin: range.begin,
    end: range.end - 1
  ]);

  const sel = Selection.create(doc, node);
  sel.addSubSelectionForNode(node, textSel);

  doc.executeCommand(DocumentCommand.createSetText(sel, NEW_TEXT));
  console.log(`Replaced text in "${name}"`);
}
}

```

11.18.3 Append text to selected frame

```

'use strict';

// Example 07c – Append text to selected frame
// Lesson 07: Text & Story
//
// Appends a suffix to the existing text in the selected text node
// without disturbing the content already there.
//
// The key difference from replacing:
//   Replace – TextSelection covering begin to end - 1 (selects all
//   editable content)
//   Append – TextSelection with begin == end == end - 1 (zero-length,
//   at the end)
//
// A zero-length selection is an insertion point. createSetText at that
// position inserts text without overwriting anything before it.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');

const SUFFIX = ' (draft)';

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  const textNodes = doc.selection.nodes.filter(n => n.isTextNode);

  if (textNodes.isEmpty) {
    console.log('Select a text frame and run again.');
```

```

  } else {
    const node = textNodes.first;
    const name = node.descriptionInterface?.description ?? '(unnamed)';

    // Always set current spread before text operations

```

```

doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

const si    = node.storyInterface;
const range = si.storyRange;

// Insertion point: begin and end both set to range.end - 1.
// This places the cursor just before the story terminator – text
// is inserted here, leaving everything before it intact.
const insertAt = range.end - 1;
const textSel  = TextSelection.create([ { begin: insertAt, end:
insertAt }]);

const sel = Selection.create(doc, node);
sel.addSubSelectionForNode(node, textSel);

doc.executeCommand(DocumentCommand.createSetText(sel, SUFFIX));
console.log(`Appended "${SUFFIX}" to "${name}".`);
}
}

```

11.18.4 Data-driven table from JSON

```

'use strict';

// Example 07d – Data-driven table from JSON
// Lesson 07: Text & Story
//
// Builds an A4 table from a hardcoded dataset.
// Cells are filled in reverse order so earlier positions stay stable.
// A bold header row is applied via StoryDelta.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection, TextSelection } = require('/selections');
const { TableTextNodeDefinition } = require('/nodes');
const { StoryDelta } = require('/storydelta');
const { FontWeight } = require('/fonts');

const DATA = [
  ['Country', 'Capital', 'Population', 'Area km²'],
  ['France', 'Paris', '67M', '551_695'],
  ['Germany', 'Berlin', '83M', '357_022'],
  ['Japan', 'Tokyo', '125M', '377_975'],
  ['Brazil', 'Brasilia', '213M', '8_515_767'],
];

const COLS = DATA[0].length;
const ROWS = DATA.length;

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  const box = { x: 40, y: 40, width: 700, height: 40 * ROWS };
  const def = TableTextNodeDefinition.create(box, { width: COLS, height:
ROWS });
  doc.addNode(def);
  const table = doc.layers.first;

  // Walk cells via HardBreakType
  const story = table.storyInterface.story;
  const len = story.length;
  const starts = [0];
  for (let i = 0; i < len; i++) {
    const hb = story.getHardBreakType(i);
    if (hb && hb.value === 2) starts.push(i + 1);
  }

  // Fill cells in REVERSE order so positions stay stable
  for (let k = DATA.length - 1; k >= 0; k--) {

```

```

    for (let j = DATA[k].length - 1; j >= 0; j--) {
      const cellIdx = k * COLS + j;
      const begin = starts[cellIdx];
      const end = (cellIdx + 1 < starts.length) ? starts[cellIdx + 1] -
1 : len;
      const textSel = TextSelection.create([begin, end]);
      const sel = Selection.create(doc, table);
      sel.addSubSelectionForNode(table, textSel);
      doc.executeCommand(DocumentCommand.createSetText(sel, String(DATA[k]
[j]))));
    }
  }

  // Bold and enlarge the header row
  const headerSel = TextSelection.create([begin: 0, end:
starts[COLS] });
  const hSel = Selection.create(doc, table);
  hSel.addSubSelectionForNode(table, headerSel);
  const boldDelta = StoryDelta.createComposite([
    StoryDelta.createWeight(FontWeight.Bold),
  ]);
  doc.executeCommand(DocumentCommand.createFormatText(hSel, boldDelta));

  console.log(`${ROWS}x${COLS} table created and populated.`);
}

```

Lesson 08 — Dialog UI

12

Scripts become dramatically more useful when they can ask the user a question before running. This lesson covers Affinity's built-in dialog system — how to build modal dialogs with columns, groups, and controls, how to read the values the user enters, and how to use those values to drive the rest of your script.

A dialog is the difference between a script that always does the same thing and one that adapts to what the user needs. Instead of hardcoding a colour, you present a colour picker. Instead of a fixed export format, you offer a dropdown. The dialog API is self-contained — no HTML, no CSS, no web views — and produces native dialogs that look and feel like the rest of the application.

The dialog system has a lifecycle that matters: controls are configured before the dialog is shown, and values are read after it closes. Attempting to read a control value before calling `runModal()`, or configuring a control after it, produces either incorrect results or silent failures. Understanding the build-then-show-then-read model before writing your first dialog will save a significant amount of debugging.

The layout system is column-based. Controls live inside groups, groups live inside columns, and columns live inside the dialog. This hierarchy is stricter than a web layout, but it means dialogs render consistently without any CSS or measurement code on your part. Reactive behaviour — enabling one control based on the state of another — is handled through per-control or per-dialog value changed callbacks.

By the end of this lesson you will be able to build dialogs that collect text, numbers, colours, fills, strokes, fonts, and dropdown selections from the user, handle OK and Cancel correctly, and use the resulting values to parameterise the rest of your script.

12.1 What this lesson covers

- Creating modal dialogs with `Dialog.create()`
- The column → group → control hierarchy
- All available control types including `FillEditor`, `StrokeEditor`, and `Font Picker`
- Reading values after the user clicks OK
- Reactive controls — `onValueChangedHandler` (per-control) and `onControl ValueChangedHandler` (dialog-level)
- Live preview dialogs — the `while (runModal() == Ok)` loop, preview flag, and `clearPreviews()`
- Mixed units in `UnitValueEditor` — store in pixels, display in mm
- `UnitType.Number` for unitless integer editors
- Resizable dialogs, `findControl()`, and attaching controls to the dialog object
- `DialogResult` — handling OK vs Cancel
- `alert()` for simple user messages

12.2 The Dialog hierarchy

```
Dialog
├── Column (one or more)
│   └── Group (one or more, with optional label)
│       ├── Controls (CheckBox, ComboBox, TextBox, FillEditor, etc.)
│       └── ColumnStack (sub-layout inside a group)
│           ├── Column
│           │   └── Group
│           │       └── Controls
```

```
// All items inherit from DialogItem base
// DialogItem exposes: .itemID (number), .itemType (string)
// Legacy aliases retained: .controlID, .controlType
```

12.3 Minimal dialog example

```
'use strict';
const { Dialog, DialogResult } = require('/dialog');

const dlg = Dialog.create('My Script');
const col = dlg.addColumn();
const grp = col.addGroup('Options');

const nameBox = grp.addTextBox('Name', 'Default text');
const enableCb = grp.addCheckBox('Enable feature', true);

const result = dlg.runModal();

if (result == DialogResult.Ok) {
  console.log('Name: ', nameBox.text);
  console.log('Enabled: ', enableCb.value);
} else {
  console.log('User cancelled.');
```

12.4 All control types**12.4.1 TextBox**

```
// Snippet
const tb = grp.addTextBox('Label', 'initial text');
// Read: tb.text
tb.isMultiline = true;           // allow multiple lines
tb.rowSpan = 3;                  // height in rows
tb.textHorizontalAlignment = 1;  // 0=Left, 1=Centre, 2=Right
```

12.4.2 CheckBox

```
// Snippet
const cb = grp.addCheckBox('Enable option', false);
```

```
// Read: cb.value (boolean)
cb.isFullWidth = true;
```

12.4.3 Switch (toggle)

```
// Snippet
const sw = grp.addSwitch('Dark mode', false);
// Read: sw.value (boolean)
```

12.4.4 ComboBox (dropdown)

```
// Snippet
const FORMATS = ['PNG', 'JPEG', 'PDF', 'SVG'];
const combo = grp.addComboBox('Format', FORMATS, 0);
// Read: FORMATS[combo.selectedIndex]
// Note: combo.value returns undefined – always use selectedIndex
combo.isFullWidth = true;
```

12.4.5 RadioGroup

```
// Snippet
const OPTIONS = ['Low', 'Medium', 'High'];
const radio = grp.addRadioGroup('Quality', OPTIONS, 1);
// Read: OPTIONS[radio.selectedIndex]
radio.isFullWidth = true;
```

12.4.6 ButtonSet (segmented control)

```
// Snippet
const ALIGN = ['Left', 'Centre', 'Right'];
const btns = grp.addButtonSet('Align', ALIGN, 0);
btns.isFullWidth = true;
// Read: ALIGN[btns.selectedIndex]
```

12.4.7 ColourPicker

```
// Snippet
const { Colour } = require('/colours');
const picker = grp.addColourPicker('Colour'); // no initial colour
// Or with an initial colour:
// const picker = grp.addColourPicker('Colour', initialColour);
picker.allowPickNone = true; // allow "no colour" selection
picker.allowNoise = true; // allow noise fills
// Read: picker.value (Colour object or null)
```

12.4.8 FillEditor

```
// Snippet
const { SolidFill } = require('/fills');
const { Colour } = require('/colours');
const initialFill = SolidFill.create(new Colour(/* ... */));
const fillEditor = grp.addFillEditor('Fill', initialFill);
fillEditor.isFullWidth = true;
fillEditor.isStrokeFill = false; // true = stroke fill mode
// Read: fillEditor.fill (returns a fill descriptor)
```

12.4.9 StrokeEditor

```
// Snippet
const strokeEditor = grp.addStrokeEditor('Stroke');
strokeEditor.isFullWidth = true;
// Read: strokeEditor.stroke
```

12.4.10 Button

A standalone push button with an `onClickHandler`. Unlike other controls it doesn't hold a value — it fires an action.

```
// Snippet
const btn = grp.addButton('Apply');
btn.isFullWidth = true;
btn.alignment = HorizontalAlignment.Centre; // Centre, Left, Right

btn.onClickHandler = () => {
```

```

    console.log('Button clicked - running action');
  };

  // Fluent chaining - all setters return the item
  grp.addCheckBox('Dark mode', false)
    .setIsFullWidth(true)
    .setIsEnabled(false);

```

12.4.11 Control quick reference

Control	Factory	Read value	Key properties
TextBox	grp.addText Box(label, text)	.text	.isMulti Line, .row Span, .isFull Width, .isEn- abled, .de- scrip- tion, .textHor- izontalAlign- ment
CheckBox	grp.addCheck Box(label, checked)	.value (bool)	.isFull Width, .isEn- abled
Switch	grp.add Switch(label, checked)	.value (bool)	.isEnabled
ComboBox	grp.addCombo Box(label, items[], in- dex)	items[.se- lectedIndex] (.value returns undefined)	.selectedIndex, .isFull Width, .isEn- abled
RadioGroup	grp.addRadio Group(label, items[], in- dex)	items[.se- lectedIndex]	.selectedIndex, .isFull Width, .isEn- abled
ButtonSet	grp.addBut- tonSet(label, items[], in- dex)	items[.se- lectedIndex]	.selectedIndex, .isFull Width, .isEn- abled
ColourPicker	grp.addColour Picker(la- bel, initial Colour?)	.value (Colour or null)	.allowPick None, .allow Noise, .isEn- abled
FillEditor	grp.addFillEd- itor(label, initialFill?)	.fill (FillDe- scriptor)	.isFull Width, .is StrokeFill, .is Enabled
StrokeEditor	grp.addStroke Editor(la- bel, initial Stroke?)	.stroke	.isFull Width, .isEn- abled
Button	grp.addBut- ton(label)	— (fires .on ClickHandler)	.isFull Width, .align-

Control	Factory	Read value	Key properties
			ment (Left/Centre/Right)
UnitValueEditor	grp.addUnitValueEditor(label, storeUnit, dispUnit, value, min?, max?) grp.addUserUnitValueEditor(label, storeUnit, dispUnit, value, min?, max?)	.value (in store unit)	.showPopup Slider, .precision, .isEnabled, .isFullWidth, .noMinValue, .noMaxValue
FontPicker	grp.addFontPicker(label)	.text(name), .fontFamily, .fontCollection	.isEnabled
SpatialAnchor	grp.addSpatialAnchor(label)	.value (SpatialAnchor enum)	.isEnabled
StaticText	grp.addStaticText(label, text)	.text	.isFullWidth (read-only display)

Common to all controls: .isEnabled, .isVisible, .itemID, .itemType, .description, .onValueChangedHandler, .setIsVisible(v), .setEnabled(v), .setIsFullWidth(v)

Common to groups: .addColumnStack(), .isVisible, .setIsVisible(v), .enableSeparator, .setEnabledSeparator(v), .controls

Common to columns: .addGroup(label), .widthProportion, .paddingFactor, .setPaddingFactor(v), .isVisible, .setIsVisible(v)

Common to dialog: .isResizable, .initialWidth, .isRunningModal, .findControl(id), .controls, .setItemsVisibility(showIDs, hideIDs), .onControlValueChangedHandler

12.4.12 ColumnStack

Columns added directly to the dialog (dlg.addColumn()) sit at the top level and affect the whole dialog width. ColumnStack lets you create a multi-column sub-layout inside a single group – useful when one section of your dialog needs a side-by-side layout while the rest uses a single column:

```
// Snippet
const stack = grp.addColumnStack();
const leftCol = stack.addColumn();
const rightCol = stack.addColumn();

leftCol.addGroup('Left').addCheckBox('Option A', true);
rightCol.addGroup('Right').addCheckBox('Option B', false);
```

12.4.13 UnitValueEditor (numeric with units)

```
// Snippet
const { UnitType } = require('/units');
```

```
// Basic – store and display in the same unit
const editor = grp.addUnitValueEditor(
  'Size', // label
  UnitType.Pixel, // storage unit – values read/written in this unit
  UnitType.Pixel, // display unit – what the user sees
  100, // initial value (in storage units)
  1, // min (null = no minimum)
  2000 // max (null = no maximum)
);
editor.showPopupSlider = true; // show a slider popup on the field
editor.precision = 1; // decimal places
editor.noMinValue = false; // allow values below the original min
editor.noMaxValue = false; // allow values above the original max
// Read: editor.value (always in storage units)

// Mixed units – store in pixels, display in millimetres
// The user sees and types mm; .value returns pixels
const mmEditor = grp.addUnitValueEditor('Offset', UnitType.Pixel,
UnitType.Millimetre, 0);

// Unitless integer – for counts, indices, sides, etc.
const sidesEditor = grp.addUnitValueEditor('Sides', UnitType.Number,
UnitType.Number, 6, 3, 20);

// UserUnitType variant – accepts UserUnitType enums instead of UnitType
// const userEditor = grp.addUserUnitValueEditor(label, storeUnit,
displayUnit, value, min?, max?);
```

12.4.14 FontPicker

DialogFontPicker lets the user choose a font interactively. It exposes three read properties after the dialog closes: text (a plain name string), font (a Font object), and fontFamily / fontCollection for family- or collection-level access.

```
'use strict';
const { Dialog, DialogResult } = require('/dialog');
const { FontFamily } = require('/fonts');
const { Document } = require('/document');

const doc = Document.current;
if (doc) {
  const dlg = Dialog.create('Choose font');
  const col = dlg.addColumn();
  const grp = col.addGroup('');

  const fp = grp.addFontPicker('Font');
  fp.text = 'Arial'; // optional: set initial font by name

  if (dlg.runModal() == DialogResult.Ok) {
    console.log('Name: ', fp.text);
    console.log('Font: ', fp.font?.postscriptName);
    console.log('Family: ', fp.fontFamily?.name);
    console.log('Collection: ', fp.fontCollection?.displayName ??
'(none)');

    // Walk all variants in the chosen family
    const fam = fp.fontFamily;
    if (fam) {
      for (const f of fam.fonts) {
        console.log(' Variant:', f.postscriptName, 'bold:', f.isBold,
'italic:', f.isItalic);
      }
    }
  }
}
```

fontCollection is null when the font is not part of a named collection (e.g. system fonts). Always null-guard before using it.

12.4.15 SpatialAnchor (9-point anchor selector)

```
// Snippet
const anchor = grp.addSpatialAnchor('Origin');
// Read: anchor.value (SpatialAnchor enum)
```

12.4.16 StaticText (label)

```
// Snippet
const label = grp.addStaticText('Info', 'This is a read-only label.');
```

```
label.isFullWidth = true;
// Update text reactively:
label.text = 'Updated text';
```

12.4.17 Per-item visibility

Every dialog item now exposes `isVisible` (read/write) and a fluent `setVisible()`:

```
// Snippet
const showAdvanced = grp.addCheckBox('Show advanced', false);
const advanced = grp.addGroup('Advanced').setVisible(false);

showAdvanced.onValueChangedHandler = () => {
  advanced.isVisible = showAdvanced.value;
};
```

For bulk changes, use `setItemsVisibility(showIDs, hideIDs)` — pass arrays of `itemID` values:

```
// Snippet
const dlg = Dialog.create('My Dialog');
const col = dlg.addColumn();
const grp = col.addGroup('Options');
const rgbIds = [redEd.itemID, greenEd.itemID, blueEd.itemID];
const cmykIds = [cyanEd.itemID, magentaEd.itemID, yellowEd.itemID,
keyEd.itemID];

modeSelector.onValueChangedHandler = () => {
  if (modeSelector.selectedIndex === 0)
    dlg.setItemsVisibility(rgbIds, cmykIds); // show RGB, hide CMYK
  else
    dlg.setItemsVisibility(cmykIds, rgbIds); // show CMYK, hide RGB
};
```

12.5 Dialog-level properties

```
// Snippet
const dlg = Dialog.create('My Script');

// Make dialog resizable
dlg.isResizable = true;
dlg.initialWidth = 400; // pixels

// Find a control by ID after building the dialog
const ctrl = dlg.findControl(someControl.controlID);

// Iterate all controls across all groups
for (const ctrl of dlg.controls) {
  console.log(ctrl.label);
}

// Check if dialog is currently showing
console.log('Is modal open?', dlg.isRunningModal);

// Column padding
col.paddingFactor = 0.5; // finer spacing control
col.setPaddingFactor(0.8); // fluent alternative

// Control description — now readable
```

Note
StaticText with empty label crashes. `addStaticText('', text)` throws `INVALID_OP` at `runModal()` time. Always pass at least ' ' if you need no visible label.

Note
 Use `itemID` (not `controlID`) with `setItemsVisibility`. The `controlID` property is an alias for `itemID` but `itemID` is the canonical name.

```
tb.description = 'Enter the filename';
console.log(tb.description); // readable description
```

12.6 Multi-column dialogs

```
'use strict';
const { Dialog, DialogResult } = require('/dialog');
const { UnitType } = require('/units');

const dlg = Dialog.create('Export Options');

const col1 = dlg.addColumn();
col1.widthProportion = 1;
const grp1 = col1.addGroup('Format');
const FORMATS = ['PNG', 'JPEG', 'PDF'];
const formatCombo = grp1.addComboBox('Type', FORMATS, 0);

const col2 = dlg.addColumn();
col2.widthProportion = 1;
const grp2 = col2.addGroup('Quality');
const qualityEdit = grp2.addUnitValueEditor('DPI', UnitType.Pixel,
UnitType.Pixel, 300, 72, 600);

const result = dlg.runModal();

if (result == DialogResult.Ok) {
  console.log('Format:', FORMATS[formatCombo.selectedIndex]);
  console.log('DPI: ', qualityEdit.value);
} else {
  console.log('Cancelled.');
```

12.7 Reactive controls

Two approaches are available. Per-control handlers use `onValueChanged` Handler set directly on each control — no argument, just read the control's current value. The dialog-level handler uses `onControlValueChangedHandler` on the dialog and receives the changed control as its argument.

❶ **Note**
Handlers fire on user interaction only. Setting a control value from script code (`ctrl.value = x`) does not trigger `onValueChangedHandler` — only genuine user interaction does. This prevents accidental handler recursion.

`setIsEnabled()` now works correctly. It correctly updates `isEnabled`.

Values modified inside a handler are reflected immediately. Changing a control's value inside an `onValueChangedHandler` updates the UI correctly.

Per-control (simpler for most cases):

```
'use strict';
const { Dialog, DialogResult } = require('/dialog');
const { UnitType } = require('/units');

const dlg = Dialog.create('Export Size');
const col = dlg.addColumn();
const grp = col.addGroup('Options');

const useCustom = grp.addCheckBox('Use custom size', false);
const widthEdit = grp.addUnitValueEditor('Width', UnitType.Pixel,
UnitType.Pixel, 800, 1, 4096);
const heightEdit = grp.addUnitValueEditor('Height', UnitType.Pixel,
UnitType.Pixel, 600, 1, 4096);

widthEdit.isEnabled = false;
heightEdit.isEnabled = false;

// onValueChangedHandler on the control — no argument, read control value directly
useCustom.onValueChangedHandler = () => {
  widthEdit.isEnabled = useCustom.value;
  heightEdit.isEnabled = useCustom.value;
};

const result = dlg.runModal();
if (result == DialogResult.Ok) {
  if (useCustom.value) {
    console.log('Width:', widthEdit.value, 'Height:', heightEdit.value);
```

```

    } else {
      console.log('Using default size.');
```

Dialog-level (useful when many controls share logic):

```

// Snippet
// onControlValueChangedHandler on the dialog – receives changed control
as argument
dlg.onControlValueChangedHandler = (ctrl) => {
  if (ctrl === useCustom) {
    widthEdit.isEnabled = useCustom.value;
    heightEdit.isEnabled = useCustom.value;
  }
};
```

12.8 Full practical example – export dialog

```

'use strict';
const { Document, FileExportOptions } = require('/document');
const { Dialog, DialogResult } = require('/dialog');
const { app } = require('/application');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

12.9 Reactive dialog example – shape creator

08d-reactive-dialog.js shows a complete reactive dialog in practice. It builds a shape creation tool where:

- A ComboBox selects the shape type – changing it enables the Sides field only when Polygon is selected
- Width and Height fields each have their own onControlValueChangedHandler – when “Lock aspect ratio” is ticked, changing one syncs the other
- A static text label updates in real time as values change, showing a preview of what will be created before the user clicks OK

The key pattern is that each control has its own `onValueChangedHandler` set before `runModal()`. The function takes no argument — read the control's value directly. The function runs inside the modal without closing it.

```
// Snippet
// Per-control handler — no argument needed
useCustom.onValueChangedHandler = () => {
  widthEdit.isEnabled = useCustom.value;
  heightEdit.isEnabled = useCustom.value;
};
```

See `08d-reactive-dialog.js` for the full working implementation.

12.10 Live preview dialogs

The most powerful dialog pattern runs preview commands while the dialog is open, so the user sees the effect of their settings in real time before committing. This uses three elements working together:

- **`doc.executeCommand(cmd, true)`** — the second argument `true` marks this as a preview. It renders immediately but is not added to the undo history.
- **`doc.clearPreviews()`** — removes all active preview commands. Must always be called when the dialog loop ends, whether the user clicks OK or Cancel.
- **`while (dlg.runModal() == DialogResult.Ok)`** — keeps the dialog open if the user clicks OK but validation fails, allowing another attempt.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');
const { UnitType } = require('/units');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const dlg = Dialog.create('Live Preview Example');
  const col = dlg.addColumn();
  const grp = col.addGroup('Settings');

  // Attach controls directly to dlg for convenient access in helper
  // functions
  dlg.opacity = grp.addUnitValueEditor('Opacity %', UnitType.Number,
UnitType.Number, 100, 0, 100);

  function update() {
    // Build and execute a preview command — rendered but not committed
    const opacity = dlg.opacity.value / 100;
    const cmd = DocumentCommand.createSetOpacity(doc.selection, opacity);
    doc.executeCommand(cmd, true); // true = preview
  }

  // Update preview every time a control changes
  dlg.onControlValueChangedHandler = update;

  // Loop keeps dialog open if validation fails
  while (dlg.runModal() == DialogResult.Ok) {
    if (dlg.opacity.value < 0 || dlg.opacity.value > 100) {
      alert('Opacity must be between 0 and 100.');
```

```
// Always clear previews – whether OK or Cancel
doc.clearPreviews();
}
```

12.11 Attaching controls to the dialog object

Rather than storing all controls in separate variables, you can attach them as properties directly on the dialog object. This keeps related controls together and makes helper functions cleaner — they only need the `dlg` argument:

```
// Snippet
const dlg = Dialog.create('Add Guides');
const col = dlg.addColumn();
const grp = col.addGroup('Position');

// Attach controls as properties on dlg itself
dlg.left = grp.addSwitch('Left');
dlg.right = grp.addSwitch('Right');
dlg.top = grp.addSwitch('Top');
dlg.bottom = grp.addSwitch('Bottom');

// You can also attach helper methods
dlg.anySelected = () => dlg.left.value || dlg.right.value || dlg.top.value
|| dlg.bottom.value;

while (dlg.runModal() == DialogResult.Ok) {
  if (!dlg.anySelected()) {
    alert('Please select at least one side.');
```

12.12 alert()

For simple validation messages, `alert()` is available as a global:

```
// Snippet
if (!doc) {
  alert('This script requires an open document.');
```

It blocks execution and shows a native system alert. Use it for error conditions only — not as a substitute for `console.log`.

12.13 The canonical example — `addGuides.js`

The SDK ships a built-in example at `examples/addGuides.js` that demonstrates the full production-quality dialog pattern: live preview with the while loop, per-control `onValueChangedHandler`, mixed units (`UnitType.Pixel` stored / `UnitType.Millimetre` displayed), controls attached to the dialog object, and `clearPreviews()` always called after the loop. Reading that script alongside this lesson is the most direct way to see all the patterns working together.

12.14 Key gotchas

combo.selectedIndex not combo.value. `combo.value` returns undefined. Always read the selection as `combo.selectedIndex` and look up the value in your options array.

Per-control handler is `onValueChangedHandler`, dialog handler is `onControlValueChangedHandler`. These are different properties — `onValueChangedHandler` lives on each control and takes no argument. `onControlValueChangedHandler` lives on the dialog and receives the changed control.

Note
`doc.clearPreviews()` must always be called after the loop. If the user cancels, previews would otherwise remain visible on the canvas permanently.

Note
Previews don't stack. Only one preview command is active at a time. Rebuild everything in a single command (or compound command) on each change rather than layering preview commands on top of each other. The SDK replaces the previous preview with the new one automatically.

UnitType.Pixel not UnitType.Pixels, UnitType.Number for unitless. All enum values use singular spelling. Use `UnitType.Number` when you want a plain integer with no unit — e.g. a count of sides or a percentage value.

Storage unit vs display unit in `addUnitValueEditor`. The second argument is the storage unit (what `.value` returns), the third is the display unit (what the user sees). They can differ — e.g. (`UnitType.Pixel`, `UnitType.Millimetre`, `0`) stores pixels, displays mm.

Previews don't stack — rebuild everything each time. `doc.executeCommand(cmd, true)` replaces the previous preview, not adds to it. Build a single compound command covering all changes on each update.

Always call `doc.clearPreviews()` after the dialog loop. If the user cancels, previews remain on the canvas permanently without this call.

Read control values only after `runModal()` returns. Values are set by user interaction — reading them before the dialog is shown returns initial values only.

Fill/Stroke editors and buttons return proper typed wrappers. `addFillEditor`, `addStrokeEditor` and `addButton` return their correct types — use them directly without casting. `createTypedDialogControl` is not needed.

① **Note**
`result == DialogResult.Ok` — title-case `Ok` not `OK`. The simpler `==` comparison works directly. Wrong case returns undefined and your check silently fails.

① **Note**
There is no `addSlider`. The SDK dialog has no slider control. Use `addComboBox` for a fixed set of options, `addUnitValueEditor` for a free numeric input, or set `showPopupSlider = true` on a `UnitValueEditor`.

12.15 Exercises

12.15.1 Exercise 8.1 — Colour picker dialog

Build a dialog with a `ColourPicker` and a `Switch` for “apply to stroke”. Apply the chosen colour to all selected nodes when confirmed.

12.15.2 Exercise 8.2 — Fill editor dialog

Build a dialog with a `FillEditor`. Apply the chosen fill to all selected nodes when confirmed.

12.15.3 Exercise 8.3 — Export preset picker

Show a `ComboBox` with all available export preset names. Export the current document using the chosen preset on OK.

12.15.4 Exercise 8.4 — Reactive form

Build a dialog where a `CheckBox` controls whether two `UnitValueEditor` fields (width and height) are enabled. Use `onValueChangedHandler` on the checkbox. Use `UnitType.Pixel` for storage and `UnitType.Millimetre` for display.

12.15.5 Exercise 8.5 — Reactive shape creator

See `08d-reactive-dialog.js` for a full implementation. Extend it by adding a `ColourPicker` control and applying the chosen fill colour to the newly created shape after it is added to the document.

12.15.6 Exercise 8.6 — Live preview opacity

See `08e-live-preview-dialog.js` for a full working implementation. Extend it by adding a second `UnitValueEditor` for blur radius, and applying both opacity and a Gaussian blur as a combined preview command using `CompoundCommandBuilder`.

12.16 SDK modules used in this lesson

- `/dialog` — `Dialog`, `DialogResult`, all control classes
- `/colours` — `Colour`
- `/fills` — `SolidFill`
- `/units` — `UnitType`

- /document — FileExportOptions
- /shapes — ShapeRectangle, ShapeEllipse, etc. (used in 08d)
- /nodes — ShapeNodeDefinition (used in 08d)

12.17 See also

- Lesson 07 — Text & Story
- Lesson 09 — Export & Files
- examples/08d-reactive-dialog.js — reactive shape creator
- examples/08e-live-preview-dialog.js — live preview with opacity
- SDK example: examples/addGuides.js — canonical production-quality dialog pattern

12.18 Examples

12.18.1 Colour picker dialog

```
'use strict';

// Example 08a — Colour picker dialog
// Lesson 08: Dialog UI
//
// Shows a dialog with a colour picker and stroke toggle.
// Applies the chosen colour to selected nodes on OK.

const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');
const { FillDescriptor } = require('/fills');
const { Colour } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some objects and run again.')}
else {
  const dlg = Dialog.create('Apply Colour');
  const col = dlg.addColumn();
  const grp = col.addGroup('Colour');

  // addColourPicker accepts a Colour object or null (no initial colour)
  const picker = grp.addColourPicker('Fill colour');
  const strokeSw = grp.addSwitch('Also apply to stroke', false);

  const result = dlg.runModal();

  if (result == DialogResult.Ok) {
    const colour = picker.value;
    if (!colour) { console.log('No colour selected.')}
    else {
      const fill = FillDescriptor.createSolid(colour);
      const compound = CompoundCommandBuilder.create();

      compound.addCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, doc.selection));
      if (strokeSw.value) {
        compound.addCommand(DocumentCommand.createSetPenFill({ fillDescriptor:
fill }, doc.selection));
      }

      doc.executeCommand(compound.createCommand());
      console.log('Colour applied' + (strokeSw.value ? ' to fill and
stroke.' : ' to fill.'));
    }
  } else {
    console.log('Cancelled.');
```

```

    }
  }

```

12.18.2 Batch rename dialog

```

'use strict';

// Example 08b – Batch rename dialog
// Lesson 08: Dialog UI
//
// Renames all selected layers by wrapping their current names
// with a prefix and/or suffix entered by the user.

const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');
const { Selection } = require('/selections');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some layers and run again.')}
} else {
  const dlg = Dialog.create('Batch Rename Layers');
  const col = dlg.addColumn();
  const grp = col.addGroup('Name pattern');

  const prefixBox = grp.addTextBox('Prefix', '');
  const suffixBox = grp.addTextBox('Suffix', '');

  const result = dlg.runModal();

  if (result == DialogResult.Ok) {
    const prefix = prefixBox.text;
    const suffix = suffixBox.text;
    const nodes = doc.selection.nodes;
    const compound = CompoundCommandBuilder.create();
    let count = 0;

    for (const node of nodes) {
      const currentName = node.descriptionInterface?.description ?? '';
      const newName = `${prefix}${currentName}${suffix}`;
      const nodeSel = Selection.create(doc, node);
      compound.addCommand(DocumentCommand.createSetDescription(nodeSel,
newName));
      count++;
    }

    doc.executeCommand(compound.createCommand());
    console.log(`Renamed ${count} layer(s).`);
  } else {
    console.log('Cancelled.')}
  }
}

```

12.18.3 Fill and stroke editor dialog

```

'use strict';

// Example 08c – Fill and stroke editor dialog
// Lesson 08: Dialog UI
//
// Shows a dialog with a FillEditor and StrokeEditor – the most capable
// fill/stroke controls available in the dialog API. Applies the chosen
// fill and stroke to all selected nodes on OK.
//
// FillEditor supports solid, gradient, and pattern fills.
// StrokeEditor supports full stroke style configuration.

const { Document } = require('/document');

```

```

const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some objects and run again.')}
else {
  const dlg = Dialog.create('Apply Fill & Stroke');
  dlg.isResizable = true;

  const col = dlg.addColumn();

  const fillGrp = col.addGroup('Fill');
  const fillEditor = fillGrp.addFillEditor('Fill');
  fillEditor.isFullWidth = true;

  const strokeGrp = col.addGroup('Stroke');
  const strokeEditor = strokeGrp.addStrokeEditor('Stroke');
  strokeEditor.isFullWidth = true;

  const result = dlg.runModal();

  if (result == DialogResult.Ok) {
    const fill = fillEditor.fill;
    const stroke = strokeEditor.stroke;

    const compound = CompoundCommandBuilder.create();
    if (fill)
      compound.addCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, doc.selection));
    if (stroke)
      compound.addCommand(DocumentCommand.createSetPenFill({ fillDescriptor:
stroke }, doc.selection));
    doc.executeCommand(compound.createCommand());

    console.log('Fill and stroke applied.')}
  else {
    console.log('Cancelled.')}
  }
}

```

12.18.4 Reactive dialog with value changed callbacks

```

'use strict';

// Example 08d – Reactive dialog with value changed callbacks
// Lesson 08: Dialog UI
//
// Demonstrates onValueChangedHandler on individual controls.
// Each control has its own handler – no need to branch on which control
// fired.
//
// This example builds a shape creation dialog where:
// - A ComboBox selects the shape type
// - Sides field enables only when Polygon is selected
// - A "lock aspect ratio" checkbox keeps width and height in sync
// - A live preview label updates as values change

const { Document } = require('/document');
const { ShapeNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder, DocumentCommand } = require('/
commands');
const { Dialog, DialogResult } = require('/dialog');
const { UnitType } = require('/units');
const {
  ShapeRectangle,
  ShapeEllipse,
  ShapePolygon,

```

```

    ShapeStar,
    ShapeTriangle
  } = require('/shapes');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const SHAPES      = ['Rectangle', 'Ellipse', 'Polygon', 'Star',
    'Triangle'];
  const SHAPE_CTORS = [ShapeRectangle, ShapeEllipse, ShapePolygon,
    ShapeStar, ShapeTriangle];

  // — Build dialog

  const dlg = Dialog.create('Add Shape');
  dlg.isResizable = true;

  const col = dlg.addColumn();

  const typeGrp = col.addGroup('Shape');
  const typeCombo = typeGrp.addComboBox('Type', SHAPES, 0);

  const sizeGrp = col.addGroup('Size');
  const widthEdit = sizeGrp.addUnitValueEditor('Width', UnitType.Pixel,
    UnitType.Pixel, 200, 10, 2000);
  const heightEdit = sizeGrp.addUnitValueEditor('Height', UnitType.Pixel,
    UnitType.Pixel, 200, 10, 2000);
  widthEdit.showPopupSlider = true;
  heightEdit.showPopupSlider = true;

  const optGrp = col.addGroup('Options');
  const lockAR = optGrp.addCheckBox('Lock aspect ratio', false);
  const sidesEdit = optGrp.addUnitValueEditor('Sides', UnitType.Number,
    UnitType.Number, 6, 3, 20); // unitless integer
  sidesEdit.isEnabled = false;

  const infoGrp = col.addGroup('');
  const infoLabel = infoGrp.addStaticText('', `${SHAPES[0]} 200 × 200
    px`);
  infoLabel.isFullWidth = true;

  // — Helper

  function updateLabel() {
    const name = SHAPES[typeCombo.selectedIndex];
    const w = Math.round(widthEdit.value);
    const h = Math.round(heightEdit.value);
    infoLabel.text = `${name} ${w} × ${h} px`;
  }

  // — Per-control handlers

  // onValueChangedHandler is set on each control individually.
  // The dialog-level onControlValueChangedHandler is also available
  // if you prefer a single entry point.

  typeCombo.onValueChangedHandler = () => {
    sidesEdit.isEnabled = (SHAPES[typeCombo.selectedIndex] === 'Polygon');
    updateLabel();
  };

  let updatingFromLock = false;

  widthEdit.onValueChangedHandler = () => {
    if (lockAR.value && !updatingFromLock) {
      updatingFromLock = true;
    }
  }

```

```

        heightEdit.value = widthEdit.value;
        updatingFromLock = false;
    }
    updateLabel();
};

heightEdit.onValueChangedHandler = () => {
    if (lockAR.value && !updatingFromLock) {
        updatingFromLock = true;
        widthEdit.value = heightEdit.value;
        updatingFromLock = false;
    }
    updateLabel();
};

lockAR.onValueChangedHandler = () => {
    if (lockAR.value) {
        heightEdit.value = widthEdit.value;
        updateLabel();
    }
};

// — Run

```

```

const result = dlg.runModal();

if (result !== DialogResult.Ok) {
    console.log('Cancelled. ');
} else {
    const idx    = typeCombo.selectedIndex;
    const shape = SHAPE_CTORS[idx].create();
    const w     = widthEdit.value;
    const h     = heightEdit.value;

    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.setBoundingBox({ x: 100, y: 100, width: w, height: h });

    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    const cmd = builder.createCommand(true);
    doc.executeCommand(cmd);

    if (SHAPES[idx] === 'Polygon') {
        const sides = Math.round(sidesEdit.value);
        doc.executeCommand(DocumentCommand.createSetShapeIntParam(doc.selection,
            'NumberOfPoints', sides));
    }

    console.log(`${SHAPES[idx]} added — ${Math.round(w)} ×
    ${Math.round(h)} px`);
}

```

12.18.5 Live preview dialog

```

'use strict';

// Example 08e — Live preview dialog
// Lesson 08: Dialog UI
//
// Demonstrates the live preview pattern:
// - doc.executeCommand(cmd, true) — preview flag: renders but not in
//   undo history
// - doc.clearPreviews() — removes all previews (always call
//   after loop)
// - while (runModal() == Ok) — loop for validation, re-showing
//   dialog on failure

```

```

// - onControlValueChangedHandler - dialog-level reactive callback
//
// Select one or more objects and run.
// The opacity changes live as you move the slider - cancel restores the
// original.

const { Document } = require('/document');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');
const { UnitType } = require('/units');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some objects and run again.');
} else {
  const sel = doc.selection;

  // — Build dialog

  const dlg = Dialog.create('Set Opacity');
  const col = dlg.addColumn();
  const grp = col.addGroup('Settings');

  // UnitType.Number = unitless integer, shown without unit suffix
  dlg.opacity = grp.addUnitValueEditor('Opacity %', UnitType.Number,
UnitType.Number, 100, 0, 100);
  dlg.opacity.showPopupSlider = true;
  dlg.opacity.precision = 0;

  dlg.infoLabel = grp.addStaticText('', 'Opacity: 100%');
  dlg.infoLabel.isFullWidth = true;

  // — Live preview

  // Dialog-level handler fires on every control change.
  // executeCommand with preview=true renders immediately but is not
  // committed.

  function update() {
    const pct = Math.round(dlg.opacity.value);
    dlg.infoLabel.text = `Opacity: ${pct}%`;
    const cmd = DocumentCommand.createSetOpacity(sel, pct / 100);
    doc.executeCommand(cmd, true); // true = preview
  }

  dlg.onControlValueChangedHandler = update;

  // — Run loop

  // while loop allows re-showing the dialog with an alert if validation
  // fails.

  while (dlg.runModal() == DialogResult.Ok) {
    const pct = dlg.opacity.value;
    if (pct < 0 || pct > 100) {
      alert('Opacity must be between 0 and 100.');
      continue;
    }
    // Commit - run without preview flag so it goes into the undo history
    doc.executeCommand(DocumentCommand.createSetOpacity(sel, pct / 100));
    console.log('Opacity set to ${Math.round(pct)}%.`);
    break;
  }

  // ALWAYS clear previews - if user cancels, previews stay on canvas
  // without this

```

```
    doc.clearPreviews();  
}
```


Lesson 09 — Export & File I/O

13

Getting content out of Affinity — as PNG files, PDFs, SVGs, or any of the 60+ supported formats — is one of the most valuable things a script can do. This lesson covers the full export pipeline: choosing a format, defining what to export, writing the output to disk, and optionally generating a JSON manifest alongside it. It also covers reading and writing plain text files and making HTTP requests.

Export automation is where scripts deliver the most time savings. A script that exports every artboard to a named PNG, writes a manifest, and uploads to an API can replace hours of manual work. This lesson gives you all the pieces: `FileExportOptions`, `FileExportArea`, `doc.export()`, the `File` class, and `HttpRequest`.

The export API has a matrix of choices that can be disorienting at first: format presets, area types, export areas, and result records all need to be understood together. The format preset selects what kind of file to produce; the export area selects what part of the document to include. They are independent axes, and the combination you choose determines the entire output. This lesson works through the most common combinations so you can recognise the right one for any situation.

File access and networking complete the picture. The `fs` module lets you write JSON manifests, configuration files, and any other text output directly to the Desktop without leaving the script environment. The `network` module lets you make HTTP calls — uploading exports, fetching data for data-driven layouts, or integrating with external services. The `network` module has one syntax detail that trips up every developer who encounters it for the first time: requests are executed with `req['do']()` rather than `req.do()`, because `do` is a

reserved word in JavaScript. That single character difference is the source of an error that can take considerable time to track down without prior warning.

By the end of this lesson you will have a complete pipeline for export automation: format selection, area targeting, file writing, manifest generation, and external communication. The example scripts in this lesson are among the most directly reusable in the entire tutorial — export pipelines tend to be the same shape regardless of the project they serve.

13.1 What this lesson covers

- `FileExportOptions` — choosing and configuring export formats
- `FileExportArea` — what part of the document to export
- `doc.export()` — synchronous export
- `doc.promises.export()` — async export
- Checking export results
- `fs` module — reading and writing files on the Desktop
- `network` module — HTTP requests from scripts

13.2 Export overview

Three things combine to describe an export operation:

Object	What it defines
<code>FileExportOptions</code>	Format (PNG, JPEG, PDF, SVG, etc.) and format-specific settings
<code>FileExportArea</code>	Which part of the document (whole doc, current spread, artboard, selection)
Path string	Where the file goes (Desktop only)

13.3 Getting available export presets

`FileExportOptions.allPresetNames` lists every format preset available in the current Affinity session.

```
'use strict';
const { FileExportOptions } = require('/document');

const presets = FileExportOptions.allPresetNames;
console.log('Available presets:');
for (const name of presets) {
  console.log(' ', name);
}
// PNG, JPEG, PDF, SVG, EPS, TIFF, ...
```

13.4 Synchronous export

Call `doc.export()` with a format preset, area, and output path — returns an array of result records.

```
'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/document');
const { app } = require('/application');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForWholeDocument();
  const outPath = `${desktop}/${doc.title}.png`;
```

```

const records = doc.export(outPath, opts, area, null);

for (const rec of records.all) {
  if (rec.isSuccess) {
    console.log('Exported to:', rec.path);
  } else {
    console.log('Export failed:', rec.errorMessage);
  }
}
}
}

```

13.5 Export areas

FileExportArea provides factory methods for targeting different parts of the document — whole doc, current spread, specific artboards, pages, or selected nodes.

```

'use strict';
const { Document, FileExportArea } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  // Whole document
  const wholeDoc = FileExportArea.createForWholeDocument();

  // Current spread only
  const spread = FileExportArea.createForCurrentSpread();

  // Current page only
  const page = FileExportArea.createForCurrentPage();

  // Specific artboard — ai comes from spread.getArtboard(i)
  // const ai      = doc.spreads.first.getArtboard(0);
  // const artboard = FileExportArea.createForArtboard(ai);

  // Selection bounding box
  const selArea = FileExportArea.createForSelectionArea(doc.selection);

  // Specific pages by string (e.g. "1-3,5")
  const pages = FileExportArea.createForPages('1-3');

  // Specific spreads by string
  const spreads = FileExportArea.createForSpreads('1-2');

  // Exact selection bounding box (vs selectionArea which pads to page)
  const sel = FileExportArea.createForSelection(doc.selection);

  console.log('Export areas ready.')}
}

```

13.6 Export every artboard

Iterate spreads and artboards, exporting each one as a separate file named after the artboard.

```

'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/document');
const { app } = require('/application');

const doc      = Document.current;
const desktop  = app.userDesktopPath;

if (!doc) { console.log('No document open.')} }
else {
  const opts = FileExportOptions.createWithPresetName('PNG');

  for (const spread of doc.spreads) {

```

```

    for (let i = 0; i < spread.artboardCount; i++) {
      const ai = spread.getArtboard(i); // returns ArtboardInterface
      directly
      const name = ai.description ?? `artboard-${i}`;
      const area = FileExportArea.createForArtboard(ai);
      const path = `${desktop}/${doc.title}_${name}.png`;

      const records = doc.export(path, opts, area, null);
      const rec = records.all[0];
      console.log(rec?.isSuccess ? `OK: ${path}` : `FAIL: ${name}`);
    }
  }
}

```

13.7 Async export with doc.promises

Use `DocumentPromises.getCurrent()` and `docP.export()` for async export workflows that don't block the UI.

```

'use strict';
async function exportAsync() {
  const { DocumentPromises, FileExportOptions, FileExportArea } =
  require('/document');
  const { app } = require('/application');

  const docP = await DocumentPromises.getCurrent();
  if (!docP) { console.log('No doc.');
```

```

  return; }

  const opts = FileExportOptions.createWithPresetName('PDF');
  const area = FileExportArea.createForWholeDocument();
  const desktop = app.userDesktopPath;
  const path = `${desktop}/output.pdf`;

  try {
    const records = await docP.export(path, opts, area, null);
    for (const rec of records.all) {
      console.log(rec.isSuccess ? 'Exported: ' + rec.path : 'Failed.');
```

```

    }
  } catch (err) {
    console.log('Export error:', err);
  }
}

exportAsync();

```

13.8 Saving the document

`doc.save()` writes in place; `doc.saveAs(path)` writes to a new location. New documents require `saveAs()` first.

```

'use strict';
const { Document } = require('/document');
const { app } = require('/application');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  const desktop = app.userDesktopPath;

  // Save in place (fails if mustSaveAs is true)
  doc.save();

  // Save to a new path
  doc.saveAs(`${desktop}/my-file.af`);

  // Async versions
  // doc.promises.save();
  // doc.promises.saveAs(`${desktop}/my-file.af`);
}

```

13.9 File system — reading and writing text

Use `File` from `/fs` to read and write text files. Only the Desktop directory is accessible.

```
'use strict';
const { File } = require('/fs');
const { Buffer } = require('/buffer');
const { app } = require('/application');

const desktop = app.userDesktopPath;

// Write a text file
const wf = new File(`${desktop}/output.txt`, 'w');
wf.writeString('Hello from Affinity SDK');
wf.close();
console.log('File written.');
```

```
// Read a text file
const rf = new File(`${desktop}/output.txt`, 'r');
const buf = Buffer.create(rf.length);
rf.read(buf, buf.length);
rf.close();
console.log(buf.toString());
```

13.10 JSON sidecar pattern

A common pattern — export assets and write a JSON manifest alongside them:

```
'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/document');
const { app } = require('/application');
const { File } = require('/fs');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
  const manifest = [];

  for (const spread of doc.spreads) {
    for (let i = 0; i < spread.artboardCount; i++) {
      const ai = spread.getArtboard(i); // returns ArtboardInterface
      directly
      const name = ai.description ?? `artboard-${i}`;
      const path = `${desktop}/${name}.png`;

      const area = FileExportArea.createForArtboard(ai);
      const records = doc.export(path, opts, area, null);
      if (records.all[0]?.isSuccess) {
        manifest.push({ name, path, exported: new Date().toISOString() });
      }
    }
  }

  if (manifest.length === 0) {
    console.log('No artboards found — add artboards to your document first.');
```

```
  } else {
    const manifestPath = `${desktop}/manifest.json`;
    const mf = new File(manifestPath, 'w');
    mf.writeString(JSON.stringify(manifest, null, 2));
    mf.close();
    console.log(`Manifest written: ${manifest.length} artboard(s).`);
  }
}
```

Note
Desktop only. Any path outside the Desktop folder returns `PERMISSION_DENIED`.

13.11 Network requests

- ① **Note**
Network access requires permission. In Affinity Settings, the user must enable

network access for scripts. If not enabled, `HttpRequest.create()` throws `PERMISSION_DENIED`.

- ① **Note**
`req['do']()` not `req.do()`. `do` is a reserved word in JavaScript — calling `req.do()` produces `INVALID_HANDLE`. Always use bracket notation.

```
'use strict';
const { HttpRequest, RequestMethod } = require('/network');

// GET request — uses httpbin.org, a public HTTP testing service
try {
  const req = HttpRequest.create('https://httpbin.org/get',
    RequestMethod.Get);
  const result = req['do']();
  const response = result.response;
  console.log('Status:', response.statusCode); // statusCode is an enum
  // use == 200 not === 200
  console.log('Body: ', response.content);
} catch(e) {
  console.log('Network error:', e.message);
  // PERMISSION_DENIED → enable network access in Affinity Settings
  // INVALID_HANDLE → the server could not be reached (bad URL or no
  // connectivity)
}

// POST with JSON body
try {
  const req = HttpRequest.create('https://httpbin.org/post',
    RequestMethod.Post);
  req.setHeaderValue('Content-Type', 'application/json');
  const result = req['do']();
  console.log('POST status:', result.response.statusCode); // statusCode
  // is an enum object — use == 200 not === 200
} catch(e) {
  console.log('Network error:', e.message);
}
```

13.12 Key gotchas

`req['do']()` not `req.do()`. `do` is a reserved word in JavaScript strict mode — calling `req.do()` throws `INVALID_HANDLE`. Always use bracket notation: `req['do']()`.

`response.statusCode == 200` not

13.13 200

`.statusCode` is an enum object, not a number or string. Strict equality always returns false. Use loose equality (`==`) or `statusCode.equals(200)`.

Paths elsewhere fail with `PERMISSION_DENIED`. Always use `app.userDesktopPath` as the base.

mustSaveAs on new documents. Call `doc.saveAs()` first before `doc.save()` or export — a fresh document has no path.

Check `rec.isSuccess` on every record. Export can partially succeed with warnings. Always iterate `records.all` and check each record.

- ① **Note**
Export area is optional — pass null to export the whole document by default.

13.14 Exercises

13.14.1 Exercise 9.1 — Export current spread

Export the current spread as a PNG to the Desktop, named after the document title.

13.14.2 Exercise 9.2 — Export all artboards

Export every artboard in the document as separate PNG files. Name each file after the artboard.

13.14.3 Exercise 9.3 – JSON manifest

After exporting all artboards, write a manifest.json to the Desktop listing each exported file path, artboard name, and timestamp.

13.14.4 Exercise 9.4 – Export with dialog

Combine Lesson 08 (Dialog) with this lesson: show a dialog to choose the export format and output path suffix, then export.

13.15 SDK modules used in this lesson

- /document — Document, FileExportOptions, FileExportArea, Document Promises
- /application — app
- /fs — File, Buffer, Directory, FileSystemPromises
- /network — HttpRequest, RequestMethod, HttpStatusCode

13.16 See also

- Lesson 08 — Dialog UI
- Lesson 10 — AI Commands
- examples/

13.17 Examples**13.17.1 Export all artboards with JSON manifest**

```
'use strict';

// Example 09a — Export all artboards with JSON manifest
// Lesson 09: Export & Files
//
// Exports every artboard in the document as PNG files to the Desktop.
// Writes a manifest.json alongside them.
//
// NOTE: spread.getArtboard(i) returns the ArtboardInterface directly.
// Pass it straight to FileExportArea.createForArtboard().
// The artboard name is in .description, not .name.

const { Document, FileExportOptions, FileExportArea } = require('/document');
const { app } = require('/application');
const { File } = require('/fs');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
  const manifest = [];
  let exported = 0;
  let failed = 0;
  let total = 0;

  for (const spread of doc.spreads) total += spread.artboardCount;

  if (total === 0) {
    console.log('No artboards found. Add artboards to your document first.');
```

```
  } else {
    for (const spread of doc.spreads) {
      for (let i = 0; i < spread.artboardCount; i++) {
        const ai = spread.getArtboard(i); // returns
        ArtboardInterface directly
        const name = (ai.description ?? `artboard-${exported + failed + 1}`)
          .replace(/^[a-zA-Z0-9_-]/g, '_');
        const path = `${desktop}/${doc.title}_${name}.png`;
```

```

const area      = FileExportArea.createForArtboard(ai);
const records = doc.export(path, opts, area, null);
const rec       = records.all[0];

if (rec?.isSuccess) {
  console.log('OK:', path);
  manifest.push({
    artboard: ai.description ?? name,
    file:      path,
    timestamp: new Date().toISOString(),
  });
  exported++;
} else {
  console.log('FAIL:', name, rec?.errorMessage ?? '');
  failed++;
}
}
}

const manifestPath = `${desktop}/${doc.title}_manifest.json`;
const mf = new File(manifestPath, 'w');
mf.writeString(JSON.stringify(manifest, null, 2));
mf.close();

console.log(`\nDone. ${exported} exported, ${failed} failed.`);
console.log('Manifest:', manifestPath);
}
}

```

13.17.2 Network requests

```

'use strict';

// Example 09b – Network requests
// Lesson 09: Export & Files
//
// Demonstrates GET and POST requests using the /network module.
// Uses httpbin.org – a public HTTP testing service that echoes requests
// back.
//
// Before running: enable network access in Affinity Settings > Scripting.

const { HttpRequest, RequestMethod } = require('/network');

// GET request
try {
  const req = HttpRequest.create('https://httpbin.org/get',
    RequestMethod.Get);
  const result = req['do'](); // 'do' is a reserved word – bracket
  // notation is required
  const response = result.response;
  console.log('GET status:', response.statusCode); // statusCode is an
  // enum object – use == 200 not === 200
  console.log('GET body: ', response.content);
} catch (e) {
  console.log('Network error:', e.message);
  // PERMISSION_DENIED → enable network access in Affinity Settings
  // INVALID_HANDLE → server could not be reached (bad URL or no
  // connectivity)
}

// POST with JSON body
try {
  const req = HttpRequest.create('https://httpbin.org/post',
    RequestMethod.Post);
  req.setHeaderValue('Content-Type', 'application/json');
  const result = req['do']();
  console.log('POST status:', result.response.statusCode); // statusCode

```

```
is an enum object – use == 200 not === 200
  console.log('POST body: ', result.response.content);
} catch (e) {
  console.log('Network error:', e.message);
}
```


Phase 4 — Advanced

PART

IV

Phase 4 — Advanced

14

Phase 3 gave you the tools to build scripts that interact with users, process text, and produce output. Phase 4 goes deeper — into the capabilities of the SDK that require more careful handling, reward more careful thinking, and unlock the most powerful automation possibilities the platform offers.

This phase covers six areas that sit at the edge of what scripts can do: AI-powered image operations, layer effects and adjustments, production-grade async patterns, a survey of the SDK modules, direct pixel manipulation, and programmatic font and typography control. None of these are prerequisites for writing useful scripts — you could spend months working productively with only Phases 1 through 3. But each lesson here opens a category of work that simply is not possible without it.

The AI commands lesson is particularly significant. Affinity’s generative capabilities — image generation, inpainting, background removal, depth mapping, colourisation — are available directly from JavaScript without any external API keys, quotas, or network dependencies. Scripts can drive them programmatically, chain them with export pipelines, and present them through dialogs in ways the standard UI does not support.

Pixel processing is the other lesson that unlocks a qualitatively different class of scripts. Once you have direct read/write access to individual pixels, any image processing algorithm you can express mathematically becomes implementable: custom colour grading, histogram analysis, procedural texture generation, per-channel adjustments, and effects that depend on spatial relationships between pixels.

The pro patterns lesson ties the phase together — it covers the async techniques, error handling, and external communication that production scripts need when they move from personal experiments to tools used by a team.

14.1 Lessons

Lesson	Topic	Key skill you'll gain
10 — AI commands	<code>generateImage</code> , <code>removeBackground</code> , <code>selectSubject</code>	Driving Affinity's AI features from scripts
11 — Layer effects	Shadows, glows, adjustments, filters	Applying and batching visual effects across many nodes
12 — Pro patterns	Async pipelines, networking, timers, error handling	Writing scripts that are robust enough for production use
13 — Special modules	Pixel access, fonts, brushes, buffers, metadata	Using the full breadth of the SDK's module library
14 — Pixel processing	<code>PixelReaderWriterRGBA8</code> , <code>Bitmap</code> , <code>NodeRenderingEngine</code>	Direct per-pixel read and write on any raster layer
15 — Fonts & typography	<code>FontFamily</code> , <code>PANOSE</code> , font distance, typography automation	Inspecting, classifying, and applying fonts programmatically
16 — Raster adjustments	<code>BrightnessContrast</code> , <code>Levels</code> , <code>Curves</code> , <code>HSL</code> , all 19 types	Creating and modifying non-destructive adjustment layers
17 — Align & distribute	<code>GroupTransformData</code> , <code>GroupTransformType</code> , <code>GroupTransformAnchor</code>	Aligning and distributing nodes on any axis programmatically

14.2 What to expect

By the end of Phase 4 you should be able to write a script that:

- Generates an image from a text prompt and exports it in one pipeline
- Applies a consistent set of layer effects across every node in a document
- Processes every pixel of a raster layer to implement a custom image filter
- Enumerates all installed fonts, classifies them by type, and finds the closest substitute for a missing typeface
- Handles save failures on unsaved documents, network errors, and export failures gracefully
- Chains multiple async operations — `execute`, `export`, `save` — without blocking the UI

You will also encounter the SDK's hardest-won lessons in this phase: that `LineType.Dash` is required before dashes render, that `req['do']()` is the only safe way to execute a network request, that `mustSaveAs` must be checked before any save operation, and that pixel alpha is always `px.alpha` never `px.a`. These are not intuitive, they do not produce helpful error messages, and they are documented here precisely because they cost time to discover independently.

Lesson 10 — AI Commands

15

Affinity 3.2 exposes the application's AI capabilities directly to scripts. This lesson covers all seven AI operations: generating images from prompts, generative inpainting, background removal, subject selection, depth map generation, colourisation, and image tracing. It shows how to call them correctly and how to build practical workflows that combine AI generation with the rest of the SDK.

These commands let you build things that would otherwise require external AI APIs, credentials, and network round-trips. Because they run natively inside Affinity, they respect the document model, integrate with the undo stack, and work on whatever the user has open without any additional setup. For automating creative workflows, they represent a significant capability.

There is a setup requirement that every AI command shares and that is easy to miss: the command must be called with a correct spread and selection context, and it must be awaited properly. Scripts that call AI commands synchronously, or that have no active spread set, fail without a clear error message. The sequencing — establish spread, establish selection, then call the AI command with `await` — is documented here so you do not need to discover it through failed runs.

The AI commands also vary in what they operate on. `generateImage` creates a new raster node from a text prompt and requires no existing selection. `generativeEditImage` fills a masked area of an existing raster layer — it requires both a selection and a valid subject region. `removeBackground` and `selectSubject` operate on the selected raster node. `detectDepth` and `colourise` each produce a new layer from an existing one. `imageTrace` converts a raster to a

Note
AI features require a paid Canva subscription. These APIs only work when the user is signed into a Canva account with an active subscription. Scripts that call AI commands silently fail or return errors without one.

vector PolyCurve. Knowing which commands consume an existing node and which produce a new one prevents a class of errors that produce either empty results or operations on the wrong layer.

By the end of this lesson you will be able to call all seven AI commands correctly, chain them with export pipelines, and build dialog-driven workflows that let users configure the generation parameters before running. The combination of AI generation, command execution, and export — all in a single script — is one of the most powerful things the Affinity SDK makes possible.

15.1 What this lesson covers

- `generateImage(prompt)` — AI image generation
- `generativeEditImage(prompt)` — inpainting / generative fill
- `removeBackground()` — background removal
- `selectSubject()` — smart subject selection
- `detectDepth()` — depth map generation
- `colourise()` — grayscale to colour
- `imageTrace()` — raster to vector
- Running AI commands correctly (spread, selection, async)

15.2 Why use the SDK AI commands?

Note
Prefer these over external AI APIs. The SDK AI commands are native to Affinity, respect the edit model, integrate with the undo stack, and don't require network credentials.

15.3 Generate an image from a prompt

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No doc.');
} else {
  // Set the spread first — generateImage places onto the CURRENT spread
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  doc.generateImage('A misty mountain landscape at dawn, digital art style');
  console.log('Image generation started.');
```

Note
Set currentSpread explicitly before calling generateImage. The image is placed on whichever spread is current at the time the command runs.

15.4 Generative edit (inpainting)

Select the region you want to edit (use a raster selection or select a layer), then:

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const doc = Document.current;
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

// Requires a raster selection or selected image node
doc.generativeEditImage('Replace with a clear blue sky');
```

15.5 Remove background

`doc.removeBackground()` strips the background from a selected image node, leaving the subject on a transparent layer.

```
'use strict';
const { DocumentCommand } = require('/commands');
const { Document } = require('/document');

const doc = Document.current;
const node = doc.selection.nodes.filter(n => n.isImageNode || n.isRasterNode).first;
```

```

if (!node) {
  console.log('Select an image node first.');
```

```

} else {
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));
  doc.removeBackground();
  console.log('Background removal started.');
```

```

}

```

15.6 Select subject

AI-powered selection of the main subject in the current image:

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  doc.selectSubject();
  console.log('Subject selection applied.');
```

```

  // The raster selection is now set – you can use it for further
  operations
}

```

15.7 Detect depth

Generates a depth map from the selected image:

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  doc.detectDepth();
  console.log('Depth detection started.');
```

```

}

```

15.8 Colourise

Converts a grayscale image to colour:

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  doc.colourise();
  console.log('Colourisation started.');
```

```

}

```

15.9 Image trace (raster to vector)

Converts the selected raster/image node into vector curves:

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else if (doc.selection.nodes.isEmpty) { console.log('Select a raster or
image node first.');
```

```

} else {
  const edgeThreshold = 0.5; // 0.0–1.0 – how sharp edges need
  to be detected
  const curveFittingTolerance = 2.0; // lower = tighter fit, more nodes

  // doc.imageTrace is the convenience form
  doc.imageTrace(edgeThreshold, curveFittingTolerance, doc.selection);
}

```

```
    console.log('Image trace complete.');
```

```
  }
```

15.10 Batch image generation

Generate one image per prompt, each on its own artboard:

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');

const PROMPTS = [
  'A serene forest path in autumn',
  'A futuristic cityscape at night',
  'An abstract geometric pattern in blue and gold',
];

const doc = Document.current;
if (!doc) { console.log('No doc.');
```

```
  }
```

15.11 Using AI commands with doc.promises (async)

For long-running AI operations, use `doc.promises` to avoid blocking the script (e.g. when chaining AI generation with export or document operations).

```
'use strict';
const { DocumentCommand } = require('/commands');
async function generateAndExport(prompt, outputPath) {
  const { DocumentPromises, FileExportOptions, FileExportArea } =
  require('/document');
  const docP = await DocumentPromises.getCurrent();
  if (!docP) return;

  const doc = docP.document;
  await
docP.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  doc.generateImage(prompt);

  // Export after generation
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForCurrentSpread();
  const records = await docP.export(outputPath, opts, area, null);
  console.log(records.all[0]?.isSuccess ? 'Done.' : 'Failed.');
```

```
  }
```

```
// Call with an example prompt and Desktop output path
```

```
// const { app } = require('/application');
```

```
// generateAndExport('Abstract blue waves', app.userDesktopPath + '/' +
generated.png');
```

15.12 Dialog-driven image generator

Combine Dialog (Lesson 08) with AI commands for a practical tool:

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');

const doc = Document.current;
if (!doc) { console.log('No doc.');
```

```
  }
```

```
    const dlg = Dialog.create('Generate Image');
```

```

const col = dlg.addColumn();
const grp = col.addGroup('Prompt');

const promptBox = grp.addTextBox('Describe the image', '');
promptBox.isMultiline = true;
promptBox.rowSpan = 4;

const result = dlg.runModal();

if (result == DialogResult.Ok) {
  const prompt = promptBox.text.trim();
  if (!prompt) {
    console.log('No prompt entered.');
```

15.13 Key gotchas

Always set currentSpread before AI commands. generateImage, generativeEditImage, removeBackground, and detectDepth all operate on the current spread. Forgetting this means the command runs on the wrong page.

AI commands are async under the hood. They trigger processing but return before it finishes. Use executeCommandAsync or doc.promises.executeCommand if you need to chain operations after completion.

selectSubject and removeBackground require an image node to be selected. They fail silently if the selection is wrong.

15.14 Exercises

15.14.1 Exercise 10.1 — Single image generator

Build a dialog with a text box for a prompt. Generate an image on the current spread when the user confirms.

15.14.2 Exercise 10.2 — Background removal batch

Find all image nodes in the selection. Remove the background from each one.

15.14.3 Exercise 10.3 — Multi-prompt generator

Accept a multi-line prompt box where each line is a separate prompt. Generate one image per line.

15.14.4 Exercise 10.4 — Trace and style

Select a raster image. Run imageTrace() to vectorise it, then apply a solid colour fill to the result.

15.15 SDK modules used in this lesson

- /commands — DocumentCommand
- /document — Document, DocumentPromises
- /dialog — Dialog, DialogResult

15.16 See also

- Lesson 09 — Export & Files
- Lesson 11 — Layer Effects
- examples/

15.17 Examples**15.17.1 Dialog-driven image generator**

```

'use strict';

// Example 10a — Dialog-driven image generator
// Lesson 10: AI Commands
//
// Shows a prompt dialog and generates an image on the current spread.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const dlg = Dialog.create('AI Image Generator');
  const col = dlg.addColumn();

  const grp1 = col.addGroup('Prompt');
  const promptBox = grp1.addTextBox('Describe the image you want', '');
  promptBox.isMultiline = true;
  promptBox.rowSpan = 5;

  const grp2 = col.addGroup('Options');
  const onNewSpread = grp2.addSwitch('Place on new spread', false);

  const result = dlg.runModal();

  if (result == DialogResult.Ok) {
    const prompt = promptBox.text.trim();

    if (!prompt) {
      console.log('Enter a prompt and try again.')}
    } else {
      // Always set current spread before AI commands
      doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

      doc.generateImage(prompt);
      console.log('Image generation started for prompt:');
      console.log(' ', prompt);
    }
  } else {
    console.log('Cancelled.')}
  }
}

```

Lesson 11 — Layer Effects & Adjustments

16

Layer effects — shadows, glows, outlines, bevels, blurs — are among the most visually impactful properties you can set from a script. This lesson covers the full effect system: how to apply and configure individual effects, how to batch-apply them across many nodes, how to apply adjustment layers, and how to use destructive filters on raster content.

Scripting effects is valuable anywhere you need consistent visual treatment across many layers: applying matching shadows to every icon in a set, adding a bloom effect to all highlight layers, or clearing effects from every node before re-exporting. The SDK gives you full programmatic control over the same effect parameters you would otherwise adjust manually in the Effects panel.

The effect system is structured around three distinct categories that use different application mechanisms. Live effects — shadows, glows, outlines, and bevels — are applied via individual `DocumentCommand` calls and persist as non-destructive properties on the node. Adjustment layers — Levels, Curves, HSL, Brightness/Contrast — are created programmatically via `*AdjustmentRasterNodeDefinition` factory classes and added as live, non-destructive child nodes (see Lesson 16 for the full workflow). Destructive filters — Blur, Noise, Vignette, Bloom — are applied directly to raster pixel data and cannot be removed without undo.

There is also an important pattern for working with live effects: the `enableIfDisabled` flag. When you apply an effect using `createSetEffects()`, you are replacing the entire effects list on the node, not adding to it. If the node already has effects you want to keep, you must first read the existing list, append to it, and then write the whole thing back. The `enableIfDisabled` pattern handles the case where the effect type already exists — it updates the existing entry

rather than duplicating it. This lesson shows both the naive approach and the correct one, so you understand when each applies.

By the end of this lesson you will be able to apply live effects to any node, set parameters on existing adjustment layers, apply destructive filters to raster content, and clear or replace effects in bulk.

16.1 What this lesson covers

- Outer shadow, outer glow, inner shadow, inner glow
- Colour overlay, bevel and emboss
- Outline (stroke effect) and Gaussian blur layer effect
- Adjustment layer parameters (set on existing adjustment nodes)
- Destructive filters on raster nodes (`create*Filter` factory methods)
- The `enableIfDisabled` flag and index parameter
- Removing all effects at once

16.2 Effect categories

The SDK splits effects into three categories:

Category	Examples	How they're applied
Layer effects	Outer/Inner Shadow, Outer/Inner Glow, Outline, Bevel & Emboss, Colour Overlay, Gaussian Blur	Individual <code>doc.executeCommand</code> calls per property using <code>Selection.create(doc, node)</code>
Adjustment layers	Levels, Curves, HSL, Vibrance	Created via <code>*AdjustmentRasterNodeDefinition</code> factories and/or modified via parameter commands
Destructive filters	Blur, Noise, Vignette, Bloom	Raster nodes only — use <code>create*Filter</code> factory methods

16.3 Outer shadow

Apply individual effect properties via `DocumentCommand.createSetOuterShadow*` factories — each command addresses one parameter.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const doc = Document.current;
const node = doc.selection.nodes.first;

if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);

  // Effect commands must be executed individually —
  // CompoundCommandBuilder
  // does not work with layer effect commands (silent no-op).

  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectColour(sel,
    RGBA8(0, 0, 0, 180), 0, true));

  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOffsetAngle(sel,
    6, 315, 0, true));
```

```

doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
0, 8, true));

doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel,
0.7, 0, true));

    console.log('Shadow applied.');
```

16.4 The `enableIfDisabled` parameter

Most effect commands accept `enableIfDisabled` as a final parameter. When `true`, the effect is automatically added to the node if it doesn't already have one.

```

// Snippet
// If there's no outer shadow, this creates one AND sets its radius
DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 10, true);

// If there's no outer shadow, this does nothing
DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 10, false);
```

Always pass `true` when your script is adding effects from scratch.

16.5 The index parameter

Effects that can be duplicated (OuterShadow, InnerShadow, Outline, ColourOverlay, GradientOverlay) use an index to address each instance. The first effect is index 0.

```

// Snippet
// First shadow
DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 8, true);

// Duplicate it first, then address index 1
compound.addCommand(DocumentCommand.createDuplicateOuterShadowLayerEffect(sel,
0));
compound.addCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
1, 20, false));
```

16.6 Outline (stroke effect)

Outline adds a stroke as a layer effect (separate from the line style stroke on vector nodes).

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { RGBA8 } = require('/colours');
const { Selection } = require('/selections');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (!node) { console.log('Select something first.');
```

16.7 Inner glow

Applies a glow effect inside the edges of the layer — uses colour, radius, opacity, and intensity.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
```

```

const { RGBA8 } = require('/colours');
const { Selection } = require('/selections');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectColour(sel,
    RGBA8(255, 200, 0, 255), true)); // bright yellow

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectRadius(sel,
    50, true)); // 50px radius

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectOpacity(sel,
    1.0, true)); // full opacity

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectIntensity(sel,
    1.0, true)); // full intensity
}

```

16.8 Gaussian blur (layer effect)

A live blur effect applied to the entire layer — non-destructive and adjustable after application.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetGaussianBlurLayerEffectRadius(sel,
    30, true)); // 30px — clearly visible

  doc.executeCommand(DocumentCommand.createSetGaussianBlurLayerEffectOpacity(sel,
    1.0, true));
}

```

16.9 Outer glow

A glow effect radiating outward from the layer edges — set colour, radius, opacity, and intensity.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectColour(sel,
    RGBA8(255, 230, 100, 255), true));

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectRadius(sel,
    25, true));

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectOpacity(sel,
    0.9, true));

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectIntensity(sel,
    0.8, true));
}

```

```

    console.log('Outer glow applied.');
```

```

}
```

16.10 Inner shadow

Inner shadow uses an index (like outer shadow) and adds an intensity parameter.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
    const sel = Selection.create(doc, node);

    doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectColour(sel,
    RGBA8(0, 0, 0, 200), 0, true));

    doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectOffsetAngle(sel,
    8, 315, 0, true));

    doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectRadius(sel,
    0, 15, true));

    doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectOpacity(sel,
    0, 0.7, true));

    doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectIntensity(sel,
    0, 0.5, true));
    console.log('Inner shadow applied.');
```

```

}
```

16.11 Colour overlay

Colour overlay tints the entire layer with a solid colour at a given blend mode.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
    const sel = Selection.create(doc, node);

    doc.executeCommand(DocumentCommand.createSetColourOverlayLayerEffectColour(sel,
    RGBA8(255, 0, 100, 255), 0, true));

    doc.executeCommand(DocumentCommand.createSetColourOverlayLayerEffectOpacity(sel,
    0, 0.5, true));
    console.log('Colour overlay applied.');
```

```

}
```

16.12 Bevel and emboss

Bevel and emboss adds a 3D lighting effect. Unlike indexed effects, there is only one bevel per layer.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
    const sel = Selection.create(doc, node);

    doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectRadius(sel,
```

```

10, true));
doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectDepth(sel,
2.0, true));
doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectOpacity(sel,
0.8, true));
doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectDirection(sel,
135, 45, true)); // azimuth, elevation
console.log('Bevel and emboss applied.');
```

16.13 Remove all effects

`createRemoveAllLayerEffects` clears every live effect from the selected node at once.

```

// Snippet – assumes doc and sel are already defined
doc.executeCommand(DocumentCommand.createRemoveAllLayerEffects(sel));
console.log('All effects removed.');
```

16.14 Gradient overlay

Gradient overlay applies a gradient fill on top of the layer. Like `ColourOverlay` and `Outline` it is indexed — you can stack multiple gradient overlays.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { GradientFill } = require('/fills');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const sel = Selection.create(doc, node);

  // Enable a gradient overlay at index 0 (creates it if not present)
  const gf = GradientFill.createDefault(); // black-to-white linear
  gradient

  doc.executeCommand(DocumentCommand.createSetGradientOverlayLayerEffectFill(sel,
gf, true));

  // Adjust blend mode and opacity
  doc.executeCommand(DocumentCommand.createSetGradientOverlayLayerEffectBlendMode(sel,
doc.blendMode, 0, true));

  doc.executeCommand(DocumentCommand.createSetGradientOverlayLayerEffectOpacity(sel,
0.6, 0, true));

  console.log('Gradient overlay applied.');
```

Remove a specific gradient overlay with `createRemoveGradientOverlayLayerEffect(sel, index)`.

16.14.1 GradientOverlayLayerEffect.calculateTransform

A static helper for computing the fill transform for gradient overlay effects:

```

// Snippet
const { GradientOverlayLayerEffect } = require('/layereffects');
const xf = GradientOverlayLayerEffect.calculateTransform(1, 1, 0, 0);
// scaleX, scaleY, transformX, transformY → returns a Transform
```

`OutlineLayerEffect` has the same static helper: `OutlineLayerEffect.calculateTransform(scaleX, scaleY, transformX, transformY)`.

16.15 Phong bevel

Phong bevel is a lighting-based 3D bevel effect distinct from the flat Bevel & Emboss. It adds ambient, diffuse, and specular light parameters for a more realistic surface simulation.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const sel = Selection.create(doc, node);

  // Radius (bevel depth in pixels), opacity
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectRadius(sel,
    8, true));

  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectOpacity(sel,
    0.9, true));

  // Lighting parameters
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectAmbient(sel,
    0.2, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectDiffuse(sel,
    0.8, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectShininess(sel,
    40, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectSpecular(sel,
    0.5, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectDepth(sel,
    5.0, true));

  // Ambient and specular colours
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectAmbientColour(sel,
    RGBA8(255, 255, 255, 255), true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectSpecularColour(sel,
    RGBA8(255, 255, 220, 255), true));

  console.log('Phong bevel applied.');
```

Remove with `createRemovePhongBevelLayerEffect(sel)`.

Adjustment layer parameter commands modify an *existing* adjustment layer. Adjustment layers can also be created programmatically via their `*AdjustmentRasterNodeDefinition` factory classes — see **Lesson 16 — Raster Adjustments** for the full workflow. The snippet below shows how to modify parameters on an existing node:

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const Nodes = require('/nodes');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node) {
  console.log('Select a Brightness/Contrast adjustment layer first.');
```

```

} else if (!(node instanceof
Nodes.BrightnessContrastAdjustmentRasterNode)) {
  console.log('Selected node is not a Brightness/Contrast adjustment
layer.');
```

```

} else {
  const sel = Selection.create(doc, node);

doc.executeCommand(DocumentCommand.createSetBrightnessContrastAdjustmentParameters(sel,
{
  brightness: 0.5,    // -1.0 to 1.0
  contrast:   0.4,    // -1.0 to 1.0
  isLinear:   false
}));
console.log('Parameters set.');
```

```

}

```

① **Note**

Adjustment parameter commands require an adjustment node. Calling a parameter command on a shape, group, or `doc.selection` directly is a silent no-op. Always build the selection with `Selection.create(doc, node)` where `node` is the adjustment layer.

16.16 Destructive filters on raster nodes

Destructive filters modify pixel data directly and cannot be undone without the undo stack. They only work on raster nodes — applying them to a shape or group is a silent no-op.

There are two sets of filter commands:

- `createGaussianBlurFilter(sel, params)` — creates and applies the filter. Use this.
- `createSetGaussianBlurFilterParameters(sel, params)` — sets parameters on an *already existing* filter. Same pattern as adjustment layers — silent no-op if no filter exists yet.

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand, RasterSelectionLogicalOperation } = require('/commands');
const { HalftoneDotType, HalftoneScreenType } = require('affinity:dom');
```

```

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  // createGaussianBlurFilter — creates and applies in one step
doc.executeCommand(DocumentCommand.createGaussianBlurFilter(doc.selection,
{ radius: 20 }));

doc.executeCommand(DocumentCommand.createVignetteFilter(doc.selection, {
  radius: 0.6,
  intensity: 0.8
}));

doc.executeCommand(DocumentCommand.createAddNoiseFilter(doc.selection, {
  intensity: 0.3,
  monochromatic: false
}));

// Halftone filter
doc.executeCommand(DocumentCommand.createHalftoneFilter(doc.selection, {
  dotType: HalftoneDotType.Dot,
  screenType: HalftoneScreenType.Frequency,
  frequency: 20,
  angle: 45,
  dotSize: 0.5
}));

console.log('Destructive filters applied.');
```

```

}

```

① **Note**

Use `create*Filter` not `createSet*FilterParameters` to apply a new filter. The `createSet*` variants modify an existing filter and are silent no-ops if none exists yet.

16.17 Drawing scale & margins

Read page geometry and drawing scale properties from the current spread and document preset.

```
'use strict';
const { Document } = require('/document');
const { DrawingScale } = require('/drawingscale');

const doc    = Document.current;
const spread = doc.currentSpread;

// Margins – spread.marginsInterface returns undefined in the current SDK
release
// Use physicalRootPropertiesInterface for page geometry instead
const prpi = spread.physicalRootPropertiesInterface;
console.log('Page count:', prpi?.pageCount);
const box = prpi?.pageBoxInterface?.getPageBoundingBox(0);
console.log('Page box:', box);

// Drawing scale on the document preset
// IMPORTANT: pass a raw integer – NOT UnitType.* (those return undefined
// and cause INVALID_ARGS)
// 15 = Metres, 8 = Millimetres, 9 = Centimetres, 11 = Inches, 5 = Pixels
const defaults = DrawingScale.getDefaults(15); // 15 = Metres
console.log(`${defaults.length} standard metre scales available`);
for (const s of defaults.slice(0, 5)) {
  console.log(' ', s.toString(false, true, false));
}

// Create a custom scale
const myScale = DrawingScale.createFromIntegers(1, 200, true);
console.log('My scale:', myScale.toString(false, true, false));
```

16.18 Raster brush properties

The `/rasterbrush` module exposes raster brush dynamics. Access is via a brush handle from the application context (when a brush is active in the UI):

```
'use strict';
const { RasterBrush, BrushDynamic, BrushDynamicControllerType } =
  require('/rasterbrush');

// Once you have a brush handle (from app context):
// const brush = ...;

// All brush dynamic properties have .value, .variance,
// and .controllerType
// brush.size.value           – base diameter
// brush.flow.value          – flow rate (0-1)
// brush.opacity             – brush opacity (0-1)
// brush.spacing             – spacing between dabs
// brush.hardness.value       – edge hardness
// brush.angle.value         – brush angle
// brush.scatterX.value       – X scatter
// brush.scatterY.value       – Y scatter
// brush.hueShift.value       – hue shift (HSL)
// brush.satShift.value       – saturation shift
// brush.lumShift.value       – luminance shift
// brush.wetEdges            – wet edges on/off
// brush.blendMode            – blend mode enum
```

16.19 Key gotchas

create*Filter not createSet*FilterParameters to apply a new filter. `createSetGaussianBlurFilterParameters` modifies an existing filter – silent no-op if none exists. Use `createGaussianBlurFilter` to create and apply in one step.

Note
`RasterBrush.createDefault()` is not implemented. The only way to obtain a `RasterBrush` instance is from a brush handle when the C++ layer provides one from tool state. There is no direct SDK API to get the currently active brush.

Adjustment layers are created via static factories, not new. Use `*Adjustment.RasterNodeDefinition.create(params)` or `.createDefault()` — instantiating nodes directly with `new` throws `Invalid handle`. See Lesson 16 for the full workflow.

Effect commands do not work inside `CompoundCommandBuilder`. Wrapping effect commands in a compound command is a silent no-op — the effect appears in the panel but doesn't render. Execute each effect command individually with `doc.executeCommand`.

Use `Selection.create(doc, node)` not `doc.selection`. Effect commands require a per-node selection built with `Selection.create`. Passing `doc.selection` directly silently does nothing.

The argument order is `(sel, opacity, index, enableIfDisabled)` — not `(sel, index, opacity, enableIfDisabled)`. Passing the index value (0) as opacity makes the shadow fully transparent with no error or warning.

The index parameter. For effects that support multiple instances (shadows, outlines), always pass the index explicitly. Forget it and you'll address the wrong instance.

Check `adjustment_ranges` and `filter_ranges` docs in the SDK for valid parameter ranges before applying numeric values — out-of-range values may be clamped or rejected silently.

`removeAllLayerEffects` vs `removing one`. Use `createRemoveOuterShadowLayerEffect(sel, index)` to remove a specific effect; `createRemoveAllLayerEffects(sel)` removes everything.

ⓘ **Note**
`enableIfDisabled: true` for new effects. Without it, setting a property on a non-existent effect does nothing silently.

16.20 Exercises

16.20.1 Exercise 11.1 — Shadow batch

Apply a consistent drop shadow (6px offset, 45° angle, 8px blur, 70% opacity) to every selected node using a compound command.

16.20.2 Exercise 11.2 — Glow ring

Apply an outer glow and an inner glow to selected nodes. Make the outer glow amber and the inner glow white.

16.20.3 Exercise 11.3 — Effect cleaner

Write a script that removes all layer effects from every node in the document.

16.20.4 Exercise 11.4 — Adjustment stack

Select an image node. Apply a Brightness/Contrast adjustment and a Vibrance adjustment in sequence.

16.21 SDK modules used in this lesson

- `/commands` — `DocumentCommand`, `CompoundCommandBuilder`
- `/layereffects` — layer effect parameter types
- `/colours` — `RGBA8`
- `/drawingscale` — `DrawingScale`
- `/rasterbrush` — `RasterBrush`
- `adjustment_ranges` — valid parameter ranges for adjustments
- `filter_ranges` — valid parameter ranges for filters

16.22 See also

- Lesson 10 — AI Commands
- Lesson 12 — Pro Patterns
- `examples/`

16.23 Examples

16.23.1 DIAGNOSTIC – Shadow command tester

```
'use strict';

// DIAGNOSTIC – Shadow command tester
// Lesson 11: Layer Effects
//
// Applies each outer shadow property one at a time with console output,
// so you can see exactly which command produces a visible result and
// which doesn't.
//
// Select a single object and run. Check the canvas after each log line.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  const nodes = doc.selection.nodes;
  if (nodes.isEmpty) { console.log('Select one object and run again.')} }
  else {
    const node = nodes.first;
    const sel = Selection.create(doc, node);

    console.log('Running shadow diagnostic...');

    // 1. Colour – sets the shadow colour and creates the effect
    (enableIfDisabled=true)
    doc.executeCommand(
      DocumentCommand.createSetOuterShadowLayerEffectColour(sel,
        RGBA8(255, 0, 0, 255), 0, true)
    );
    console.log('1. Colour set (bright red, fully opaque) – check Effects
panel');

    // 2. Opacity – two argument order variants, trying index-last first
    doc.executeCommand(
      DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel, 1.0, 0,
true)
    // variant A: (sel, opacity, index, enable) – opacity=1.0, index=0
    );
    console.log('2a. Opacity variant A: (sel, 1.0, 0, true) – opacity
should be 100%');

    // 3. Radius – large value so it's unmistakable if it works
    doc.executeCommand(
      DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 20,
true)
    // as documented: (sel, index, radius, enable)
    );
    console.log('3. Radius set to 20px');

    // 4. Offset+Angle – large offset so it's unmistakable
    doc.executeCommand(
      DocumentCommand.createSetOuterShadowLayerEffectOffsetAngle(sel, 30,
315, 0, true)
    // as documented: (sel, offset, angle, index, enable) – 30px offset,
bottom-right
    );
    console.log('4. Offset 30px at 315 degrees – shadow should be clearly
visible bottom-right');

    console.log('Done. If shadow still invisible on canvas, try variant B
for opacity:');
  }
}
```

```

    doc.executeCommand(
      DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel, 0, 1.0,
true)
      // variant B: (sel, index, opacity, enable) – index=0, opacity=1.0
    );
    console.log('2b. Opacity variant B: (sel, 0, 1.0, true) applied');
  }
}

```

16.23.2 Shadow batch

```

'use strict';

// Example 11a – Shadow batch
// Lesson 11: Layer Effects
//
// Applies a consistent drop shadow to every selected node.
//
// Note: effect commands must be executed individually –
CompoundCommandBuilder
// does not work with layer effect commands (silent no-op).

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { RGBA8 } = require('/colours');

const SHADOW_COLOUR = RGBA8(0, 0, 0, 180);
const SHADOW_OFFSET = 6; // pixels
const SHADOW_ANGLE = 315; // degrees (315 = bottom-right)
const SHADOW_RADIUS = 8; // blur radius in pixels
const SHADOW_OPACITY = 0.7; // 0.0–1.0
const SHADOW_INDEX = 0; // first shadow slot

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const nodes = doc.selection.nodes;
  if (nodes.isEmpty) {
    console.log('Select some objects and run again.')}
  else {
    for (const node of nodes) {
      const sel = Selection.create(doc, node);

      doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectColour(sel,
SHADOW_COLOUR, SHADOW_INDEX, true));

      doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOffsetAngle(sel,
SHADOW_OFFSET, SHADOW_ANGLE, SHADOW_INDEX, true));

      doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
SHADOW_INDEX, SHADOW_RADIUS, true));

      doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel,
SHADOW_OPACITY, SHADOW_INDEX, true));
    }

    console.log('Drop shadow applied to ${nodes.length} node(s).')}
  }
}

```

Lesson 12 — Pro Patterns: 17

Async, Hints & Network

Once you can write scripts that work, the next step is writing scripts that work reliably in production — handling errors gracefully, chaining async operations safely, communicating with external services, and saving your scripts for reuse. This lesson covers the `doc.promises` API for async workflows, HTTP networking, timers, and the community hint system.

These patterns matter when scripts move from personal experiments to tools that other people use. A script that crashes on an unsaved document, silently ignores a failed export, or blocks the UI during a long operation is frustrating to use. The techniques in this lesson — promise chaining, `mustSaveAs` checks, error handling, and the hint pool — are what separate a finished tool from a fragile prototype.

The async story in the Affinity SDK has a specific shape that differs from typical Node.js async patterns. The `doc.promises` API wraps the most common document operations — `executeCommand`, `export`, `save`, `saveAs` — in Promises that can be chained with `await`. The synchronous equivalents (`doc.executeCommand`, `doc.export`) do not return promises and cannot be awaited. Mixing the two in the same async function — calling `doc.executeCommand()` synchronously inside an async function that also awaits other operations — is a common source of ordering bugs that are difficult to trace. This lesson establishes the correct pattern: either work fully synchronously, or use the `doc.promises` equivalents throughout.

Save handling deserves particular attention. New documents that have never been saved have no file path — `doc.mustSaveAs` is `true`, and calling `doc.promises.save()` on them will fail. The correct check is always to test `mustSaveAs` before saving and branch accordingly. Scripts that skip this check

work correctly on all documents the developer happens to test on, then fail unexpectedly for users who run them on freshly created files. This lesson makes the pattern explicit so you apply it by default, not by discovery.

The hint system — `search_sdk_hints` and `add_sdk_hint` — is a community knowledge base embedded in the SDK tooling. When you encounter a behaviour that is not in the documentation, searching the hint pool often surfaces a working pattern contributed by another developer. When you discover something new, contributing a hint makes it available to everyone. This lesson closes by showing both operations, because the habit of checking the hint pool before debugging from scratch is one of the highest-leverage practices in Affinity SDK development.

17.1 What this lesson covers

- The `doc.promises` API — full Promise-based workflow
- Chaining async operations safely
- Error handling in async scripts
- `network.js` — HTTP from scripts
- `timers.js` — timeouts and intervals
- `search_sdk_hints` — community knowledge base
- `add_sdk_hint` — contributing solutions back
- `save_script_to_library` — saving production scripts
- Building robust, production-grade scripts

17.2 The `doc.promises` API

Every document operation has a Promise-based equivalent under `doc.promises`. This is the preferred pattern for scripts that do multiple sequential operations.

```
'use strict';
const { Document, DocumentPromises } = require('/document');
const { app } = require('/application');

async function main() {
  // Get current document (Promise-based)
  const docP = await DocumentPromises.getCurrent();
  if (!docP) {
    console.log('No document open.');
    return;
  }

  const doc = docP.document;
  console.log('Document:', doc.title);

  // Save — new/unsaved documents need saveAs first
  try {
    if (doc.mustSaveAs) {
      // Document has never been saved — save to Desktop first
      const desktop = app.userDesktopPath;
      await docP.saveAs(`${desktop}/${doc.title || 'untitled'}.af`);
      console.log('Saved to Desktop.');
    } else {
      await docP.save();
      console.log('Saved.');
    }
  } catch (err) {
    console.log('Save failed:', err);
  }
}

main();
```

doc.mustSaveAs is true for unsaved documents. Calling docP.save() on a document that has never been saved throws SAVE_TO_TEMPORARY_ARCHIVE_ERROR. Always check doc.mustSaveAs and use docP.saveAs(path) with a Desktop path for new documents.

17.3 Processing and exporting the current document

```
'use strict';
const { DocumentPromises, FileExportOptions, FileExportArea } = require('/
document');
const { app } = require('/application');

async function main() {
  const docP = await DocumentPromises.getCurrent();
  if (!docP) { console.log('No document open.');
```

`return; }`

```

  const doc = docP.document;
  const desk = app.userDesktopPath;

  // Export as PNG to the Desktop
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForWholeDocument();
  const outputPath = `${desk}/${doc.title}.png`;

  try {
    const records = await docP.export(outputPath, opts, area, null);
    const rec = records.all[0];
    console.log(rec?.isSuccess ? 'Exported: ' + outputPath : 'Export
failed.');


```

 } catch (err) {
 console.log('Export error:', err);
 }
}

main();
```


```

17.4 Full async pipeline

Load → generate AI image → export → save → close, all chained safely:

```
'use strict';
const { DocumentPromises, FileExportOptions, FileExportArea } = require('/
document');
const { DocumentCommand } = require('/commands');
const { app } = require('/application');

async function generateAndExport(prompt) {
  const desktop = app.userDesktopPath;
  const docP = await DocumentPromises.getCurrent();
  if (!docP) throw new Error('No document open.');


```

 const doc = docP.document;

 // Set spread before AI command
 await docP.executeCommand(
 DocumentCommand.createSetCurrentSpread(doc.currentSpread)
);

 // Generate image (async – resolves when generation starts)
 doc.generateImage(prompt);

 // Export
 const opts = FileExportOptions.createWithPresetName('PNG');
 const area = FileExportArea.createForCurrentSpread();
 const path = `${desktop}/generated.png`;
 const records = await docP.export(path, opts, area, null);

 // Save – check mustSaveAs for unsaved documents
 if (doc.mustSaveAs) {
```


```

```

        await docP.saveAs(`${desktop}/generated.af`);
    } else {
        await docP.save();
    }

    return records.all[0]?.path ?? null;
}

async function main() {
    try {
        const path = await generateAndExport('Abstract blue waves');
        console.log('Done:', path);
    } catch (err) {
        console.log('Pipeline failed:', err.message ?? err);
    }
}

main();

```

17.5 Network requests

① **Note**
Network access requires permission. Enable it in Affinity Settings → Scripts.

HttpRequest.create() throws PERMISSION_DENIED if not enabled.

The response body is in result.response.content (not .body).

```

'use strict';
const { HttpRequest, RequestMethod } = require('/network');

// GET
try {
    const req = HttpRequest.create('https://jsonplaceholder.typicode.com/todos/1', RequestMethod.Get);
    req.setTimeoutInSec(10);
    const result = req['do']();
    if (result.response.statusCode === 200) {
        const data = JSON.parse(result.response.content);
        console.log('Title:', data.title);
    }
} catch (err) {
    console.log('Network error:', err.message);
    // If PERMISSION_DENIED: enable network access in Affinity Settings
}

// POST with JSON body
try {
    const req = HttpRequest.create('https://jsonplaceholder.typicode.com/todos', RequestMethod.Post);
    req.setHeaderValue('Content-Type', 'application/json');
    const result = req['do']();
    console.log('Response status:', result.response.statusCode);
    console.log('Response body: ', result.response.content);
} catch (err) {
    console.log('Network error:', err.message);
}

```

17.6 Remote data → shapes pattern

Fetch data from an API and create one shape per item:

```

'use strict';
const { Document } = require('/document');
const { CompoundCommandBuilder } = require('/commands');
const { HttpRequest, RequestMethod } = require('/network');
const { RGBA8 } = require('/colours');

const doc = Document.current;
if (!doc) { console.log('No doc.');
} else {
    let items = [];
    try {
        const req = HttpRequest.create('https://jsonplaceholder.typicode.com/todos?_limit=5', RequestMethod.Get);
        req.setTimeoutInSec(10);

```

```

    const result = req['do']();
    if (result.response.statusCode == 200) {
        items = JSON.parse(result.response.content);
    }
} catch(e) {
    console.log('Network error:', e.message);
    // If PERMISSION_DENIED: enable network access in Affinity Settings
}

console.log(`Fetched ${items.length} items.`);

items.forEach((item, i) => {
    const colour = item.completed ? RGBA8(80, 200, 120, 255) : RGBA8(220,
80, 60, 255);
    // ... build and add shape nodes here
    // See Lesson 06 for AddChildNodesCommandBuilder usage
    console.log(`${i + 1}. ${item.title} - ${item.completed ? 'done' :
'pending'}`);
});
}

```

17.7 Timers

```

'use strict';
const { setTimeout, setInterval } = require('/timers');

// setTimeout(delay, callback) - delay in milliseconds, callback receives
errorCode
setTimeout(2000, (err) => {
    if (err) { console.log('Timer error:', err); return; }
    console.log('Ran after 2 seconds.');
```

```

});

// setInterval(delay, callback) - repeats until timer.cancel() is called
let count = 0;
const timer = setInterval(1000, (err) => {
    if (err) return; // err is set when cancelled
    count++;
    console.log('Tick', count);
    if (count >= 3) timer.cancel();
});

```

17.8 search_sdk_hints – community knowledge base

Before experimenting on a hard problem, always search hints first:

```

// Via the MCP tool (when using Claude with Affinity):
search_sdk_hints("How to replace text in a frame without appending")
search_sdk_hints("TextSelection end - 1")
search_sdk_hints("setCurrentSpread clears selection")

```

This queries a global pool of solved problems from other sessions. It often returns exactly the gotcha you're about to spend an hour debugging.

17.9 add_sdk_hint – contributing back

When you solve a novel problem through experimentation, add a hint:

```

// Via the MCP tool:
add_sdk_hint("To replace all text in a TextNode, create a TextSelection
with
end = storyRange.end - 1. Using storyRange.end causes text to append
rather
than replace.")

```

Your hint will be visible to all future sessions.

17.10 save_script_to_library

When a script is working reliably and you want to keep it, use the Affinity_save_script_to_library MCP tool to persist it to the script library.

Note
Timers only run while the script engine is active. Do not use them for deferred saves or long idle waits — the engine may not be running.

The library stores scripts in `scripts.propcol` — a proprietary binary file. They are **not** accessible as `.js` files from within scripts at runtime.

ⓘ **Note**
There is no Save to Library button in the Scripts panel UI. Library saving is only available via the MCP server for now. This may change in a future release.

17.11 Production script checklist

Before shipping a script for others to use:

- [] Guard against no document open (`if (!doc) return`)
- [] Guard against empty selection (`if (sel.nodes.isEmpty) return`)
- [] Use compound commands for multi-step changes (one undo step)
- [] Capture `doc.selection` BEFORE any `setCurrentSpread` call
- [] Wrap file operations in `try/catch`
- [] Log success/failure clearly with `console.log`
- [] Use `app.userDesktopPath` for all file paths
- [] Test with a document that has multiple spreads
- [] Test with nothing selected (should fail gracefully)
- [] Save to library when stable

17.12 Key gotchas

Always call `main()` at the bottom. Async functions are not auto-invoked.

`setTimeout(delay, callback)` — delay comes first. This is the opposite of Node/browser where the callback is first. Passing a callback as the first argument produces `TypeError: expected parameter callback to be Function` from `waitAsync`. The callback receives an `errorCode` which is set on cancellation.

`timer.cancel()` not `clearInterval(timer)`. The SDK timer returns an object with a `.cancel()` method. There is no global `clearInterval`.

`generate_image` is fire-and-forget. It starts the generation but returns immediately. Chain with `executeCommandAsync` callback if you need to act on the result.

>`search_sdk_hints` before experimenting. Many subtle bugs (selection cleared by `setCurrentSpread`, end-1 rule, `Dialog.Ok` casing) are already documented. Search first.

17.13 Exercises

17.13.1 Exercise 12.1 — Async pipeline

Write a fully async script using `doc.promises` that: loads the current document, exports it as PDF, then saves it — with proper error handling at each step.

17.13.2 Exercise 12.2 — Remote shapes

Fetch a JSON array from `https://jsonplaceholder.typicode.com/todos?_limit=8`. For each item, log its title and completion status. (Bonus: create a coloured shape for each.)

17.13.3 Exercise 12.3 — Production tool

Take your favourite script from any earlier lesson and harden it with the production checklist above. Add a dialog for configuration and save it to the library.

17.13.4 Exercise 12.4 — Hint contribution

Solve a problem you encountered during the tutorial that isn't covered in the preamble hints. Formulate it as a clear, concise hint and note it down for contributing via `add_sdk_hint`.

17.14 SDK modules used in this lesson

- /document — DocumentPromises
- /network — HttpRequest, RequestMethod, HttpStatusCode
- /timers — setTimeout, setInterval, setImmediate
- /commands — DocumentCommand
- /fs — File, Buffer, Directory, FileSystemPromises

17.15 See also

- Lesson 11 — Layer Effects
- Tutorial README
- examples/

17.16 Examples**17.16.1 Full async pipeline**

```
'use strict';

// Example 12a — Full async pipeline
// Lesson 12: Pro Patterns
//
// Loads the current document, exports as PNG, then saves.
// Demonstrates the doc.promises pattern with error handling.

const { Document, DocumentPromises, FileExportArea, FileExportOptions } =
  require('/document');
const { app } = require('/application');

async function main() {
  const desktop = app.userDesktopPath;

  // Step 1 — get current document
  let docP;
  try {
    docP = await DocumentPromises.getCurrent();
  } catch (err) {
    console.log('Could not get document:', err);
    return;
  }

  if (!docP) {
    console.log('No document is open.');
```

```
    console.log('Exported to:', rec.path);

    // Step 3 — save the document
    // New/unsaved documents need saveAs() first — save() throws
    SAVE_TO_TEMPORARY_ARCHIVE_ERROR
    try {
        if (docP.document.mustSaveAs) {
            await docP.saveAs(`${desktop}/${title}.af`);
        } else {
            await docP.save();
        }
        console.log('Document saved.');
    } catch (err) {
        console.log('Save failed:', err);
    }

    console.log('Pipeline complete.');
```

```
}

// Always call the async entry point at the bottom
main();
```

Lesson 13 — Special Modules: Pixel Access, Brushes & More

This lesson surveys a collection of SDK modules that extend the core API with less commonly discovered but powerful capabilities: direct pixel access, brush inspection, binary buffers, and document metadata. Each module is introduced with its key types, access patterns, and a practical example.

These modules unlock entire categories of scripts that the core API alone cannot support: pixel-level image processing, binary data handling, and document metadata inspection.

The survey format of this lesson reflects the nature of the content: these modules are each self-contained, they solve different problems, and they are rarely all needed in the same script. Rather than building a single narrative example that forces all modules together artificially, the lesson introduces each one on its own terms — what it contains, what problem it solves, the correct access pattern, and a minimal example. For the most significant module — pixel access — the full treatment continues in Lesson 14. Font enumeration and typography are covered in full in Lesson 15.

One module warrants extra attention here: `/pixelaccessor`. Direct pixel read/write is the most powerful capability in this lesson, and its access pattern is not obvious from the module name — you do not instantiate a `PixelAccessor` directly, but rather obtain one from a `Bitmap` via `bitmap.createPixelReaderWriter()`. Understanding this before you look at the code prevents the natural but wrong assumption that you are looking for a class constructor. This lesson

establishes the entry point clearly; Lesson 14 then covers the full pipeline in depth.

Use this lesson as a map. When a future script needs something you are not sure the SDK provides, this is the first place to look.

18.1 What this lesson covers

This lesson documents SDK modules covering direct pixel read/write access and the supporting library of brushes, buffers, and metadata. Font enumeration is covered in Lesson 15.

18.2 Modules at a glance

Module	What it adds
/pixelaccessor	Read and write individual pixels in any colour format
/fonts	Font enumeration — covered in full in Lesson 15
/rasterbrush	Inspect and modify raster brush properties
/vectorbrush	Vector brush access
/buffer	Binary buffer creation and manipulation
/rasterobject	Raster object wrapper (used by brushes)
/documentproperties	Document metadata — DPI, size, colour space
/drawingscale	Drawing scale (map scale) access
/taginterface	Per-node custom key-value tag storage
/marginsinterface	Margin information on spread nodes
/pageboxinterface	Page box (bleed, trim, slug) access
/artboardproperties	Artboard metadata

18.3 `/pixelaccessor` — Direct pixel read/write

Read and write individual pixels across all of Affinity's colour formats. The accessor class must match the bitmap's format exactly — mismatches throw `INVALID_ARGS`.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
if (!node) { console.log('Select a raster or image node.')}
else {
  const ri = node.rasterInterface;

  // Always check format before creating an accessor — mismatch throws
  INVALID_ARGS
```

```

if (String(ri.format) !== 'RGBA8') {
  console.log('Format is', ri.format, '- this example requires RGBA8.');
```

console.log('Open an RGBA8 image (File > New or a standard JPEG/PNG) and run again.');

```

} else {
  const bitmap = ri.createCompatibleBitmap(true); // true = copy contents
  const rw      = PixelReaderWriterRGBA8.create(bitmap);

  // Read a pixel — returns { r, g, b, alpha } all 0-255
  // Note: the alpha key is 'alpha', not 'a'
  const px = rw.readPixel(10, 10);
  console.log('Pixel at (10,10):', px.r, px.g, px.b, px.alpha);

  // Write a pixel
  rw.writePixel(10, 10, { r: 255, g: 0, b: 0, alpha: 255 });

  // Push changes back — see 13a-pixel-inverter.js for a full loop
  example
  doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
  bitmap));
  console.log('Pixel written.');
```

```

}
}

```

18.3.1 All supported pixel formats

Class	Format	Bits/channel	Notes
PixelReader RGBA8 / Pixel ReaderWriter RGBA8	RGBA	8-bit	Standard web/ screen
PixelReader RGBA16 / Pixel ReaderWriter RGBA16	RGBA	16-bit	High precision
PixelReader IA8 / Pixel ReaderWriter IA8	Intensity+Alpha	8-bit	Greyscale
PixelReader IA16 / Pixel ReaderWriter IA16	Intensity+Alpha	16-bit	Greyscale HD
PixelReader CMYKA8 / Pixel ReaderWriter CMYKA8	CMYK+Alpha	8-bit	Print
PixelReader LABA16 / Pixel ReaderWriter LABA16	L_a_b*+Alpha	16-bit	Perceptual
PixelReaderM8 / PixelReader WriterM8	Mask	8-bit	Single channel
PixelReader M16 / Pixel	Mask	16-bit	Single channel HD

Class	Format	Bits/channel	Notes
ReaderWriter M16			
PixelReaderRGB Auf / Pixel ReaderWriter RGBAuf	RGBA	Float	HDR
PixelReaderMf / PixelReader WriterMf	Mask	Float	HDR single channel

① **Note**
PixelReaderWriter
RGBA8.create(bitmap)
throws **INVALID_ARGS** if the
format isn't **RGBA8**. Always
check `node.raster`
`Interface.format` before cre-
ating an accessor. Use
`String(ri.format)` to com-
pare — the format value is
an enum object, not a plain
string.

① **Note**
Current limitation: `node.tag`
`Interface` returns undefined
— reading tags back at run-
time

is not yet exposed in the SDK.
Writing via `DocumentCommand`
works correctly.

Open the image as a document (File > Open) rather than placing it on a page. A placed image has `isImageNode: true` and `isRasterNode: undefined` — `createReplaceBitmap` silently does nothing on it. If you need to process a placed image in-place, rasterise it first via `DocumentCommand.createRasteriseObjects(doc.selection, false, false)`.

18.4 `/taginterface` — Per-node custom metadata

`TagInterface` lets you write arbitrary string key-value pairs to any node via `DocumentCommand`. These persist inside the `.af` file.

```
'use strict';
const { PredefinedTagKey } = require('/taginterface');
const { DocumentCommand } = require('/commands');
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select a node
first.')}
else {
  const sel = doc.selection;

  // Write a custom tag (persists in the file)
  doc.executeCommand(
    DocumentCommand.createSetTagValueForKey(sel, 'my-script-data', 'hello
world')
  );

  // Write a predefined tag
  doc.executeCommand(
    DocumentCommand.createSetTagValueForPredefinedKey(
      sel, PredefinedTagKey.ScriptLabel, 'my-label')
  );

  console.log('Tags written.')}
}
```

18.5 `/documentproperties` — Document metadata

```
'use strict';
const { Document } = require('/document');

// Document properties are accessed directly on the doc object
const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  console.log('DPI: ', doc.dpi);
  console.log('Width (px): ', doc.widthPixels);
  console.log('Height (px): ', doc.heightPixels);
  console.log('Format: ', doc.format);
}
```

```

    console.log('Colour profile:', doc.colourProfile);
}

```

18.6 `/drawingscale` — Drawing/map scale

For technical drawing and mapping workflows:

```

'use strict';
const { DrawingScale } = require('/drawingscale');

// Create a 1:100 scale
const scale = DrawingScale.createFromIntegers(1, 100, true);
console.log(scale.toString(false, true, false)); // "1 : 100"
console.log('Left:', scale.leftValue);
console.log('Right:', scale.rightValue);

// Get default scales for a unit
// IMPORTANT: pass a raw integer — NOT UnitType.* (those return undefined
// and cause INVALID_ARGS)
// 8 = Millimetres, 9 = Centimetres, 15 = Metres, 11 = Inches, 5 = Pixels
const defaults = DrawingScale.getDefaults(8); // 8 = Millimetres
for (const s of defaults) {
    console.log(s.toString(true, true, false));
}

// Identity scale (1:1)
const identity = DrawingScale.identity;

```

18.7 `/marginsinterface` — Spread margins

For now, page geometry can be accessed via `spread.physicalRootPropertiesInterface`:

```

'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
    const spread = doc.currentSpread;

    // Available page geometry
    const prpi = spread.physicalRootPropertiesInterface;
    const props = spread.physicalRootInterface?.physicalRootProperties;

    console.log('Page count:', prpi?.pageCount);

    // Page bounding box (in document pixels)
    const pbi = prpi?.pageBoxInterface;
    if (pbi) {
        const box = pbi.getPageBoundingBox(0); // index 0 = first page
        console.log('Page box:', box);
    }
}

```

18.8 `/rasterbrush` — Brush property access

For scripts that inspect or modify raster brush settings:

```

'use strict';
const { RasterBrush, BrushDynamic, BrushDynamicControllerType,
RasterBrushTextureMode } = require('/rasterbrush');

// Access brush from context (brush must be active)
// const brush = app.currentBrush; — access path TBC
// console.log('Size value:', brush.size.value);
// console.log('Opacity:', brush.opacity);
// console.log('Spacing:', brush.spacing);
// console.log('Blend mode:', brush.blendMode);

```

ⓘ **Note**
doc.documentProperties
does not exist as a property.
Document metadata is read

directly
from `doc.dpi`, `doc.widthPixels`, `doc.format`, etc. The `DocumentProperties`

export from `/commands` is an internal type used by `createSetDocumentProperties` — it

is not user-constructable in the current SDK release.

ⓘ **Note**
Current limitation: `spread.marginsInterface` returns undefined — the access path

is not yet wired up in the current SDK release. The `MarginsInterface` class is defined

with `hasMargins` and `useMargins` properties, but `MarginsInterfaceApi` has no exported

methods, making it inaccessible at runtime. This is expected to be resolved in a future

SDK update.

① **Note**

Note: The access path for the current raster brush from a script context is not yet

fully documented. Raster Brush objects are returned from other API calls. The class is fully functional once you have a handle.

18.9 `DocumentCommand` — full method reference

`commands.js` exposes 43 static factory methods across the following groups:

18.9.1 Macro recording/playback

```
// Snippet
DocumentCommand.createStartRecordingMacro()
DocumentCommand.createStopRecordingMacro()
DocumentCommand.createReplayMacro()
DocumentCommand.createClearMacro()
DocumentCommand.createImportMacro(path)
DocumentCommand.createExportMacro(path)
```

18.9.2 Raster selection commands

```
// Snippet
DocumentCommand.createRasterSelectAll()
DocumentCommand.createRasterDeselect()
DocumentCommand.createRasterInvertSelection()
DocumentCommand.createRasterReselect()
DocumentCommand.createSetRasterSelectionFromPolygon(polygon, operation,
antialias, feather) // replaces createRasterSelectPolygon
```

18.9.3 Opacity control (separate from visibility)

```
// Snippet
DocumentCommand.createSetOpacity(selection, opacity) // layer
opacity
DocumentCommand.createSetBrushFillOpacity(selection, opacity, options) //
fill opacity
DocumentCommand.createSetLineFillOpacity(selection, opacity, options) //
stroke opacity
```

18.9.4 Fill anchor control

```
// Snippet
DocumentCommand.createSetBrushFillIsAnchoredToSpread(selection, bool,
options)
DocumentCommand.createSetPenFillIsAnchoredToSpread(selection, bool,
options)
DocumentCommand.createSetTransparencyFillIsAnchoredToSpread(selection,
bool, options)
```

18.9.5 Tag metadata

```
// Snippet
DocumentCommand.createSetTagValueForKey(selection, key, value)
DocumentCommand.createSetTagValueForPredefinedKey(selection,
predefinedKey, value)
```

18.9.6 Blend options

```
// Snippet
DocumentCommand.createSetBlendGamma(selection, gamma)
DocumentCommand.createSetBlendRanges(selection, blendOptions)
DocumentCommand.createSetAntialiasingMode(selection, antialiasingMode)
```

18.9.7 Document-level

```
// Snippet
DocumentCommand.createSetDocumentProperties(documentProperties)
DocumentCommand.createConvertDocumentFormat(format)
DocumentCommand.createGroupTransform(selection, xDataOrNull, yDataOrNull)
DocumentCommand.createSetWindingMode(selection, windingOrder)
DocumentCommand.createSetExportConfig(selection, exportConfig)
DocumentCommand.createMoveNodes(selection, target, moveType,
childListType)
DocumentCommand.createHideSelection(selection)
DocumentCommand.createFlatten()
DocumentCommand.createMergeVisible()
DocumentCommand.createSplitCurve(selection, parametricDistance)
DocumentCommand.createInsertGlyph(selection, glyph)
```

18.9.8 Snapshots

```
// Snippet
DocumentCommand.createAddDocumentSnapshot(description)
DocumentCommand.createDeleteDocumentSnapshot(snapshot)
DocumentCommand.createRestoreDocumentSnapshot(snapshot)
DocumentCommand.createSetCurrentSnapshot(snapshot)
DocumentCommand.createSetCurrentSnapshotFromHistoryItem(historyItem)
```

18.9.9 History navigation

```
// Snippet
DocumentCommand.createSetHistoryIndex(index)
DocumentCommand.createCycleAlternateFutures(index)
```

18.10 `createSetEditable` — argument order

Selection is the first argument:

```
// Snippet
// CORRECT: selection first, editable flag second
DocumentCommand.createSetEditable(selection, true) // unlock
DocumentCommand.createSetEditable(selection, false) // lock
```

18.11 `AddChildNodesCommandBuilder` — insertion options

Setters for controlling insertion behaviour:

```
'use strict';
const { AddChildNodesCommandBuilder, InsertionMode } = require('./
commands');
const builder = AddChildNodesCommandBuilder.create();
builder.autoName = true; // auto-assign layer name
builder.forcePixelAlignment = true; // snap to pixel grid
builder.clearCurrentSelection = false; // don't deselect before adding
builder.appendCurrentSelection = true; // add to existing selection
builder.setInsertionTarget(targetNode); // insert relative to a
node
builder.setInsertionTargetSelection(targetSel); // insert relative to a
selection
builder.setInsertionMode(InsertionMode.After); // Before / After /
Into
```

`addContainerNode` adds a container node, and `createCommandAndReset` allows reusing the builder.

18.12 SDK native binding modules

Two native C++ binding modules underpin the pixel and brush APIs:

Module	Contains
<code>affinity:raster</code>	All <code>PixelReader*Api</code> and <code>PixelReaderWriter*Api</code> classes
<code>affinity:brushes</code>	<code>RasterBrushApi</code> , <code>BrushDynamicApi</code> , brush enums

18.13 Exercises

18.13.1 Exercise 13.1 — Pixel inverter

Select a raster node. Read every pixel and write back its inverted colour (255-r, 255-g, 255-b). Use `PixelReaderWriterRGBA8`.

18.13.2 Exercise 13.2 — Tag all selected nodes

Build a dialog asking for a key and a value. Apply that tag to every selected node using `createSetTagValueForKey`.

18.13.3 Exercise 13.3 — Macro recorder UI

Build a dialog with buttons to start recording, stop recording, replay, and clear a macro. Wire each to the corresponding DocumentCommand.

18.13.4 Exercise 13.4 — Raster selection from polygon

Create a rectangular raster selection programmatically using `createSetRasterSelectionFromPolygon` (replaces `createRasterSelectPolygon`), then invert it with `createRasterInvertSelection`.

18.14 New node types

Three new concrete node classes are available:

18.14.1 TableTextNode

A grid-based text node with cells separated by `HardBreakType.TableCell` glyphs. See Lesson 07 for the full pattern.

```
const { TableTextNodeDefinition } = require('/nodes');
const def = TableTextNodeDefinition.create({ x: 0, y: 0, width: 300,
height: 200 }, { width: 3, height: 3 });
doc.addNode(def);
```

18.14.2 PatternRasterNode

A raster node that tiles a repeating pattern. Useful for procedural fill effects.

```
const { PatternRasterNodeDefinition } = require('/nodes');
const def = PatternRasterNodeDefinition.createDefault();
```

18.14.3 EnclosureRasterNode

A raster node that wraps raster content in an enclosure. Less commonly used directly — typically created through other SDK operations.

```
const { EnclosureRasterNodeDefinition } = require('/nodes');
const def = EnclosureRasterNodeDefinition.createDefault();
```

All three use standard node patterns: `doc.addNode(def)`, `AddChildNodesCommandBuilder` for insertion targeting, and access to `rasterInterface` / `storyInterface` where applicable.

18.15 SDK modules used in this lesson

- `/pixelaccessor` — All `PixelReader` and `PixelReaderWriter` classes
- `/taginterface` — `TagInterface`, `PredefinedTagKey`
- `/rasterbrush` — `RasterBrush`, `BrushDynamic`
- `/drawingscale` — `DrawingScale`
- `/marginsinterface` — `MarginsInterface`
- `/documentproperties` — `DocumentProperties`
- `/commands` — updated `DocumentCommand`, `AddChildNodesCommandBuilder`
- `/nodes` — `TableTextNodeDefinition`, `PatternRasterNodeDefinition`, `EnclosureRasterNodeDefinition`

18.16 See also

- Lesson 03 — Node Model
- Lesson 04 — Command Pattern
- Lesson 14 — Pixel Processing
- `examples/`

18.17 Examples**18.17.1 Pixel inverter**

```
'use strict';

// Example 13a — Pixel inverter
// Lesson 13: New modules
```

```

//
// Inverts every pixel in the selected raster node.
// Demonstrates the PixelReaderWriterRGBA8 API.
//
// Open the image as a document (File > Open), not as a placed image on a
// page.
// Select the layer and run.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.');
  } else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;
    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '- this script requires RGBA8.
Open an RGBA8 image and run again.');
    } else {
      const bitmap = ri.createCompatibleBitmap(true); // copy contents
      const rw = PixelReaderWriterRGBA8.create(bitmap);

      console.log(`Inverting ${w}x${h} pixels...`);

      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = rw.readPixel(x, y);
          rw.writePixel(x, y, {
            r: 255 - px.r,
            g: 255 - px.g,
            b: 255 - px.b,
            alpha: px.alpha
          });
        }
      }

      doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
      console.log('Done - pixels inverted.');
    }
  }
}

```


Lesson 14 — Pixel Processing

Pixel processing gives you direct access to the raw colour data of any raster layer in the document. This lesson covers the full pipeline: getting a bitmap from a raster node, reading and writing individual pixels, creating bitmaps from scratch, loading images from disk, and rendering any node to a bitmap. It covers all available colour formats and shows how to use the `Buffer` class for raw binary data.

Pixel-level access unlocks image processing algorithms that the high-level SDK cannot express: custom colour grading, per-pixel analysis, procedural texture generation, histogram computation, and effects that depend on spatial relationships between pixels. If you can describe it mathematically, you can implement it here.

The pipeline has a specific shape that is easy to get wrong in two places. The first is the entry point: pixel access begins with a `Bitmap`, not with the raster node directly. You obtain a bitmap from a `RasterNode` via `node.getBitmap()`, from a file via `Bitmap.createFromFile()`, or by rendering any node to a new bitmap via `NodeRenderingEngine`. Attempting to read pixel data directly from a node — which is the natural first instinct — does not work. The second is alpha access: individual pixel objects expose their alpha channel as `px.alpha`, not `px.a`. This naming inconsistency with the `RGBA8` constructor (`r, g, b, alpha`) is easy to overlook, and code that reads `px.a` returns `undefined` silently, producing subtle bugs rather than an immediate error.

Performance is also a real concern in pixel loops. JavaScript running inside Affinity is not the fastest environment for iterating millions of pixels, and a naive loop over a 4000×3000 image without care for access patterns or early termination can make the application feel unresponsive. This lesson docu-

ments the patterns that keep pixel scripts acceptably fast: working on smaller bitmaps where possible, using typed access formats that reduce conversion overhead, and structuring loops to minimise repeated property lookups.

By the end of this lesson you will have a working mental model of the full pixel pipeline — from node to bitmap to pixel reader/writer and back — and you will have working examples of colour inversion, grayscale conversion, procedural generation, and histogram analysis. These examples are templates: once you understand the pipeline, any per-pixel algorithm you can express becomes straightforward to implement.

19.1 What this lesson covers

- What a pixel is and when to use pixel processing vs built-in filters
- The complete pixel pipeline: bitmap → pixel accessor → write back
- All 10 colour formats and how to choose the right one
- Reading and writing individual pixels
- Creating bitmaps from scratch and loading from file
- Rendering any node to a bitmap with `NodeRenderingEngine`
- The `Buffer` class for raw binary data
- Performance considerations for pixel loops
- Real-world patterns: histogram analysis, posterisation, procedural generation

19.2 What a pixel is in this context

When you open an image in Affinity, what you see on screen is a rendered representation. The underlying data is a rectangular grid of numbers — each cell in the grid is a pixel, and each pixel is a small set of channel values that describe its colour.

For an RGBA8 image — the most common format — each pixel has four channels: red, green, blue, and alpha (transparency). Each channel is an 8-bit integer: a whole number between 0 and 255. A pure red pixel is { `r: 255, g: 0, b: 0, alpha: 255` }. A 50% transparent mid-grey is { `r: 128, g: 128, b: 128, alpha: 128` }. That's all a pixel is — a small structure of numbers in a large grid.

Pixel processing means reading those numbers, computing new ones, and writing them back. Any operation you can express as a mathematical transformation on those channel values is something you can implement as a pixel script.

19.3 Why pixel processing rather than filters or adjustments

Affinity's built-in filters and adjustment layers cover a wide range of common operations — levels, curves, HSL, blur, noise — and they run fast because they execute in C++ on optimised code paths. For those operations, you should always prefer the built-in tools.

Pixel processing becomes the right choice when you need something the built-in tools cannot express:

- **Custom algorithms** — a specific colour grading formula, a specialised sharpening kernel, a per-pixel mathematical transform that isn't in the filter list.
- **Analysis without editing** — reading pixel data to compute statistics, detect clipping, measure average colour, or build a histogram. The built-in tools don't expose this data to scripts.
- **Conditional processing** — applying different transforms to different regions of the image based on pixel values. For example: desaturate only pixels where red dominates, or boost contrast only in shadow regions.

- **Procedural generation** — creating images from scratch using mathematical functions, noise algorithms, or data from outside the document.
- **Format-specific operations** — working directly in CMYK or LAB colour space without a conversion round-trip.

19.4 Why pixel processing is slow — and what to do about it

JavaScript is not a fast language for numerical loops. A 4000×3000 image has 12 million pixels. Visiting each one, calling `readPixel`, performing arithmetic, and calling `writePixel` takes measurably longer in JavaScript than the same operation in C++ — potentially several seconds for a large image.

This doesn't mean pixel processing is unusable, but it does mean you should think before writing a loop:

- **Do you need every pixel?** Analysis tasks often work well on a sampled grid — visiting every 4th pixel in each direction gives you 1/16th the work with results that are still statistically representative.
- **Do you need to write back?** Read-only analysis (histograms, colour statistics) doesn't need `createReplaceBitmap`. Skip the write-back entirely.
- **Can you work at a smaller size?** If the script's effect is not detail-sensitive, export a scaled-down version, process it, and re-import.
- **Is there a built-in alternative?** A Curves adjustment layer that approximates your formula will always be faster than a per-pixel script. Use pixel processing for what the SDK genuinely can't do otherwise.

19.5 Why pixel processing matters

Scripts can read raster metadata (width, height, format) and write pixels through high-level operations (filters, adjustments, AI). Pixel processing drops below all of that and gives you direct access to the numbers. You can implement any image processing algorithm you want, entirely in JavaScript, with results pushed back as undoable commands.

19.6 The pipeline

```
node.rasterInterface
  ↓
.createCompatibleBitmap(true) ← copy current pixels into a writable
bitmap
  ↓
PixelReaderWriterRGBA8.create(bitmap) ← create a typed accessor
  ↓
rw.readPixel(x, y) / rw.writePixel(x, y, pixel)
  ↓
DocumentCommand.createReplaceBitmap(selection, bitmap) ← push back
(undoable)
```

Every step is explicit. The bitmap is a detached copy — you never modify live document pixels directly. The final `createReplaceBitmap` command is what actually changes the document, and it sits in the undo stack like any other operation.

19.7 Choosing the right format

Before creating a pixel accessor, check the node's format:

```
'use strict';
const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
```

```

    console.log('Select a raster or image node first.');
```

```

  } else {
    const ri      = node.rasterInterface;
    const format = ri.format; // RasterFormat enum value
    console.log('Format:', format);
    console.log('Width: ', ri.width, 'Height:', ri.height);
  }
}

```

Then match the accessor class to the format:

RasterFormat value	Reader class	ReaderWriter class	Pixel object shape
RGBA8	PixelReaderRGBA8	PixelReaderWriterRGBA8	{ r, g, b, alpha } 0–255
RGBA16	PixelReaderRGBA16	PixelReaderWriterRGBA16	{ r, g, b, alpha } 0–65535
IA8	PixelReaderIA8	PixelReaderWriterIA8	{ i, alpha } 0–255
IA16	PixelReaderIA16	PixelReaderWriterIA16	{ i, alpha } 0–65535
CMYKA8	PixelReaderCMYKA8	PixelReaderWriterCMYKA8	{ c, m, y, k, alpha } 0–255
LABA16	PixelReaderLABA16	PixelReaderWriterLABA16	{ l, a, b, alpha }
M8	PixelReaderM8	PixelReaderWriterM8	{ m } 0–255
M16	PixelReaderM16	PixelReaderWriterM16	{ m } 0–65535
RGBAuf	PixelReaderRGBAuf	PixelReaderWriterRGBAuf	{ r, g, b, alpha } float
Mf	PixelReaderMf	PixelReaderWriterMf	{ m } float

Note
Mismatching format and accessor throws `INVALID_ARGS`.
 Always check `ri.format` before choosing your accessor class. The format is an enum object — use `String(ri.format)` to compare: `String(ri.format) === 'RGBA8'`.

19.8 Core pixel read/write pattern

The pipeline always follows the same five steps: get the raster interface, copy pixels into a writable bitmap, create a typed accessor, process pixels, push back with `createReplaceBitmap`.

```

// Snippet
const ri      = node.rasterInterface;
const bitmap = ri.createCompatibleBitmap(true); // true = copy current pixels
const rw      = PixelReaderWriterRGBA8.create(bitmap);

for (let y = 0; y < ri.height; y++) {
  for (let x = 0; x < ri.width; x++) {
    const px = rw.readPixel(x, y); // { r, g, b, alpha } — 'alpha' not 'a'
    rw.writePixel(x, y, { /* modified values */ });
  }
}

doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection, bitmap));

```

See 13a-pixel-inverter.js for a complete runnable example using this pattern.

19.9 Read-only analysis (no write-back needed)

When you only need to inspect pixels — colour sampling, histogram, dominant colour — use the read-only `PixelReaderRGBA8` (no `Writer` in the name). It doesn't need a writable bitmap.

```
'use strict';
const { Document } = require('/document');
const { PixelReaderRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) { console.log('Select a raster or image node.')}
  else {
    const ri = node.rasterInterface;
    const bitmap = ri.createCompatibleBitmap(true);
    const reader = PixelReaderRGBA8.create(bitmap);

    // Sample a 5x5 grid across the image
    const w = ri.width, h = ri.height;
    let totalR = 0, totalG = 0, totalB = 0, count = 0;

    for (let gy = 0; gy < 5; gy++) {
      for (let gx = 0; gx < 5; gx++) {
        const x = Math.floor(gx * w / 5);
        const y = Math.floor(gy * h / 5);
        const px = reader.readPixel(x, y);
        totalR += px.r; totalG += px.g; totalB += px.b;
        count++;
      }
    }

    console.log('Average colour:',
      Math.round(totalR/count),
      Math.round(totalG/count),
      Math.round(totalB/count)
    );
  }
}
```

19.10 Creating a bitmap from scratch

You don't need an existing node to start. The `Bitmap` class lets you create one from dimensions and format, or load one from a file:

```
'use strict';
const { Document } = require('/document');
const { Bitmap, RasterFormat } = require('/rasterobject');
const { RasterNodeDefinition } = require('/nodes');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');
const { AddChildNodesCommandBuilder } = require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Create a new 200x200 RGBA8 bitmap
  const bitmap = Bitmap.create(200, 200, RasterFormat.RGBA8);
  const rw = PixelReaderWriterRGBA8.create(bitmap);

  // Draw a gradient
  for (let y = 0; y < 200; y++) {
    for (let x = 0; x < 200; x++) {
      rw.writePixel(x, y, {
```

```

        r: Math.round(x / 200 * 255),
        g: Math.round(y / 200 * 255),
        b: 128,
        alpha: 255
      });
    }
  }

  // Add it to the document as a new raster node
  const nodeDef = RasterNodeDefinition.create(RasterFormat.RGBA8);
  nodeDef.bitmap = bitmap;

  const builder = AddChildNodesCommandBuilder.create();
  builder.addRasterNode(nodeDef);
  doc.executeCommand(builder.createCommand());
  console.log('Gradient raster node added.');
```

19.11 Loading a bitmap from a file

`Bitmap.loadFromFile` reads any image from the Desktop into a `Bitmap` object. The file must exist — it throws a generic `ERROR` if the path is wrong or the file is missing.

```

'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/document');
const { Bitmap, RasterFormat } = require('/rasterobject');
const { app } = require('/application');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  // Export the current document as PNG first, then load it back as a
  // Bitmap
  const outPath = `${desktop}/sdk_bitmap_test.png`;
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForWholeDocument();
  const records = doc.export(outPath, opts, area, null);

  if (!records.all[0]?.isSuccess) {
    console.log('Export failed — cannot load.');
```

```

  } else {
    // Now load it back
    const bitmap = Bitmap.loadFromFile(outPath, RasterFormat.RGBA8);
    console.log('Loaded:', bitmap.width, 'x', bitmap.height);

    // bitmap.format returns a plain integer (0 = RGBA8, 1 = RGBA16, etc.)
    // Use RasterFormat.keys/values to look up the name
    const fmtKeys = RasterFormat.keys.toString().split(',');
    const fmtVals = RasterFormat.values.toString().split(',').map(Number);
    const fmtIdx = fmtVals.indexOf(bitmap.format);
    const fmtName = fmtIdx !== -1 ? fmtKeys[fmtIdx] :
    String(bitmap.format);
    console.log('Format:', fmtName); // e.g. 'RGBA8'
  }
}
```

① **Note**
Desktop-only restriction applies. `Bitmap.loadFromFile` can only read files from the Desktop.

Any other path throws a generic `ERROR`. Always use `app.userDesktopPath` as your base path.

19.12 Rendering a node to a bitmap with `NodeRenderingEngine`

This lets you render any node — including vector, text, or group nodes — to pixels. Useful for generating thumbnails, doing pixel analysis on non-raster nodes, or compositing.

```

'use strict';
const { Document } = require('/document');
```

```

const { NodeRenderingEngine, RasterFormat } = require('/rasterobject');
const { PixelReaderRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.first;
  if (!node) { console.log('Select any node first.')}
  else {
    const engine = NodeRenderingEngine.createDefault(node,
    RasterFormat.RGBA8);

    // engine has width/height from the node's bounding box
    const bitmap = engine.createCompatibleBitmap(true);
    const reader = PixelReaderRGBA8.create(bitmap);

    // Sample the centre pixel
    const cx = Math.floor(engine.width / 2);
    const cy = Math.floor(engine.height / 2);
    console.log('Centre pixel:', reader.readPixel(cx, cy));
  }
}

```

19.13 The Buffer class

Buffer is a raw byte container — not directly for pixel data (use Bitmap or Pixel Buffer for that), but useful for binary I/O, encoding text, or building binary structures.

```

'use strict';
const { Buffer } = require('/buffer');

// Create and fill
const buf = Buffer.create(4);
const arr = buf.array; // Uint8Array view
arr[0] = 0xFF; arr[1] = 0x00; arr[2] = 0x80; arr[3] = 0xFF;

// UTF-8 encode a string
const utf8buf = Buffer.utf8('Hello, Affinity!');
console.log('Byte length:', utf8buf.size);
console.log('As string: ', utf8buf.toString());

// Concatenate
const combined = buf.concat(utf8buf);
console.log('Combined size:', combined.size);

// Slice (copy)
const sliced = buf.slice(0, 2);

// ArrayBuffer for interop with JS typed arrays
const ab = buf.arrayBuffer; // standard ArrayBuffer

```

19.13.1 `PixelBuffer` — pixel data without a GPU bitmap

PixelBuffer is a CPU-side pixel array, useful when you want the raw bytes without the overhead of a full Bitmap:

```

'use strict';
const { RGBA8 } = require('/colours');
const { PixelBuffer, RasterFormat } = require('/rasterobject');

const pb = PixelBuffer.create(100, 100, RasterFormat.RGBA8);
const ab = pb.buffer; // raw ArrayBuffer — 100×100×4 bytes
const u8 = new Uint8Array(ab);

// Fill with solid red
for (let i = 0; i < u8.length; i += 4) {
  u8[i] = 255; // R
  u8[i+1] = 0; // G
  u8[i+2] = 0; // B
}

```

```
    u8[i+3] = 255; // A
  }
```

19.14 Performance considerations

JavaScript pixel loops are slow. For large images (> 2000×2000px) a full-image nested for-loop will take several seconds. Strategies to manage this:

- Use `PixelReaderWriterRGBA8` not a generic accessor — the typed classes have direct C++ backing.
- Work on a sampled grid rather than every pixel when analysis (not editing) is the goal.
- Use `doc.executeCommandAsync` to avoid blocking the UI during heavy processing.
- Prefer the SDK's built-in filters (adjustments, layer effects) for common operations — they run in C++ at full speed. Drop to pixel level only when you need something the SDK doesn't offer natively.

19.15 Key gotchas

alpha not a — on both read and write. `readPixel` returns { r, g, b, alpha } — the key is alpha, not a. Using `px.a` reads undefined. On `writePixel`, passing { r, g, b, a: someValue } silently ignores the value and writes 255 (fully opaque) instead — no error is thrown. Always use alpha as the key name on both sides.

createReplaceBitmap requires a raster document, not a placed image. Open the image as a document (File > Open) rather than placing it on a page. A placed image has `isImageNode: true` — `createReplaceBitmap` silently does nothing on it. Rasterise first via `DocumentCommand.createRasteriseObjects(doc.selection, false, false)` if you need to process a placed image in-place.

Format mismatch gives corrupt data silently. Always check `ri.format` first. If the node is CMYK and you use `PixelReaderWriterRGBA8`, you'll read wrong values and write corrupted channels with no error message.

createCompatibleBitmap(false) gives an uninitialized bitmap. Pass `true` to copy existing pixel content. Pass `false` only when you're going to fill every pixel yourself (e.g. procedural generation).

PixelBuffer.buffer vs Bitmap.createCompatibleBuffer — `PixelBuffer` is a straight byte array; `Bitmap` has GPU backing for rendering. Use `Bitmap` when you need to pass data to pixel accessors. Use `PixelBuffer` when you just want raw bytes.

Desktop-only for file loading. `Bitmap.loadFromFile` follows the same Desktop restriction as `fs` — no other paths are accessible.

19.16 Raster pixel selection

The SDK exposes commands for working with the pixel-level selection (the “marching ants” selection in Affinity Photo). These are distinct from the node selection used elsewhere.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand, RasterSelectionLogicalOperation } = require('/commands');

const doc = Document.current;
if (doc) {
  // Check current pixel selection state
  const rsel = doc.rasterSelection;
  console.log('Has pixel selection:', !rsel.isSelectNone);
}
```

```

console.log('Is select-all/none:', rsel.isSelectAllOrNone);
console.log('Is current pixel sel:', rsel.isCurrentPixelSelection);

// Select all pixels
doc.rasterSelectAll();

// Invert the pixel selection
doc.rasterInvertSelection();

// Deselect all
doc.rasterDeselect();

// Reselect last selection
doc.rasterReselect();

// Select a polygon region (array of {x, y} points)
const polygon = [
  { x: 100, y: 100 },
  { x: 300, y: 100 },
  { x: 300, y: 300 },
  { x: 100, y: 300 },
];
doc.setRasterSelectionFromPolygon(
  polygon,
  RasterSelectionLogicalOperation.Replace, // Replace / Add /
  Subtract / Intersect
  true, // antialias
  0 // feather radius (pixels)
);

// Raster selection refinement
doc.growShrinkRasterSelection(10, false); // expand by 10px
(circular=false = square)
doc.featherRasterSelection(5); // soften edge by 5px
doc.smoothRasterSelection(3); // smooth boundary by 3px
doc.outlineRasterSelection(8, alignment, false); // outline at radius
8px

// Select based on a node's alpha channel (use intensity=false for
binary)
doc.setRasterSelectionFromObject(node, false,
RasterSelectionLogicalOperation.Replace);
}

```

`doc.rasterSelection` returns a `RasterSelection` object with read-only state. The modification commands (`setRasterSelectionFromPolygon`, `setRasterSelectionFromObject`, `rasterSelectAll`, `rasterDeselect`, etc.) are all on `doc` directly. `RasterSelectionLogicalOperation` controls how a new selection combines with the existing one.

19.16.1 Exercise 14.1 — Colour inverter

See 13a-pixel-inverter.js in lesson 13 for a working implementation.

19.16.2 Exercise 14.2 — Average colour sampler

Select any node (raster, image, or vector). Sample a 10×10 grid of pixels from a rendered bitmap and compute the average R, G, B. Display the result in a `console.log`.

19.16.3 Exercise 14.3 — Greyscale converter

See 14b-greyscale-converter.js for a working implementation using the luminance formula.

19.16.4 Exercise 14.4 — Procedural gradient node

See 14c-procedural-gradient.js for a working implementation.

① Note

`rasterSelectPolygon` is deprecated — use `setRasterSelectionFromPolygon` instead. The old method still works but prints a deprecation warning.

19.16.5 Exercise 14.5 — Histogram analysis

See 14e-histogram-analysis.js for a full implementation with per-channel statistics, clipping detection, and tonal distribution.

19.16.6 Exercise 14.6 — Posterise

See 14f-posterise.js for a working implementation with a configurable level count via dialog.

19.16.7 Exercise 14.7 — Node thumbnail

Render the selected (non-raster) node to a bitmap using NodeRendering Engine, then save it as a PNG to the Desktop using Bitmap and file export.

19.17 SDK modules used in this lesson

- /pixelaccessor — All PixelReader* and PixelReaderWriter* classes
- /commands — DocumentCommand, AddChildNodesCommandBuilder, RasterSelectionLogicalOperation
- /document — Document
- /rasterobject — Bitmap, PixelBuffer, NodeRenderingEngine, RasterFormat
- /buffer — Buffer
- /commands — DocumentCommand.createReplaceBitmap, AddChildNodesCommandBuilder
- /nodes — RasterNodeDefinition
- /document — Document

19.18 See also

- Lesson 03 — Node Model — how to find raster nodes
- Lesson 04 — Command Pattern — how commands integrate with undo
- Lesson 11 — Layer Effects — fast built-in image operations
- examples/

19.19 Examples**19.19.1 Colour inverter**

```
'use strict';

// Example 14a — Colour inverter
// Lesson 14: Pixel Processing
//
// Inverts all pixel colours in the selected raster/image node.
// Demonstrates the full bitmap pipeline: read → process → write back.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.');
}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.');
  }
  else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;

    console.log(`Node: ${node.descriptionInterface?.description ??
'unnamed'}`);
    console.log(`Size: ${w}x${h} Format: ${ri.format}`);

    if (String(ri.format) !== 'RGBA8') {
```

```

    console.log('This script requires RGBA8 format. Open an RGBA8 image
and run again.');
```

```

    } else {
      // Copy current pixels into a detached, writable bitmap
      const bitmap = ri.createCompatibleBitmap(true);
      const rw = PixelReaderWriterRGBA8.create(bitmap);

      let count = 0;
      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = rw.readPixel(x, y);
          rw.writePixel(x, y, {
            r: 255 - px.r,
            g: 255 - px.g,
            b: 255 - px.b,
            alpha: px.alpha // preserve alpha channel
          });
          count++;
        }
      }

      // Push back as a single undoable command
      doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
      console.log(`Done - inverted ${count} pixels.`);
    }
  }
}

```

19.19.2 Greyscale converter

```

'use strict';

// Example 14b - Greyscale converter
// Lesson 14: Pixel Processing
//
// Converts a selected RGBA8 raster node to greyscale using the
// photometric luminance formula:  $L = 0.299 * R + 0.587 * G + 0.114 * B$ 

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.');
```

```

  } else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;

    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '- this script requires RGBA8.
Open an RGBA8 image and run again.');
```

```

    } else {
      const bitmap = ri.createCompatibleBitmap(true);
      const rw = PixelReaderWriterRGBA8.create(bitmap);

      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = rw.readPixel(x, y);
          // Photometric luminance
          const L = Math.round(0.299 * px.r + 0.587 * px.g + 0.114 * px.b);
          rw.writePixel(x, y, { r: L, g: L, b: L, alpha: px.alpha });
        }
      }
    }
  }
}

```

```

    }

    doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
    console.log(`Converted ${w}x${h} pixels to greyscale.`);
  }
}
}

```

19.19.3 Procedural gradient node

```

'use strict';
const { RGBA8 } = require('/colours');

// Example 14c – Procedural gradient node
// Lesson 14: Pixel Processing
//
// Creates a new 512x512 raster node from scratch containing a radial
gradient.
// White at the centre, black at the edges.
// Demonstrates creating bitmaps without an existing node.

const { Document } = require('/document');
const { AddChildNodesCommandBuilder } = require('/commands');
const { Bitmap, RasterFormat } = require('/rasterobject');
const { RasterNodeDefinition } = require('/nodes');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');

const W = 512;
const H = 512;

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // 1. Create a blank bitmap
  const bitmap = Bitmap.create(W, H, RasterFormat.RGBA8);
  const rw = PixelReaderWriterRGBA8.create(bitmap);

  const cx = W / 2;
  const cy = H / 2;
  const maxDist = Math.sqrt(cx * cx + cy * cy);

  // 2. Fill with radial gradient
  for (let y = 0; y < H; y++) {
    for (let x = 0; x < W; x++) {
      const dx = x - cx;
      const dy = y - cy;
      const dist = Math.sqrt(dx * dx + dy * dy);
      const t = Math.min(dist / maxDist, 1.0); // 0 = centre, 1 =
edge
      const v = Math.round((1 - t) * 255); // bright centre,
dark edge

      rw.writePixel(x, y, { r: v, g: v, b: v, alpha: 255 });
    }
  }

  // 3. Wrap in a raster node definition
  const nodeDef = RasterNodeDefinition.create(RasterFormat.RGBA8);
  nodeDef.bitmap = bitmap;

  // 4. Add to document
  const builder = AddChildNodesCommandBuilder.create();
  builder.addRasterNode(nodeDef);
  const cmd = builder.createCommand(true);
  doc.executeCommand(cmd);

  const newNode = cmd.newNodes[0];
  if (newNode) {

```

```

    newNode.userDescription = 'Radial Gradient';
  }

  console.log(`Radial gradient (${W}x${H}) added to document.`);
}

```

19.19.4 Colour histogram

```

'use strict';

// Example 14d – Colour histogram
// Lesson 14: Pixel Processing
//
// Reads every pixel and builds a luminance histogram.
// Reports the 5 most common brightness values and average colour.

const { Document } = require('/document');
const { PixelReaderRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.');
  } else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;
    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '- this script requires RGBA8.
Open an RGBA8 image and run again.');
    } else {
      const bitmap = ri.createCompatibleBitmap(true);
      const reader = PixelReaderRGBA8.create(bitmap); // read-only, no
write needed

      const histogram = new Array(256).fill(0);
      let totalR = 0, totalG = 0, totalB = 0, opaqueCount = 0;

      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = reader.readPixel(x, y);
          if (px.alpha > 0) {
            const L = Math.round(0.299 * px.r + 0.587 * px.g + 0.114 *
px.b);
            histogram[L]++;
            totalR += px.r; totalG += px.g; totalB += px.b;
            opaqueCount++;
          }
        }
      }

      // Find top 5 most common luminance values
      const sorted = histogram
        .map((count, value) => ({ value, count }))
        .sort((a, b) => b.count - a.count)
        .slice(0, 5);

      console.log('=== Histogram:
${node.descriptionInterface?.description ?? '(unnamed)'} ===');
      console.log(`Size: ${w}x${h} Opaque pixels: ${opaqueCount}`);
      console.log('');
      console.log('Top 5 luminance values:');
      for (const { value, count } of sorted) {
        const bar = '█'.repeat(Math.round(count / opaqueCount * 40));
        console.log(` ${String(value).padStart(3)}: ${bar} (${count})`);
      }
    }
  }
}

```

```

    if (opaqueCount > 0) {
      console.log('');
      console.log('Average colour:',
        Math.round(totalR / opaqueCount),
        Math.round(totalG / opaqueCount),
        Math.round(totalB / opaqueCount)
      );
    }
  }
}
}

```

19.19.5 Histogram analysis

```

'use strict';

// Example 14e – Histogram analysis
// Lesson 14: Pixel Processing
//
// Reads every pixel in the selected raster document and produces a
// detailed
// tonal and colour report: per-channel statistics, clipping detection,
// shadow/midtone/highlight distribution, and a text-based histogram bar
// chart.
//
// This is a read-only analysis – no pixels are modified and no write-back
// is needed.
//
// Open the image as a document (File > Open), select the layer, and run.

const { Document } = require('/document');
const { PixelReaderRGBA8 } = require('/pixelaccessor');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
    n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.')}
  else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;
    const name = node.descriptionInterface?.description ?? '(unnamed)';

    // Read-only accessor – no write-back needed for analysis
    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '– this script requires RGBA8.
        Open an RGBA8 image and run again.')}
    else {
      const bitmap = ri.createCompatibleBitmap(true);
      const reader = PixelReaderRGBA8.create(bitmap);

      // Per-channel histograms (256 buckets each)
      const histR = new Array(256).fill(0);
      const histG = new Array(256).fill(0);
      const histB = new Array(256).fill(0);
      const histL = new Array(256).fill(0); // luminance

      // Per-channel min/max
      let minR = 255, maxR = 0;
      let minG = 255, maxG = 0;
      let minB = 255, maxB = 0;

      // Totals for averages
      let totalR = 0, totalG = 0, totalB = 0;

```

```

// Tonal zone counts
let shadows = 0, midtones = 0, highlights = 0;

// Clipping counts (pixels at 0 or 255)
let clippedShadow = 0, clippedHighlight = 0;

let opaqueCount = 0;

console.log(`Analysing ${w}x${h} pixels...`);

for (let y = 0; y < h; y++) {
  for (let x = 0; x < w; x++) {
    const px = reader.readPixel(x, y);

    // Skip fully transparent pixels – they don't contribute to
    // visible tones
    if (px.alpha === 0) continue;
    opaqueCount++;

    // Accumulate channel histograms
    histR[px.r]++;
    histG[px.g]++;
    histB[px.b]++;

    // Luminance (perceptual weighting)
    const L = Math.round(0.299 * px.r + 0.587 * px.g + 0.114 * px.b);
    histL[L]++;

    // Per-channel min/max
    if (px.r < minR) minR = px.r;
    if (px.r > maxR) maxR = px.r;
    if (px.g < minG) minG = px.g;
    if (px.g > maxG) maxG = px.g;
    if (px.b < minB) minB = px.b;
    if (px.b > maxB) maxB = px.b;

    // Totals
    totalR += px.r;
    totalG += px.g;
    totalB += px.b;

    // Tonal zones: shadows 0-84, midtones 85-169, highlights 170-255
    if (L <= 84) shadows++;
    else if (L <= 169) midtones++;
    else highlights++;

    // Clipping
    if (L === 0) clippedShadow++;
    if (L === 255) clippedHighlight++;
  }
}

if (opaqueCount === 0) {
  console.log('No opaque pixels found.');
```

```

} else {
  const avgR = Math.round(totalR / opaqueCount);
  const avgG = Math.round(totalG / opaqueCount);
  const avgB = Math.round(totalB / opaqueCount);
  const avgL = Math.round(0.299 * avgR + 0.587 * avgG + 0.114 * avgB);

  console.log('');
  console.log(`=== ${name} - ${w}x${h} (${opaqueCount} opaque pixels`);

  console.log('');
  console.log(`--- Channel ranges -----`);
  console.log(` Red:  min ${String(minR).padStart(3)} max`);
  console.log(`${String(maxR).padStart(3)}  avg ${String(avgR).padStart(3)} `);

```

```

        console.log(`  Green: min ${String(minG).padStart(3)} max
${String(maxG).padStart(3)}  avg ${String(avgG).padStart(3)}`);
        console.log(`  Blue: min ${String(minB).padStart(3)} max
${String(maxB).padStart(3)}  avg ${String(avgB).padStart(3)}`);
        console.log(`  Luma:  avg ${String(avgL).padStart(3)}`);

        console.log('');
        console.log('— Tonal distribution —————');
        const pct = (n) => ((n / opaqueCount) * 100).toFixed(1).padStart(5)
+ '%';
        console.log(`  Shadows    (0-84):  ${pct(shadows)}  (${shadows}
px)`);
        console.log(`  Midtones   (85-169): ${pct(midtones)}  (${midtones}
px)`);
        console.log(`  Highlights(170-255): ${pct(highlights)}
(${highlights} px)`);

        console.log('');
        console.log('— Clipping —————');
        const clipPctS = ((clippedShadow / opaqueCount) *
100).toFixed(2);
        const clipPctH = ((clippedHighlight / opaqueCount) *
100).toFixed(2);
        console.log(`  Shadow clip  (L=0):  ${clipPctS}%  (${clippedShadow}
px)`);
        console.log(`  Highlight clip (L=255): ${clipPctH}%
(${clippedHighlight} px)`);
        if (parseFloat(clipPctS) > 1) console.log(' ⚠ Shadow clipping
exceeds 1%');
        if (parseFloat(clipPctH) > 1) console.log(' ⚠ Highlight clipping
exceeds 1%');

        // Luminance histogram bar chart — 32 buckets of 8 values each
        console.log('');
        console.log('— Luminance histogram (0=dark → 255=bright) —');
        const bucketCount = 32;
        const buckets = new Array(bucketCount).fill(0);
        for (let i = 0; i < 256; i++) {
            buckets[Math.floor(i / (256 / bucketCount))] += histL[i];
        }
        const maxBucket = Math.max(...buckets);
        const barWidth = 30;
        for (let i = 0; i < bucketCount; i++) {
            const low = Math.round(i * 256 / bucketCount);
            const filled = Math.round((buckets[i] / maxBucket) * barWidth);
            const bar = '█'.repeat(filled) + '░'.repeat(barWidth - filled);
            console.log(`  ${String(low).padStart(3)}  ${bar}`);
        }
    }
}
}
}

```

19.19.6 Posterise

```

'use strict';

// Example 14f — Posterise
// Lesson 14: Pixel Processing
//
// Reduces each colour channel to N discrete levels, creating a flat,
// graphic
// poster-like effect. The level count is configurable via a dialog (2-
// 16).
//
// Posterisation works by quantising each channel value to the nearest
// step
// in a set of N evenly-spaced levels across 0-255. With 2 levels each
// channel

```

```
// is either 0 or 255. With 4 levels the steps are 0, 85, 170, 255.
//
// This effect cannot be expressed as a simple adjustment layer – it
requires
// per-pixel arithmetic, which is why it belongs in a pixel script.
//
// Open the image as a document (File > Open), select the layer, and run.

const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Dialog, DialogResult } = require('/dialog');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');

// — Dialog


---


// addComboBox is used for level selection – the SDK dialog has no slider
control.

const dlg = Dialog.create('Posterise');
const col = dlg.addColumn();
const grp = col.addGroup('Settings');
const LEVELS = ['2', '3', '4', '5', '6', '8'];
const combo = grp.addComboBox('Levels', LEVELS, 2); // default index 2 =
4 levels

const result = dlg.runModal();
if (result !== DialogResult.Ok) {
  console.log('Cancelled.');
```

```
  } else {
    const levels = parseInt(LEVELS[combo.selectedIndex], 10); // read
selectedIndex, not .value

    // — Pixel processing


---


    const doc = Document.current;
    if (!doc) { console.log('No document open.');
```

```
    } else {
      const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
      if (!node) {
        console.log('Select a raster or image node and run again.');
```

```
      } else {
        const ri = node.rasterInterface;
        const w = ri.width;
        const h = ri.height;
        if (String(ri.format) !== 'RGBA8') {
          console.log('Format is', ri.format, '– this script requires RGBA8.
Open an RGBA8 image and run again.');
```

```
        } else {
          const bitmap = ri.createCompatibleBitmap(true);
          const rw = PixelReaderWriterRGBA8.create(bitmap);

          console.log(`Posterising ${w}x${h} to ${levels} levels...`);

          // Pre-compute a lookup table: for each input value 0–255, what is
the
          // nearest quantised output? This avoids repeating the arithmetic
per pixel.
          //
          // For N levels, the step size is 255 / (N-1).
          // The quantised value is: round(input / step) * step, clamped to 0–
255.
          const lut = new Uint8Array(256);
          const step = 255 / (levels - 1);
          for (let i = 0; i < 256; i++) {
            lut[i] = Math.min(255, Math.round(Math.round(i / step) * step));
          }
        }
      }
    }
  }
}
```

```
for (let y = 0; y < h; y++) {  
  for (let x = 0; x < w; x++) {  
    const px = rw.readPixel(x, y);  
    rw.writePixel(x, y, {  
      r: lut[px.r],  
      g: lut[px.g],  
      b: lut[px.b],  
      alpha: px.alpha // preserve transparency  
    });  
  }  
}  
  
doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,  
bitmap));  
  console.log(`Done — posterised to ${levels} levels.`);  
}  
}  
}
```

Lesson 15 — Fonts & Typography

Typography automation requires knowing what fonts are available, how to classify them, and how to apply them to text. This lesson covers the full font inspection API: enumerating font families, querying properties like weight, width, and serif classification, using the PANOSE system for font matching, and combining font data with the text and story API to build practical typography tools.

Font-related tasks are common in professional design workflows: checking that a document uses only approved fonts, substituting a missing typeface with the closest available alternative, generating a font usage report, or applying a consistent typographic style across an entire document. This lesson gives you the tools to automate all of them.

The font API has two distinct layers that serve different purposes. The `/fonts` module gives you the enumeration layer — `FontFamily.all`, font metadata, classification properties, and PANOSE descriptors — and is the right tool for auditing, reporting, and font-matching logic. The `/glyphatts` and `/paragraphatts` modules give you the application layer — setting specific fonts on text ranges via `StoryDelta` — and are what you reach for when you want to change how text looks in a document. Scripts that confuse these two layers — trying to apply formatting using the enumeration API, or trying to query font metadata through the text API — end up with either no-ops or errors that give no clear indication of the underlying mismatch.

The PANOSE system deserves particular attention. PANOSE is a ten-digit numeric classification that describes a typeface's visual characteristics: family kind, serif style, weight, proportion, contrast, stroke variation, arm style, letterform, midline, and x-height. This makes it possible to find the closest

available alternative to a missing font without relying on name matching. A font substitution based on PANOSE similarity will usually produce a more visually appropriate result than one based on shared name fragments. This lesson shows how to extract PANOSE values and use them to score candidate fonts — a pattern that is directly useful in any workflow where documents must render consistently across machines with different font sets installed.

By the end of this lesson you will be able to enumerate all fonts installed on the system, classify them by type and weight, find the closest available substitute for any named typeface, and apply specific fonts to text ranges via the story API. Combined with the text lesson from Phase 3, these capabilities give you everything needed to build a full font audit and remediation tool in a single script.

20.1 What this lesson covers

- Enumerating all installed fonts and font families
- Font classification: serif, sans-serif, monospaced, condensed
- The PANOSE system for font matching
- Querying font properties: weight, width, bold/italic availability
- Font collections
- Combining font data with the text/story API for practical scripts
- Finding the best available font for a given style need
- Document font usage reporting

20.2 The fonts module

The `/fonts` module adds a complete view of the system font library. Before this, you could apply a named style via `StoryDelta`, but you couldn't ask "is this font installed?" or "what serif fonts do I have?". Now you have the full picture.

```
'use strict';
const { Font, FontFamily, FontCollection, FontWeight, FontWidth } =
  require('/fonts');
```

20.3 Three levels of font data

```
FontCollection ← named groups (e.g. "Microsoft Office Fonts", "Adobe
  Fonts")
  └─ FontFamily ← a named family (e.g. "Helvetica Neue")
    └─ Font      ← an individual variant (e.g. "Helvetica Neue Bold
  Italic")
```

You can enter the hierarchy at any level.

20.4 Enumerating font families

```
'use strict';
const { FontFamily } = require('/fonts');
const { EnumerationResult } = require('affinity:common');

// Array of all families (convenient but allocates a lot)
const families = FontFamily.all;
console.log(`${families.length} font families installed`);

// Callback-based (memory-efficient for large font libraries)
FontFamily.enumerate(family => {
  console.log(family.name, '-', family.fontCount, 'font(s)');
  return EnumerationResult.Continue;
});
```

20.5 Inspecting fonts within a family

```
'use strict';
const { FontFamily } = require('/fonts');
const family = FontFamily.all.find(f => f.name === 'Arial');
if (!family) {
```

```

    console.log('Arial not installed.');
```

```

} else {
  console.log(`${family.name}: ${family.fontCount} variant(s)`);
  console.log('Has variable fonts: ${family.hasVariations}');

  for (let i = 0; i < family.fontCount; i++) {
    const font = family.getFont(i);
    console.log(`  [${i}] ${font.traitsName}`);
    console.log(`           Bold: ${font.isBold} Italic: ${font.isItalic}`);
    console.log(`           Weight: ${font.weight} Width: ${font.width}`);
    console.log(`           PostScript: ${font.postscriptName}`);
  }

  // Or iterate directly
  for (const font of family.fonts) {
    console.log(' ', font.postscriptName);
  }
}

```

20.6 Getting a Font object

The correct way to get a Font object is via FontFamily.

```

'use strict';
const { FontFamily } = require('/fonts');

// Build a name-keyed map of all installed families
const familyMap = new Map();
for (const fam of FontFamily.all) {
  familyMap.set(fam.name.toLowerCase(), fam);
}

// Get the regular weight font from a named family
function getFontByFamily(name, preferBold = false) {
  const fam = familyMap.get(name.toLowerCase());
  if (!fam) return null;
  let best = null, bestDist = Infinity;
  const targetWeight = preferBold ? 0.75 : 0;
  for (const f of fam.fonts) {
    if (!f.isValid) continue;
    const d = Math.abs(f.normalizedWeight - targetWeight);
    if (d < bestDist) { bestDist = d; best = f; }
  }
  return best;
}

const georgia = getFontByFamily('Georgia');
if (georgia) {
  console.log('Valid: ', georgia.isValid);
  console.log('Serif: ', georgia.isSerif);
  console.log('PS name: ', georgia.postscriptName);
}

// Platform defaults (these always work with no arguments)
const { Font } = require('/fonts');
const def = Font.createDefault();
const mono = Font.createDefaultMonospaced();
const sym = Font.createDefaultSymbol();

```

20.7 Font classification via PANOSE

PANOSE is a 10-byte classification system for fonts. The SDK exposes the key classification queries directly on the Font object:

```

'use strict';
const { Font } = require('/fonts');

for (const font of Font.all.slice(0, 50)) {
  const family = font.familyName;
  if (font.isSerif) console.log('Serif: ', family, font.traitsName);
}

```

```

    if (font.isSansSerif) console.log('Sans: ', family, font.traitsName);
    if (font.isMonospaced) console.log('Mono: ', family, font.traitsName);
  }

```

For fine-grained classification, access the Panose object directly:

```

'use strict';
const { Font, FontFamily, PanoseType } = require('/fonts');

// Get a font via FontFamily — this is always safe
const tnr = FontFamily.all.find(f => f.name === 'Times New Roman');
const font = tnr ? tnr.getFont(0) : Font.createDefault();

const panose = font.panose;

console.log('Is serif: ', panose.isSerif);
console.log('Is sans-serif:', panose.isSansSerif);
console.log('Is monospaced:', panose.isMonospaced);
console.log('Is empty: ', panose.isEmpty);
console.log('PANOSE string:', panose.toString());

// Type-specific classification
console.log('Is serif type:', panose.is(PanoseType.Serif));

```

20.8 Font distance (finding the closest match)

Both Font and Panose expose a `getDistance(other)` method. Smaller distance = closer match. Useful for finding a substitute when a font isn't installed:

```

'use strict';
const { Font, FontFamily } = require('/fonts');

// Try to find 'Futura' via FontFamily — no risk of INVALID_ARGS
const futuraFamily = FontFamily.all.find(f => f.name === 'Futura');

if (!futuraFamily) {
  console.log('Futura not installed — finding closest match...');

  // Use createEmpty() as a baseline for distance comparison
  // Or better: use the default font as the reference point
  const target = Font.createDefault();

  let bestFont = null;
  let bestDist = Infinity;

  for (const font of Font.all) {
    const dist = target.getDistance(font);
    if (dist < bestDist) {
      bestDist = dist;
      bestFont = font;
    }
  }

  if (bestFont) {
    console.log('Best match:', bestFont.familyName, bestFont.traitsName);
    console.log('Distance: ', bestDist);
  }
} else {
  console.log('Futura is installed:', futuraFamily.fontCount,
    'variant(s)');
}

```

To find a substitute for a specific missing font, the most reliable approach is to compare PANOSE data: get the PANOSE of any installed font with a similar style, then measure `font.panose.getDistance(candidate.panose)` across all installed fonts.

20.9 Font collections

Collections are named groups of fonts (Adobe Fonts, Microsoft Office Fonts, etc.):

```
'use strict';
const { FontCollection } = require('/fonts');

for (const collection of FontCollection.all) {
  console.log('Collection:', collection.displayName);
}
```

20.10 Practical: check which fonts a document uses

The Document object exposes `enumerateFontNames` — fonts referenced in text nodes:

```
'use strict';
const { Document } = require('/document');
const { Font, FontWeight, FontWidth } = require('/fonts');

const doc = Document.current;
if (!doc) { console.log('No document.'); }
else {
  const fontNames = doc.getFontNames(); // returns [{ name,
isInstalled }] pairs via enumeration

  // Or use the raw enumeration
  const missing = [];
  doc.enumerateFontNames((name, isInstalled) => {
    if (!isInstalled) missing.push(name);
    return 0; // continue
  });

  console.log('Fonts used in document:');
  doc.enumerateFontNames((name, isInstalled) => {
    console.log(` ${isInstalled} ? 'v' : 'x' } ${name}`);
    return 0;
  });

  if (missing.length > 0) {
    console.log(`\n▲ ${missing.length} font(s) missing:');
    for (const name of missing) console.log(' -', name);
  }
}
```

20.11 Practical: font picker dialog

Combine the fonts module with the Dialog API to build a font chooser:

```
'use strict';
const { Document } = require('/document');
const { Font, FontFamily } = require('/fonts');
const { Dialog, DialogResult } = require('/dialog');
const { DocumentCommand } = require('/commands');
const { StoryDelta } = require('/storydelta');
const { GlyphAttStringType } = require('/glyphatts');
const { Selection, TextSelection } = require('/selections');

const doc = Document.current;
if (!doc) { console.log('No document.'); }
else {
  const families = FontFamily.all;
  const names = families.map(f => f.name);

  const dlg = Dialog.create('Choose Font');
  const col = dlg.addColumn();
  const grp = col.addGroup('Font family');
  const combo = grp.addComboBox('Family', names, 0);

  const result = dlg.runModal();
}
```

```

    if (result !== DialogResult.Ok) { console.log('Cancelled.');
```

```

  } else {
    const chosenFamily = families[combo.selectedIndex];
    const firstFont    = chosenFamily.fontCount > 0 ?
chosenFamily.getFont(0) : null;
    if (!firstFont) { console.log('No fonts in family.');
```

```

  } else {
    // Apply to selected text via StoryDelta
    const textNode = doc.selection.nodes.filter(n =>
n.isTextNode).first;
    if (!textNode) {
      console.log('Select a text node too.');
```

```

    } else {
      doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

      const si    = textNode.storyInterface;
      const range = si.storyRange;
      const textSel = TextSelection.create([ { begin: range.begin, end:
range.end - 1 } ]);
      const sel    = Selection.create(doc, textNode);
      sel.addSubSelectionForNode(textNode, textSel);

      const delta =
StoryDelta.createGlyphString(GlyphAttStringType.FontPostscriptName,
firstFont.postscriptName);
      doc.executeCommand(DocumentCommand.createFormatText(sel, delta));
      console.log(`Applied font: ${firstFont.postscriptName}`);
    }
  }
}
}
}

```

20.12 Font properties reference

Property	Type	Description
font.isValid	bool	Font exists and is usable
font.isBold	bool	Bold weight
font.isItalic	bool	Italic style
font.isVariableBold	bool	Has variable bold axis
font.isVariableItalic	bool	Has variable italic axis
font.isBoldAvailable	bool	Bold variant exists in family
font.isItalicAvailable	bool	Italic variant exists in family
font.isCondensed	bool	Condensed width class
font.isExpanded	bool	Expanded width class
font.isSerif	bool	Serif (via PANOSE)
font.isSansSerif	bool	Sans-serif (via PANOSE)
font.isMonospaced	bool	Monospaced (via PANOSE)
font.weight	number	Numeric weight (400 = Regular, 700 = Bold)

Property	Type	Description
<code>font.normalizedWeight</code>	number	Normalised weight 0–1
<code>font.width</code>	C++ enum	Width class — read from existing font; cannot be constructed directly
<code>font.familyName</code>	string	Family name
<code>font.wwsFamily</code>	string	WWS family name
<code>font.ribbiFamily</code>	string	RIBBI family name
<code>font.traitsName</code>	string	Traits string (Bold, Italic, etc.)
<code>font.postscriptName</code>	string	PostScript name (unique identifier)
<code>font.postscriptPrefix</code>	string	PS name prefix
<code>font.variableBold</code>	number	Variable bold axis value
<code>font.variableItalic</code>	number	Variable italic axis value
<code>font.panose</code>	Panose	Full PANOSE classification

20.13 FontFamily properties reference

Property	Type	Description
<code>family.name</code>	string	Family name
<code>family.fontCount</code>	number	Number of fonts in family
<code>family.hasVariations</code>	bool	Contains variable fonts
<code>family.hasFixed</code>	bool	Contains fixed-pitch fonts
<code>family.fonts</code>	Font[]	All fonts in family
<code>family.getFont(i)</code>	Font	Font at index

20.14 Key gotchas

font.postscriptName may be empty on some fonts. Even when `isValid` is true, some properties like `postscriptName` may be empty. Always prefer getting fonts via `FontFamily.all` and `family.getFont(i)` for reliable property access.

panose.is(PanoseType.Serif) and panose.isSerif can disagree. The convenience property uses a broader classification; `panose.is()` applies a stricter type test. Prefer `font.isSerif` for general classification checks.

Font.all enumerates every individual font — not families. A family with 8 variants appears as 8 entries. For family-level work, use `FontFamily.all`.

PANOSE is not always populated. Some fonts have an empty PANOSE record (`panose.isEmpty === true`). In this case `isSerif`, `isSansSerif`, `isMonospaced` will all return `false`, not because the font isn't serif, but because the data isn't there.

font.postscriptName is the stable identifier. Use this when applying fonts via `StoryDelta` — it's more reliable than the family name, which can have platform-specific variations.

doc.enumerateFontNames callback convention. The callback receives (`name`, `isInstalled`) and must return `0` to continue (not `EnumerationResult.Continue` as in other SDK callbacks). This is an inconsistency in the API.

20.15 Exercises

20.15.1 Exercise 15.1 — Font census with classification

List all installed font families. For each, classify it as `Serif`, `Sans-serif`, `Monospaced`, or `Unknown`. Print a summary count at the end.

20.15.2 Exercise 15.2 — Missing font reporter

Open a document with some text. Use `doc.enumerateFontNames` to find any fonts that are referenced but not installed. Log their names.

20.15.3 Exercise 15.3 — Font distance finder

Ask the user for a font name via a `Dialog` `TextBox`. If not installed, find and report the three closest installed alternatives using `font.getDistance()`.

20.15.4 Exercise 15.4 — Font picker tool

Build a `Dialog` with a `ComboBox` of all installed family names. When confirmed, apply the first font in the chosen family to the current text selection using `StoryDelta` and `GlyphAttStringType.FontPostscriptName`.

20.15.5 Exercise 15.5 — Variable font detector

Find and list all font families that contain at least one variable font (`family.hasVariations === true`). For each, list the family name and how many fonts it has.

20.16 SDK modules used in this lesson

- /fonts — `Font`, `FontFamily`, `FontCollection`, `FontWeight`, `FontWidth`, `Panose`, `PanoseType`
- /document — `Document` (for `enumerateFontNames`)
- /dialog — `Dialog`, `DialogResult`
- /storydelta — `StoryDelta`
- /glyphatts — `GlyphAttStringType`
- /selections — `Selection`, `TextSelection`
- /commands — `DocumentCommand`

20.17 See also

- Lesson 07 — Text & Story
- Lesson 08 — Dialog UI
- examples/

20.18 Examples

20.18.1 Font census

```
'use strict';

// Example 15a — Font census
// Lesson 15: Fonts & Typography
```

```
//
// Lists all installed font families with classification and a summary.

const { Font, FontFamily } = require('/fonts');;

const families = FontFamily.all;
let serif = 0, sans = 0, mono = 0, variable = 0, unknown = 0;

console.log(`=== Font Census: ${families.length} families ===\n`);

for (const family of families) {
  const first = family.fontCount > 0 ? family.getFont(0) : null;
  let tag = '?';

  if (first && !first.panose.isEmpty) {
    if (first.isMonospaced) { tag = 'MONO'; mono++; }
    else if (first.isSerif) { tag = 'SERF'; serif++; }
    else if (first.isSansSerif) { tag = 'SANS'; sans++; }
    else { tag = '?'; unknown++; }
  } else {
    unknown++;
  }

  const varFlag = family.hasVariations ? ' [variable]' : '';
  if (family.hasVariations) variable++;

  console.log(`[${tag}] ${family.name} (${family.fontCount})${varFlag}`);
}

console.log(`\n=== Summary ===`);
console.log(`Serif:      ${serif}`);
console.log(`Sans-serif:    ${sans}`);
console.log(`Monospaced:    ${mono}`);
console.log(`Unknown/empty: ${unknown}`);
console.log(`Variable:      ${variable}`);
console.log(`Total:        ${families.length}`);
```

20.18.2 Missing font reporter

```
'use strict';

// Example 15b – Missing font reporter
// Lesson 15: Fonts & Typography
//
// Reports all fonts used in the document that are not installed.
// Uses doc.enumerateFontNames() – callback returns 0 to continue
// (different from the EnumerationResult pattern used elsewhere in the
// SDK).

const { Document } = require('/document');
const { FontFamily } = require('/fonts');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```
}
else {
  const installed = [];
  const missing = [];

  doc.enumerateFontNames((name, isInstalled) => {
    if (isInstalled) installed.push(name);
    else missing.push(name);
    return 0; // 0 = continue (not EnumerationResult.Continue)
  });

  const total = installed.length + missing.length;
  console.log(`Document: ${doc.title}`);
  console.log(`Fonts referenced: ${total}`);
  console.log(`Installed:      ${installed.length}`);
  console.log(`Missing:        ${missing.length}`);
```

```

if (installed.length > 0) {
  console.log('\n✓ Installed:');
  for (const name of installed) console.log(`  ${name}`);
}

if (missing.length > 0) {
  // Build a lookup of installed family names for substitute suggestions
  const installedFamilies = [];
  for (const fam of FontFamily.all) {
    installedFamilies.push(fam.name.toLowerCase());
  }

  console.log('\n✗ Missing (not installed):');
  for (const name of missing) {
    console.log(`  ${name}`);

    // Suggest the closest name match among installed families
    const target = name.toLowerCase();
    let bestName = null, bestScore = -1;
    for (const famName of installedFamilies) {
      // Simple shared-prefix length as a rough similarity score
      let score = 0;
      const minLen = Math.min(target.length, famName.length);
      while (score < minLen && target[score] === famName[score]) score++;

      if (score > bestScore) { bestScore = score; bestName = famName; }
    }

    if (bestName && bestScore > 0) {
      console.log(`    → Closest installed name: ${bestName}`);
    }
  }
} else {
  console.log('\nAll fonts are installed ✓');
}
}

```

Lesson 16 – Raster Adjustments

21.1 What this lesson covers

- Creating adjustment layers programmatically via `*AdjustmentRasterNodeDefinition` factories
- Reading existing adjustment parameters from a selected node
- Setting parameters via `DocumentCommand.createSet*AdjustmentParameters`
- Every adjustment type the SDK exposes, with parameter ranges
- Inserting adjustments above a specific raster node

Raster adjustments are non-destructive layers that sit above a pixel layer and modify its appearance without altering its pixel data. Every adjustment type has two things: a **definition class** (used to create a new node) and a **command factory** (used to modify an existing node's parameters).

21.2 Creating an adjustment layer

The pattern is the same for every adjustment type:

1. Create a definition via `XxxAdjustmentRasterNodeDefinition.create(params)` or `.createDefault()`
2. Add it via `AddChildNodesCommandBuilder.addXxxAdjustmentRasterNode(def)`
3. Set `InsertionMode.OutsideAtTop` and target the raster node to place it inside that layer

```
'use strict';
const { Document } = require('/document');
const { BrightnessContrastAdjustmentRasterNodeDefinition } = require('/nodes');
```

```

const { AddChildNodesCommandBuilder, InsertionMode } = require('/
commands');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node || !node.isRasterNode) {
  console.log('Select a raster node first.');
```

```

} else {
  const def = BrightnessContrastAdjustmentRasterNodeDefinition.create({
    brightness: 0.3,
    contrast: 0.2,
    isLinear: false
  });

  const builder = AddChildNodesCommandBuilder.create();
  builder.addBrightnessContrastAdjustmentRasterNode(def);
  builder.setInsertionTarget(node);
  builder.setInsertionMode(InsertionMode.IndentedTop);

  doc.executeCommand(builder.createCommand(true));
  console.log('Brightness/Contrast adjustment added.');
```

```

}
```

The adjustment appears in the Layers panel as a child of the raster node, identical to one added manually via the menu.

21.3 Reading parameters from an existing adjustment node

To read the current parameters of an adjustment layer, check its type and access `.parameters`:

```

'use strict';
const { Document } = require('/document');
const Nodes = require('/nodes');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node instanceof Nodes.BrightnessContrastAdjustmentRasterNode) {
  const p = node.parameters;
  console.log('Brightness:', p.brightness);
  console.log('Contrast: ', p.contrast);
  console.log('Linear:   ', p.isLinear);
}

if (node instanceof Nodes.HSLShiftAdjustmentRasterNode) {
  const p = node.parameters;
  const m = p.masterParameters;
  console.log('Hue shift: ', m.hueShift);           // radians
  console.log('Sat shift: ', m.saturationShift);
  console.log('Lum shift: ', m.luminosityShift);
}
```

21.4 Brightness / Contrast

```

// Snippet
const def = BrightnessContrastAdjustmentRasterNodeDefinition.create({
  brightness: 0.0, // -1.0 to 1.0
  contrast: 0.0, // -1.0 to 1.0
  isLinear: false
});
```

To modify an existing node:

```

// Snippet
doc.executeCommand(DocumentCommand.createSetBrightnessContrastAdjustmentParameters(sel,
{
  brightness: 0.3,
  contrast: 0.2,
```

```
    isLinear:    false
  }));
```

21.5 Levels

LevelsAdjustmentParameters holds a masterParameters object and five per-channel parameter objects (R, G, B, and two extras for colour-space-specific use). All values are normalized 0.0–1.0; gamma range is 0.0–2.0.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const {
  LevelsAdjustmentRasterNodeDefinition,
  LevelsAdjustmentChannelParameters
} = require('/nodes');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  // Create with a default definition, then adjust master parameters
  const def = LevelsAdjustmentRasterNodeDefinition.createDefault(doc);
  const params = def.parameters;

  params.masterParameters = {
    blackLevel:    0.05, // lift shadows (0.0–1.0)
    whiteLevel:    0.95, // clip highlights (0.0–1.0)
    gamma:         1.0,  // midpoint gamma (0.0–2.0; 1.0 = linear)
    outputBlackLevel: 0.0,
    outputWhiteLevel: 1.0
  };

  def.parameters = params; // write back after mutation

  // Apply to selection
  const sel = Selection.create(doc, node);
  doc.executeCommand(DocumentCommand.createSetLevelsAdjustmentParameters(sel,
    params));
}
```

21.6 Curves

CurvesAdjustmentParameters holds a master spline and optional per-channel splines. Import Spline from /geometry.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const { CurvesAdjustmentRasterNodeDefinition, CurvesAdjustmentParameters }
= require('/nodes');
const { Spline } = require('/geometry');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const params = CurvesAdjustmentParameters.create();

  // Build an S-curve: slightly boost contrast
  const sCurve = Spline.create();
  sCurve.insertPointXY(0.0, 0.0); // shadow anchor
  sCurve.insertPointXY(0.25, 0.15); // shadow lift
  sCurve.insertPointXY(0.75, 0.85); // highlight pull-down
  sCurve.insertPointXY(1.0, 1.0); // highlight anchor
  params.masterSpline = sCurve;
```

```

const def = CurvesAdjustmentRasterNodeDefinition.createDefault(doc);
def.parameters = params;

const sel = Selection.create(doc, node);
doc.executeCommand(DocumentCommand.createSetCurvesAdjustmentParameters(sel,
params));
}

```

Per-channel splines use `params.setChannelSpline(channel, spline)` where channel indices are 0 = Red, 1 = Green, 2 = Blue.

21.7 HSL Shift

`HSLShiftAdjustmentParameters.create()` returns a parameters object. Call `.setChannelParameters(channel, params)` where channel 0 = master. hueShift is in radians (full circle = $2\pi \approx 6.28$).

```

'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { Selection } = require('/selections');
const {
  HSLShiftAdjustmentRasterNodeDefinition,
  HSLShiftAdjustmentParameters
} = require('/nodes');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const hslParams = HSLShiftAdjustmentParameters.create();

  // Shift master channel
  hslParams.setChannelParameters(0, {
    hueShift: 0.1, // radians
    saturationShift: 0.2, // -1.0 to 1.0
    luminosityShift: 0.0 // -1.0 to 1.0
  });

  const def = HSLShiftAdjustmentRasterNodeDefinition.create(hslParams);

  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetHSLShiftAdjustmentParameters(sel,
hslParams));
}

```

21.8 White Balance

```

// Snippet
doc.executeCommand(DocumentCommand.createSetWhiteBalanceAdjustmentParameters(sel,
{
  whiteBalance: 0.3, // -1.0 (cool) to 1.0 (warm)
  tint: 0.0 // -1.0 to 1.0
}));

```

21.9 Vibrance

```

// Snippet
doc.executeCommand(DocumentCommand.createSetVibranceAdjustmentParameters(sel,
{
  vibrance: 0.4, // -1.0 to 1.0
  saturation: 0.1 // -1.0 to 1.0
}));

```

21.10 Exposure

```

// Snippet
doc.executeCommand(DocumentCommand.createSetExposureAdjustmentParameters(sel,
{

```

```
    exposure: 1.5    // -20.0 to 20.0 (EV stops)
  }));
```

21.11 Colour Balance

Applies to shadow, midtone, and highlight tonal ranges independently. Range index: 0 = Highlights, 1 = Midtones, 2 = Shadows (use `TonalRangeType` enum if available, or the integer directly).

```
// Snippet
doc.executeCommand(DocumentCommand.createSetColourBalanceAdjustmentParameters(sel,
{
  values: [
    { cyanRed: 0.1, magentaGreen: -0.1, yellowBlue: 0.0 }, // highlights
    { cyanRed: 0.0, magentaGreen: 0.1, yellowBlue: 0.0 }, // midtones
    { cyanRed: 0.0, magentaGreen: 0.0, yellowBlue: 0.1 }  // shadows
  ],
  preserveLuminosity: true
}));
```

21.12 Posterise

```
// Snippet
doc.executeCommand(DocumentCommand.createSetPosteriseAdjustmentParameters(sel,
{
  levels: 4    // 2 to 256 integer
}));
```

21.13 Threshold

```
// Snippet
doc.executeCommand(DocumentCommand.createSetThresholdAdjustmentParameters(sel,
{
  threshold: 0.5    // 0.0 to 1.0
}));
```

21.14 Invert

`InvertAdjustmentRasterNodeDefinition` has no parameters — it just inverts all channels.

```
// Snippet
const { InvertAdjustmentRasterNodeDefinition } = require('/nodes');
const def = InvertAdjustmentRasterNodeDefinition.create();
```

21.15 Recolour

Maps all colours to a single hue/saturation (like a tinted duotone).

```
// Snippet
doc.executeCommand(DocumentCommand.createSetRecolourAdjustmentParameters(sel,
{
  hue: 1.0,    // 0.0 to 2π radians
  saturation: 0.8, // 0.0 to 1.0
  lightness: 0.0 // -1.0 to 1.0
}));
```

21.16 Split Toning

Applies separate hue/saturation to highlights and shadows.

```
// Snippet
doc.executeCommand(DocumentCommand.createSetSplitToningAdjustmentParameters(sel,
{
  highlightsHue: 0.1,    // 0.0 to 2π
  highlightsSaturation: 0.3,
  shadowsHue: 3.5,
  shadowsSaturation: 0.2,
  balance: 0.5    // 0.0 (shadows) to 1.0 (highlights)
}));
```

21.17 Black & White

Converts to greyscale with per-channel luminance weights.

```
// Snippet
doc.executeCommand(DocumentCommand.createSetBlackAndWhiteAdjustmentParameters(sel,
{
  red:    0.4,    // -2.0 to 3.0
  green:  0.6,
  blue:   0.2,
  cyan:   0.5,
  magenta: 0.3,
  yellow: 0.5
}));
```

21.18 Tone Compression and Tone Stretch

Used in HDR / tone-mapping workflows.

```
// Snippet
const { ToneCompressionMethod } = require('/nodes');
doc.executeCommand(DocumentCommand.createSetToneCompressionAdjustmentParameters(sel,
{
  method: ToneCompressionMethod.Reinhard, // or check available enum
  values
  exposure: 1.0, // 0.0 to 2.0
  gamma:    1.0, // 0.0 to 2.0
  colour:   1.0, // 0.0 to 1.0
}));
```

21.19 Normals (normal map rotation)

Used on normal map textures.

```
// Snippet
doc.executeCommand(DocumentCommand.createSetNormalsAdjustmentParameters(sel,
{
  rotation: 0.0, // -π to π radians
  scale:    1.0, // 0.0 to 2.0
  flipX:    false,
  flipY:    false
}));
```

21.20 All adjustment types — quick reference

Adjustment	Definition class	Parameter command
Brightness/Contrast	BrightnessContrastAdjustmentRasterNodeDefinition	createSetBrightnessContrastAdjustmentParameters
Levels	LevelsAdjustmentRasterNodeDefinition	createSetLevelsAdjustmentParameters
Curves	CurvesAdjustmentRasterNodeDefinition	createSetCurvesAdjustmentParameters
HSL Shift	HSLShiftAdjustmentRasterNodeDefinition	createSetHSLShiftAdjustmentParameters
White Balance	WhiteBalanceAdjustmentRasterNodeDefinition	createSetWhiteBalanceAdjustmentParameters
Vibrance	VibranceAdjustmentRasterNodeDefinition	createSetVibranceAdjustmentParameters
Exposure	ExposureAdjustmentRasterNodeDefinition	createSetExposureAdjustmentParameters

Adjustment	Definition class	Parameter command
Colour Balance	ColourBalanceAdjustmentRasterNodeDefinition	createSetColourBalanceAdjustmentParameters
Shadows/Highlights	ShadowsHighlightsAdjustmentRasterNodeDefinition	createSetShadowsHighlightsAdjustmentParameters
Posterise	PosteriseAdjustmentRasterNodeDefinition	createSetPosteriseAdjustmentParameters
Threshold	ThresholdAdjustmentRasterNodeDefinition	createSetThresholdAdjustmentParameters
Invert	InvertAdjustmentRasterNodeDefinition	<i>(no parameters)</i>
Recolour	RecolourAdjustmentRasterNodeDefinition	createSetRecolourAdjustmentParameters
Black & White	BlackAndWhiteAdjustmentRasterNodeDefinition	createSetBlackAndWhiteAdjustmentParameters
Split Toning	SplitToningAdjustmentRasterNodeDefinition	createSetSplitToningAdjustmentParameters
Selective Colour	SelectiveColourAdjustmentRasterNodeDefinition	createSetSelectiveColourAdjustmentParameters
Tone Compression	ToneCompressionAdjustmentRasterNodeDefinition	createSetToneCompressionAdjustmentParameters
Tone Stretch	ToneStretchAdjustmentRasterNodeDefinition	createSetToneStretchAdjustmentParameters
Normals	NormalsAdjustmentRasterNodeDefinition	createSetNormalsAdjustmentParameters

21.21 Key gotchas

createDefault(doc) requires the document argument for **Levels** and **Curves**. Both `LevelsAdjustmentRasterNodeDefinition.createDefault(doc)` and `CurvesAdjustmentRasterNodeDefinition.createDefault(doc)` require a document handle. Calling them with no argument throws.

Write back after mutating a parameters object. For **Levels**: read `def.parameters`, mutate the result, then set `def.parameters = params`. The object is not a live reference.

setChannelParameters for HSL takes an integer channel index, not an enum. Channel 0 = master, 1–6 = individual colour ranges. Using the wrong index is a silent no-op.

Hue values are in radians. Both `RecolourAdjustmentParameters.hue` and `HSLShiftAdjustmentChannelParameters.hueShift` are in radians. Full circle = 2π (≈ 6.283). To convert degrees: `degrees * Math.PI / 180`.

InsertionMode.InsideAtTop places the adjustment above the raster node's content. For a simple document where the raster node has no children, any insertion mode works. If the raster node already has children (e.g. masks), use `InsideAtTop` to ensure the adjustment is the topmost child.

> **Adjustment layers only affect raster pixel data.** Adding a Curves layer above a vector shape has no visual effect — the shape must be rasterised first.

21.22 Exercises

21.22.1 Exercise 16.1 — Batch brightness boost

Select multiple raster nodes. Add a Brightness/Contrast adjustment (`brightness: 0.2`, `contrast: 0.1`) inside each. Use `CompoundCommandBuilder` to make the whole operation a single undo step.

21.22.2 Exercise 16.2 — Levels dialog

Show a dialog with five `UnitValueEditor` controls (black, white, gamma, output black, output white). Apply the chosen values as a Levels adjustment on the selected raster node. Use live preview while the dialog is open.

21.22.3 Exercise 16.3 — HSL shift batch

Walk every raster node in the document. On each, add an HSL adjustment that boosts saturation by 0.15 and shifts hue by +0.1 radians. Log a count of adjusted layers.

21.22.4 Exercise 16.4 — Invert stack reporter

Walk every node in the document tree. Find any `InvertAdjustmentRasterNode` and log its parent node's name. Count the total.

21.23 HalftoneFilter

`HalftoneFilter` is a raster filter that applies a halftone screen effect. Two approaches:

21.23.1 Direct apply (via ``/layereffects``)

Applied directly via `createSetHalftoneParameters` on a selection — no `CompoundCommandBuilder`.

```
'use strict';
const { Document } = require('/document');
const { DocumentCommand } = require('/commands');
const { HalftoneFilter } = require('/layereffects');

const doc = Document.current;
const sel = doc.selection;

if (!sel.nodes.isEmpty) {
  doc.executeCommand(DocumentCommand.createSetHalftoneParameters(
    sel, // selection
    HalftoneFilter.Screen, // screen type (Screen, Dot, Line)
    HalftoneFilter.Sine, // dot type (Sine, Round, Diamond, Line,
    Square, Ellipse)
    0.5, // frequency
    45, // angle (degrees)
    0.1 // radius
  ));
  console.log('Halftone filter applied.');
```

① **Note**
HalftoneFilter is not a CompoundCommandBuilder operation. Execute it directly via `executeCommand`. Do not wrap it in a compound command — it silently fails.

21.23.2 Live filter node (via `/nodes`)

HalftoneFilterRasterNode is a live filter node with full HalftoneFilterParameters:

```
'use strict';
const { Document } = require('/document');
const { HalftoneFilterRasterNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder, InsertionMode } = require('/commands');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isRasterNode).first;

if (!node) { console.log('Select a raster node.');
}
else {
  const def = HalftoneFilterRasterNodeDefinition.createDefault();
  const builder = AddChildNodesCommandBuilder.create();
  builder.addHalftoneFilterRasterNode(def);
  builder.setInsertionTarget(node);
  builder.setInsertionMode(InsertionMode.InsideAtTop);
  doc.executeCommand(builder.createCommand(true));

  // Modify parameters later
  const { HalftoneScreenType, HalftoneDotType } = require('/nodes');
  doc.executeCommand(DocumentCommand.createSetHalftoneFilterRasterNodeParameters(
    sel, // the halftone node
    HalftoneScreenType.Screen, // Screen, Dot, Line
    HalftoneDotType.Sine, // Sine, Round, Diamond, Line,
    Square, Ellipse
    0.5, // cell size
    45, // screen angle (degrees)
    0.1, // contrast
    0, // grey-component replacement
    0 // under-colour removal
  ));
  console.log('Live halftone filter node added.');
```

21.24 SDK modules used in this lesson

- /nodes — All *AdjustmentRasterNodeDefinition and *AdjustmentRasterNode classes, CurvesAdjustmentParameters, HSLShiftAdjustmentParameters, LevelsAdjustmentChannelParameters, ToneCompressionMethod, ToneStretchMethod
- /commands — DocumentCommand, AddChildNodesCommandBuilder, InsertionMode
- /document — Document
- /selections — Selection
- /geometry — Spline (for Curves)
- /layereffects — HalftoneFilter

21.25 See also

- Lesson 11 — Layer Effects
- Lesson 14 — Pixel Processing
- examples/

21.26 Examples

21.26.1 Brightness batch adjuster

```
'use strict';

// Example 16a — Brightness batch adjuster
// Lesson 16: Raster Adjustments
//
// Adds a Brightness/Contrast adjustment layer inside every selected
```

```

raster node.
// All additions are wrapped in a CompoundCommand for single-step undo.
//
// Usage: select one or more raster/image nodes, then run.

const { Document } = require('/document');
const { BrightnessContrastAdjustmentRasterNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder, CompoundCommandBuilder,
InsertionMode } = require('/commands');

const BRIGHTNESS = 0.2; // -1.0 to 1.0
const CONTRAST = 0.1; // -1.0 to 1.0

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const nodes = [...doc.selection.nodes].filter(n => n.isRasterNode ||
n.isImageNode);
  if (nodes.length === 0) {
    console.log('Select at least one raster or image node.')}
  else {
    const compound = CompoundCommandBuilder.create();

    for (const node of nodes) {
      const def =
BrightnessContrastAdjustmentRasterNodeDefinition.create({
        brightness: BRIGHTNESS,
        contrast: CONTRAST,
        isLinear: false
      });

      const builder = AddChildNodesCommandBuilder.create();
      builder.addBrightnessContrastAdjustmentRasterNode(def);
      builder.setInsertionTarget(node);
      builder.setInsertionMode(InsertionMode.InsideAtTop);

      compound.addCommand(builder.createCommand(false));
    }

    doc.executeCommand(compound.createCommand());
    console.log(`Brightness/Contrast adjustment added to ${nodes.length}
node(s).`);
  }
}

```

21.26.2 Levels dialog with live preview

```

'use strict';

// Example 16b — Levels dialog with live preview
// Lesson 16: Raster Adjustments
//
// Shows a dialog with controls for the five Levels master parameters.
// Applies a live preview while the dialog is open and commits on OK.
//
// Usage: select a raster or image node, then run.

const { Document } = require('/document');
const { DocumentCommand, AddChildNodesCommandBuilder, InsertionMode } =
require('/commands');
const { Dialog, DialogResult } = require('/dialog');
const { LevelsAdjustmentRasterNodeDefinition } = require('/nodes');
const { Selection } = require('/selections');
const { UnitType } = require('/units');

const doc = Document.current;
const node = doc?.selection.nodes.first;

```

```

if (!doc || !node || (!node.isRasterNode && !node.isImageNode)) {
  console.log('Select a raster or image node first.');
```

```

} else {

  const dlg = Dialog.create('Levels');
  const col = dlg.addColumn();
  const grp = col.addGroup('Master channel');

  const blackEdit = grp.addUnitValueEditor('Black',
UnitType.Percentage, UnitType.Percentage, 0, 0, 100);
  const whiteEdit = grp.addUnitValueEditor('White',
UnitType.Percentage, UnitType.Percentage, 100, 0, 100);
  const gammaEdit = grp.addUnitValueEditor('Gamma',
UnitType.Percentage, UnitType.Percentage, 100, 1, 200);
  const outBEdit = grp.addUnitValueEditor('Output black',
UnitType.Percentage, UnitType.Percentage, 0, 0, 100);
  const outWEdit = grp.addUnitValueEditor('Output white',
UnitType.Percentage, UnitType.Percentage, 100, 0, 100);

  function buildParams() {
    return {
      blackLevel:      blackEdit.value / 100,
      whiteLevel:      whiteEdit.value / 100,
      gamma:           gammaEdit.value / 100,
      outputBlackLevel: outBEdit.value / 100,
      outputWhiteLevel: outWEdit.value / 100
    };
  }

  function applyPreview(preview) {
    const def = LevelsAdjustmentRasterNodeDefinition.createDefault(doc);
    const params = def.parameters;
    Object.assign(params.masterParameters, buildParams());
    def.parameters = params;

    const builder = AddChildNodesCommandBuilder.create();
    builder.addLevelsAdjustmentRasterNode(def);
    builder.setInsertionTarget(node);
    builder.setInsertionMode(InsertionMode.InsideAtTop);

    doc.executeCommand(builder.createCommand(false), preview);
  }

  dlg.onControlValueChangedHandler = () => applyPreview(true);
  applyPreview(true);

  while (dlg.runModal() == DialogResult.Ok) {
    applyPreview(false);
    break;
  }
  doc.clearPreviews();
  console.log('Done.');
```

```

}

```


Lesson 17 — Align & Distribute

Aligning and distributing nodes is one of the most common layout operations — centring a title on a page, distributing icons evenly across a toolbar, or snapping objects to a grid. This lesson covers the `GroupTransform` system: a single entry point that handles every alignment and distribution variant through structured data objects rather than dozens of separate command factories.

Automating layout is where scripting delivers some of its most visible results. Instead of manually positioning each node with `createTransform` and calculating offsets yourself, the `applyGroupTransform` method handles the math — all you specify is what operation to perform and what reference frame to use. The system separates X and Y axes so you can align left in one dimension while distributing in the other, or leave an axis untouched.

By the end of this lesson you will be able to align nodes to edges and centres, distribute them evenly with consistent spacing, and anchor operations to the spread, page, or specific selection boundaries — all in a single command call.

22.1 What this lesson covers

- `doc.applyGroupTransform(xData, yData, selection)` — the single entry point for all align and distribute operations
- `GroupTransformData` — the data object that specifies what to do on one axis
- `GroupTransformType` — the operation (align to edge, centre, distribute, space)
- `GroupTransformAnchor` — the reference frame (spread, page, first selected, last selected, etc.)

- Practical patterns: align left, centre on page, distribute horizontally, space evenly

22.2 How it works

Every align and distribute operation in Affinity is a `GroupTransform` command. You describe what to do on the X axis and what to do on the Y axis separately using `GroupTransformData` objects, then pass both to `doc.applyGroupTransform`. Passing `null` for an axis means “do nothing on this axis”.

```
'use strict';
const { Document } = require('/document');
const { GroupTransformData, GroupTransformType, GroupTransformAnchor } =
  require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Align selected nodes to their left edges, using the spread as
  // reference
  const xData = new GroupTransformData();
  xData.type = GroupTransformType.Min; // align to minimum
  // (left) edge
  xData.anchor = GroupTransformAnchor.FirstSelected; // relative to the
  // leftmost selected node

  doc.applyGroupTransform(xData, null, doc.selection);
}
```

22.3 GroupTransformType — what to do

Value	Constant	Meaning
0	None	No operation on this axis
1	Min	Align all to the minimum edge (left / top)
2	Mid	Align all centres
3	Max	Align all to the maximum edge (right / bottom)
4	DistributeMin	Distribute by minimum edges (equal gaps between leading edges)
5	DistributeMid	Distribute by centres (equal gaps between centres)
6	DistributeMax	Distribute by maximum edges (equal gaps between trailing edges)
7	Space	Space evenly (equal gaps between objects, not edges)

22.4 GroupTransformAnchor — reference frame

Value	Constant	Meaning
0	LogicalEdge	The logical bounding edge of the selection
1	Spread	The spread (canvas) boundaries
2	SpreadMargins	The spread's margin guides
3	FirstSelected	The first (bottom in panel) selected node's bounds
4	LastSelected	The last (top in panel) selected node's bounds
5	Page	The current page bounds
6	PageMargins	The current page margin guides

22.5 Common operations**22.5.1 Align left edges (to leftmost selected node)**

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Min;
xData.anchor = GroupTransformAnchor.FirstSelected;
doc.applyGroupTransform(xData, null, doc.selection);
```

22.5.2 Centre horizontally on the spread

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Mid;
xData.anchor = GroupTransformAnchor.Spread;
doc.applyGroupTransform(xData, null, doc.selection);
```

22.5.3 Centre both axes on the page

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Mid;
xData.anchor = GroupTransformAnchor.Page;

const yData = new GroupTransformData();
yData.type = GroupTransformType.Mid;
yData.anchor = GroupTransformAnchor.Page;

doc.applyGroupTransform(xData, yData, doc.selection);
```

22.5.4 Distribute horizontally with equal spacing between objects

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Space;
xData.anchor = GroupTransformAnchor.LogicalEdge;
doc.applyGroupTransform(xData, null, doc.selection);
```

22.5.5 Distribute centres vertically

```
// Snippet
const yData = new GroupTransformData();
yData.type = GroupTransformType.DistributeMid;
```

```
yData.anchor = GroupTransformAnchor.LogicalEdge;
doc.applyGroupTransform(null, yData, doc.selection);
```

22.5.6 Align right edges to the spread's right margin

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Max;
xData.anchor = GroupTransformAnchor.SpreadMargins;
doc.applyGroupTransform(xData, null, doc.selection);
```

22.6 Full example — align & distribute dialog

```
'use strict';
const { Document } = require('/document');
const { GroupTransformData, GroupTransformType, GroupTransformAnchor } =
  require('/commands');
const { Dialog, DialogResult } = require('/dialog');

const doc = Document.current;
if (!doc || doc.selection.nodes.length < 2) {
  console.log('Select at least 2 nodes first.');
```

```
} else {
  const dlg = Dialog.create('Align & Distribute');

  const col = dlg.addColumn();

  const alignGrp = col.addGroup('Align horizontal');
  const hAlignCombo = alignGrp.addComboBox('To', [
    'Left edges', 'Centres', 'Right edges'
  ], 0);

  const anchorGrp = col.addGroup('Reference');
  const anchorCombo = anchorGrp.addComboBox('Anchor', [
    'First selected', 'Last selected', 'Spread', 'Page'
  ], 0);

  const distGrp = col.addGroup('Distribute horizontal');
  const distCombo = distGrp.addComboBox('Method', [
    'None', 'Leading edges', 'Centres', 'Trailing edges', 'Space evenly'
  ], 0);

  if (dlg.runModal() !== DialogResult.Ok) { console.log('Cancelled.');
```

```
} else {
  const anchorMap = [
    GroupTransformAnchor.FirstSelected,
    GroupTransformAnchor.LastSelected,
    GroupTransformAnchor.Spread,
    GroupTransformAnchor.Page
  ];
  const alignTypeMap = [
    GroupTransformType.Min,
    GroupTransformType.Mid,
    GroupTransformType.Max
  ];
  const distTypeMap = [
    GroupTransformType.None,
    GroupTransformType.DistributeMin,
    GroupTransformType.DistributeMid,
    GroupTransformType.DistributeMax,
    GroupTransformType.Space
  ];

  const anchor = anchorMap[anchorCombo.selectedIndex];
  const alignIdx = hAlignCombo.selectedIndex;
  const distIdx = distCombo.selectedIndex;

  // Apply align
  const xAlign = new GroupTransformData();
  xAlign.type = alignTypeMap[alignIdx];
```

```

xAlign.anchor = anchor;
doc.applyGroupTransform(xAlign, null, doc.selection);

// Apply distribute if selected
if (distIdx > 0) {
  const xDist = new GroupTransformData();
  xDist.type = distTypeMap[distIdx];
  xDist.anchor = GroupTransformAnchor.LogicalEdge;
  doc.applyGroupTransform(xDist, null, doc.selection);
}

console.log('Done.');
```

22.7 Combining align and distribute

Both axes are processed independently and can be combined in a single `applyGroupTransform` call. For example, snap all selected nodes to a grid: align to the left of the spread on X, distribute evenly on Y:

```

// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Min;
xData.anchor = GroupTransformAnchor.Spread;

const yData = new GroupTransformData();
yData.type = GroupTransformType.Space;
yData.anchor = GroupTransformAnchor.LogicalEdge;

doc.applyGroupTransform(xData, yData, doc.selection);
```

22.8 Using DocumentCommand directly

`doc.applyGroupTransform` is the convenience form. The underlying command is:

```

// Snippet
const { DocumentCommand } = require('/commands');
const cmd = DocumentCommand.createGroupTransform(selection, xData, yData);
doc.executeCommand(cmd);
```

Use the command form when you need to batch the transform inside a `CompoundCommandBuilder` alongside other operations.

22.9 Key gotchas

Requires at least 2 nodes selected for distribute operations. With only one node selected, `Space`, `DistributeMin`, `DistributeMid`, and `DistributeMax` are silent no-ops. `Min`, `Mid`, and `Max` work on a single node (they move it to align against the anchor).

GroupTransformData uses new, not a static factory. Unlike most SDK types, `GroupTransformData` is constructed with `new GroupTransformData()`. Its `type` and `anchor` properties are then set directly.

node.baseBoxInterface.baseBox reflects LOCAL coordinates after transform. The visual positions of nodes change but `baseBox` values may appear unchanged because the actual translation is encoded in the node's `transformInterface`. Use `node.getSpreadBaseBox(false)` to read the post-transform spread-space bounding box if you need to verify results.

Anchor anchors that reference selected nodes (`FirstSelected`, `LastSelected`) use panel order, not selection order. First selected in panel terms is the bottommost node in the `Layers` panel.

Note
Operations are not previewable via the preview flag — `GroupTransform` commands commit immediately.

22.10 Exercises

22.10.1 Exercise 17.1 — Centre on canvas

Select any number of nodes. Centre them all horizontally and vertically on the spread in a single operation.

22.10.2 Exercise 17.2 — Distribute evenly

Select 5 or more shapes in a row. Distribute them with equal horizontal spacing between objects. The leftmost and rightmost should stay in place.

22.10.3 Exercise 17.3 — Grid snap

Write a script that aligns all selected nodes into a 3-column grid: the first three align horizontally, the next three at a fixed Y offset, and so on. Hint: iterate selected nodes in groups of 3, using FirstSelected/LastSelected anchors.

22.10.4 Exercise 17.4 — Align dialog

Build a dialog with dropdowns for horizontal align (None / Left / Centre / Right), vertical align (None / Top / Middle / Bottom), and reference (Spread / Page / Selection). Apply the chosen combination in a single command.

22.11 SDK modules used in this lesson

- /commands — DocumentCommand, GroupTransformData, GroupTransformType, GroupTransformAnchor, CompoundCommandBuilder
- /document — Document
- /dialog — Dialog, DialogResult
- /selections — Selection

22.12 See also

- Lesson 04 — Command Pattern
- Lesson 06 — Geometry
- examples/

22.13 Examples

22.13.1 Align selection

```
'use strict';

// Example 17a — Align selection
// Lesson 17: Align & Distribute
//
// Demonstrates the six most common alignment operations using
// GroupTransform.
// Runs through each in sequence with a 0 delay between them for
// illustration.
//
// Usage: select 3 or more nodes, then run.

const { Document } = require('/document');
const { GroupTransformData, GroupTransformType, GroupTransformAnchor } =
  require('/commands');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const count = doc.selection.nodes.length;
  if (count < 2) {
    console.log('Select at least 2 nodes first.')}
  else {
    console.log(`Operating on ${count} nodes.`);

    function align(xType, xAnchor, yType, yAnchor, label) {
      const xData = xType !== null ? Object.assign(new
        GroupTransformData(), { type: xType, anchor: xAnchor }) : null;
```

```
    const yData = yType !== null ? Object.assign(new
GroupTransformData(), { type: yType, anchor: yAnchor }) : null;
    doc.applyGroupTransform(xData, yData, doc.selection);
    console.log(label, 'done.');
```

```
  }

  // Centre all nodes on the spread
  align(
    GroupTransformType.Mid, GroupTransformAnchor.Spread,
    GroupTransformType.Mid, GroupTransformAnchor.Spread,
    'Centre on spread'
  );

  // Now distribute them horizontally with equal spacing
  align(
    GroupTransformType.Space, GroupTransformAnchor.LogicalEdge,
    null, null,
    'Space evenly (horizontal)'
  );

  // Align tops to the topmost selected node
  align(
    null, null,
    GroupTransformType.Min, GroupTransformAnchor.FirstSelected,
    'Align tops'
  );

  console.log('All operations complete.');
```

```
  }
}
```


Affinity SDK – Quick Reference Cheat Sheet

Keep this file open while coding. One page covering the patterns you'll use every session.

23.1 Boilerplate – every script starts here

```
'use strict';

const { Document } = require('/document');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // your code here
}
```

23.2 Imports by task

```
// Document access
const { Document, FileExportOptions, FileExportArea, DocumentPromises } =
require('/document');
const { app } = require('/application');

// Making changes
const { DocumentCommand, CompoundCommandBuilder,
AddChildNodesCommandBuilder } = require('/commands');

// Selections
const { Selection, TextSelection } = require('/selections');

// Colours and fills
const { RGBA8, Colour } = require('/colours');
const { FillDescriptor, SolidFill } = require('/fills');
```

```
// Strokes
const { LineStyle, LineStyleMask } = require('/linestyle');

// Geometry
const { Transform, CurveBuilder, PolyCurve } = require('/geometry');

// Text
const { StoryBuilder } = require('/storybuilder');
const { StoryDelta } = require('/storydelta');
const { GlyphAttStringType } = require('/glyphatts');

// Dialogs
const { Dialog, DialogResult } = require('/dialog');

// Units
const { UnitType } = require('/units');

// Node tree
const Nodes = require('/nodes');

// Files
const { File } = require('/fs');
const { Buffer } = require('/buffer');
const { HttpRequest, RequestMethod } = require('/network');
```

23.3 The command loop

```
const sel = doc.selection;
const compound = CompoundCommandBuilder.create();

for (const node of sel.nodes) {
  const nodeSel = Selection.create(doc, node);
  compound.addCommand( DocumentCommand.createXxx(nodeSel, ...) );
}

doc.executeCommand(compound.createCommand());
```

23.4 Colours

```
const red = RGBA8(220, 50, 50, 255); // r, g, b, a — all 0-255
const clear = RGBA8(0, 0, 0, 0);
const fill = FillDescriptor.createSolid(red);
const noFill = FillDescriptor.createNone();
```

23.5 Apply fill / stroke / opacity

```
// Via DocumentCommand (SDK 3.2: options object first)
doc.executeCommand(DocumentCommand.createSetBrushFill({ fillDescriptor:
fill }, sel)); // interior
doc.executeCommand(DocumentCommand.createSetPenFill({ fillDescriptor:
fill }, sel)); // stroke

// Via doc convenience methods — accept Colour, SolidFill, or
FillDescriptor
doc.setBrushFillDescriptor(colour, sel);
doc.setPenFillDescriptor(fill, sel);

// Opacity and blend mode
doc.setOpacity(0.5, sel); // layer opacity 0.0-1.0
doc.setBrushFillOpacity(0.7, sel); // fill opacity
doc.setLineFillOpacity(0.5, sel); // stroke opacity

// Line style shorthands
doc.setLineWeight(4, sel); // pixels
doc.setLineWeightPts(2, sel); // points
doc.setLineCap(cap, sel);
doc.setLineJoin(join, sel);

// Common doc convenience methods
doc.undo(); doc.redo();
doc.selectAll();
```

```

doc.deleteSelection(sel);
doc.showSelection(sel); doc.hideSelection(sel);
doc.lockSelection(sel); doc.unlockSelection(sel);
doc.convertToCurves(sel);
doc.rasteriseObjects(sel);
doc.flatten(); doc.mergeVisible();

```

23.6 Text replacement

```

// end - 1 excludes the hidden story terminator
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

const si = node.storyInterface;
const range = si.storyRange;
const textSel = TextSelection.create([ { begin: range.begin, end:
range.end - 1 } ]);
const selection = Selection.create(doc, node); // NOT
Selection.createEmpty()
selection.addSubSelectionForNode(node, textSel);
doc.executeCommand(DocumentCommand.createSetText(selection, 'New text'));

```

23.7 Dialog

```

const dlg = Dialog.create('My Tool');
const col = dlg.addColumn();
const grp = col.addGroup('Options');

// Attach controls to dlg for clean access in helper functions
dlg.enable = grp.addCheckBox('Enable', false);
dlg.format = grp.addComboBox('Format', ['PNG', 'PDF'], 0);
dlg.colour = grp.addColourPicker('Colour');
dlg.size = grp.addUnitValueEditor('Size', UnitType.Pixel,
UnitType.Millimetre, 100); // store px, display mm
dlg.count = grp.addUnitValueEditor('Count', UnitType.Number,
UnitType.Number, 6, 1, 100); // unitless integer

// Reactive – per-control (no argument)
dlg.enable.onValueChangedHandler = () => { dlg.size.isEnabled =
dlg.enable.value; };

// Reactive – dialog-level (receives changed control)
dlg.onControlValueChangedHandler = (ctrl) => { /* ctrl === dlg.enable etc.
*/ };

// Simple run
const result = dlg.runModal();
if (result == DialogResult.Ok) { // note: Ok not OK (title-case)
  console.log(dlg.enable.value, dlg.format.selectedIndex,
dlg.colour.value);
}

// Live preview loop
dlg.onControlValueChangedHandler = () => doc.executeCommand(cmd,
true); // true = preview
while (dlg.runModal() == DialogResult.Ok) {
  if (!valid) { alert('Fix this. '); continue; }
  doc.executeCommand(cmd); break;
}
doc.clearPreviews(); // ALWAYS – even on cancel

```

23.8 Export

```

const desktop = app.userDesktopPath;
const opts = FileExportOptions.createWithPresetName('PNG');
const area = FileExportArea.createForWholeDocument();
const records = doc.export(`${desktop}/output.png`, opts, area, null);
console.log(records.all[0]?.isSuccess ? 'OK' : 'FAIL');

```

23.9 Layer effects (shadow example)

```

// sel must be Selection.create(doc, node) – not doc.selection
// Effect commands must be executed individually – CompoundCommandBuilder

```

```

is a silent no-op for effects
const idx = 0;

doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectColour(sel,
RGBA8(0,0,0,180), idx, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOffsetAngle(sel,
6, 315, idx, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
idx, 8, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel,
0.7, idx, true)); // opacity first, then index
// true = enableIfDisabled – adds the effect if not already present

```

23.10 Async pipeline

```

async function main() {
  const docP = await DocumentPromises.getCurrent();
  if (!docP) { console.log('No doc.');
```

```

  return; }

  try {
    await docP.executeCommand(someCmd);
    // mustSaveAs = true for new documents – use saveAs(path) instead of
    save()
    if (docP.document.mustSaveAs) {
      const { app } = require('/application');
      await docP.saveAs(app.userDesktopPath + '/output.af');
    } else {
      await docP.save();
    }
  } catch (err) {
    console.log('Error:', err);
  }
}
main(); // always invoke the async entry point

```

23.11 Additional APIs

23.11.1 Pixel read/write

```

const { PixelReaderWriterRGBA8 } = require('/pixelaccessor');
const bitmap = node.rasterInterface.createCompatibleBitmap(true);
const rw = PixelReaderWriterRGBA8.create(bitmap);
const px = rw.readPixel(x, y); // { r, g, b, alpha } – NOT
{ r, g, b, a }
rw.writePixel(x, y, { r, g, b, alpha }); // key is 'alpha' on write too
// // using 'a:' silently writes
255 – no error!
doc.executeCommand(DocumentCommand.createReplaceBitmap(sel, bitmap));

```

23.11.2 Font enumeration

```

const { Font, FontFamily } = require('/fonts');
const families = FontFamily.all; // all installed families
const fonts = Font.all; // all individual fonts

// Get a specific font via FontFamily
const arial = FontFamily.all.find(f => f.name === 'Arial');
const f = arial?.fontCount > 0 ? arial.getFont(0) : null;
if (f) console.log(f.isSerif, f.isMonospaced, f.postscriptName);

```

23.11.3 Custom node tags

```

const { PredefinedTagKey } = require('/taginterface');
// Write a tag (persists in file)
DocumentCommand.createSetTagValueForKey(sel, 'my-key', 'value');
DocumentCommand.createSetTagValueForPredefinedKey(sel,
PredefinedTagKey.ScriptLabel, 'label');
// Note: node.tagInterface returns undefined – read-back not yet exposed
in SDK

```

23.11.4 New opacity commands

```
DocumentCommand.createSetOpacity(sel, 0.5) // layer opacity
DocumentCommand.createSetBrushFillOpacity(sel, 0.8) // fill opacity
DocumentCommand.createSetLineFillOpacity(sel, 0.6) // stroke opacity
```

23.11.5 ContainerNode (named layer group)

```
const { ContainerNodeDefinition } = require('/nodes');
const { AddChildNodesCommandBuilder } = require('/commands');
const def = ContainerNodeDefinition.create('My Group'); // named
container
const builder = AddChildNodesCommandBuilder.create();
builder.addContainerNode(def);
doc.executeCommand(builder.createCommand(true));

doc.addNode(def); // shorthand for single-node add (no builder needed)
```

23.11.6 Node convenience methods

```
node.userDescription = 'Renamed'; // set layer name directly
node.lock(); // make non-editable
node.unlock();
const copy = node.duplicate(); // returns new node (lands on top)
node.delete(); // remove node
```

23.11.7 Application settings

```
const { app } = require('/application');
app.settings.undoLimit // read-only
app.settings.loadPSDWithEditableText // read/write
app.settings.loadPSDWithEditableText = true
```

23.11.8 Document save and close

```
const { PackageResourcesPolicy } = require('/document');
doc.save(); // save in-place
doc.saveAs('/path/to/file.af');
doc.saveAsPackage('/path/pkg', PackageResourcesPolicy.CopyLinks);
doc.close();
```

23.11.9 Macro commands

```
DocumentCommand.createStartRecordingMacro()
DocumentCommand.createStopRecordingMacro()
DocumentCommand.createReplayMacro()
DocumentCommand.createImportMacro(desktop + '/macro.afmacro')
```

23.11.10 Raster selection commands

```
DocumentCommand.createRasterSelectAll()
DocumentCommand.createRasterDeselect()
DocumentCommand.createRasterInvertSelection()

// Breaking change (SDK 3.2): renamed from rasterSelectPolygon
DocumentCommand.createSetRasterSelectionFromPolygon(polygon, operation,
antialias, feather)

// New in SDK 3.2: selection from object bounds
DocumentCommand.createSetRasterSelectionFromObject(sel, operation,
antialias, feather)

// New in SDK 3.2: refine existing raster selection
DocumentCommand.createGrowShrinkRasterSelection(10, false) // grow by
10px (circular kernel)
DocumentCommand.createFeatherRasterSelection(5) // soften
edge by 5px
DocumentCommand.createSmoothRasterSelection(3) // smooth
boundary by 3px

const { RasterSelectionOutlineAlignment } = require('/commands');
DocumentCommand.createOutlineRasterSelection(4,
RasterSelectionOutlineAlignment.Centre, false) // outline at
radius 4px
```

```
// Doc convenience methods — same signatures without create*
doc.growShrinkRasterSelection(8, true); // grow, circular kernel
doc.featherRasterSelection(5);
doc.smoothRasterSelection(3);
doc.outlineRasterSelection(4,
  RasterSelectionOutlineAlignment.Inside, false);
```

23.11.11 Drawing scale

```
const { DrawingScale } = require('/drawingscale');
const scale = DrawingScale.createFromIntegers(1, 100, true); // 1:100
```

23.12 `createSetEditable` — argument order

// Selection is first argument, editable flag is second (SDK 3.2 breaking change)

```
DocumentCommand.createSetEditable(selection, false) // lock
DocumentCommand.createSetEditable(selection, true)  // unlock
```

23.13 Document convenience shorthands (SDK 3.2)

```
// Text operations
doc.setText('Hello', sel); // replace text in selection
doc.formatText(delta, sel); // apply formatting to selection
doc.insertGlyph(0x2665, sel); // insert ♥ glyph
```

```
// Undo/redo
doc.undo();
doc.redo();
```

```
// Stroke enhancements
doc.setDashPhase(0, sel); // dash phase
doc.setBalancedDashes(true, sel); // balance dashes at corners
```

```
// Dash pattern variants
doc.setDashPattern([8, 4], sel); // regular dash
doc.setDashPattern([8, 4, 2, 4], sel); // dash-dot-dot
```

23.14 Document line style shorthands

// These doc.setXxx() shorthands are simpler than building commands manually

```
doc.setLineWeightPts(2, sel); // set weight in points (auto-converts to px)
doc.setLineCap(2, sel); // 0=Butt, 1=Square, 2=Round
doc.setLineJoin(2, sel); // 0/1=Miter, 2=Round, 3=Bevel
doc.setLineType(LineType.Dash, sel); // set line type
doc.setDashPattern([8, 4], sel); // set dash pattern
doc.setStrokeAlignment(StrokeAlignment.Centre, sel); // stroke alignment
```

23.15 StoryDelta convenience factories

```
const { StoryDelta } = require('/storydelta');
const { FontWeight } = require('/fonts');

// Simple bold + italic
const bold = StoryDelta.createWeight(FontWeight.Bold);
const italic = StoryDelta.createItalic(true);
const combined = StoryDelta.createComposite([bold, italic]);
doc.formatText(combined); // applies to current selection — no Selection object needed
```

23.16 Geometry utilities

```
const { unionRects, TransformBuilder } = require('/geometry');
const combined = unionRects(rect1, rect2); // merge two bounding boxes
```

```
// Transform properties (SDK 3.2)
const tx = node.baseToSpreadTransform;
console.log(tx.xAxis, tx.yAxis, tx.origin);
const inv = tx.inverted;
```

```
// Chainable transform methods (SDK 3.2)
const moved = Transform.translationMatrix(100, 0); //
translate
const scaled = Transform.scaleMatrix(2, 2, point); // scale
about point
const added = moved.translate(50, 0).scale(1.5, 1.5, origin); //
chainable

// around() replaces about() (breaking change SDK 3.2)
const rot = Transform.rotationMatrix(45, Transform.Degrees); // 45deg
about origin
const rot2 = Transform.rotationMatrix(45, Transform.Degrees, center); //
about center

// Static multiply, decompose (SDK 3.2)
const combined2 = Transform.multiply(txA, txB); // combine
transforms
const [sc, tr, rot3] = Transform.decompose(combined2); //
decompose

// TransformBuilder fluent builder
const builder = TransformBuilder.create();
const built = builder
  .setXAxis(1, 0)
  .setYAxis(0, 1)
  .setOrigin(100, 100)
  .build();
```

23.17 HalftoneFilter (SDK 3.2)

```
const { HalftoneFilter } = require('/layereffects');

// Direct apply (no CompoundCommandBuilder – silent no-op for layer
effects)
doc.executeCommand(DocumentCommand.createSetHalftoneParameters(
  sel, HalftoneFilter.Screen, HalftoneFilter.Sine, 0.5, 45, 0.1
));

// Or use live HalftoneFilterRasterNode
const { HalftoneFilterRasterNodeDefinition, HalftoneScreenType,
HalftoneDotType } = require('/nodes');
const def = HalftoneFilterRasterNodeDefinition.createDefault();
```

23.18 DialogButton and DialogColumnStack (SDK 3.2)

```
// Button – use addButton() on a group
const btn = grp.addButton('Click Me');
btn.alignment = HorizontalAlignment.Left;
btn.onClickHandler = () => { console.log('Clicked!'); };

// Column stack layout
const stack = grp.addColumnStack();
const leftCol = stack.addColumn();
const rightCol = stack.addColumn();

// Detect if dialog is running modal (property on Dialog instance)
if (dlg.isRunningModal) { console.log('Dialog is modal.');
```

```
// Toggle item visibility (method on Dialog instance)
dlg.setItemsVisibility([label.itemID, []]); // show label
dlg.setItemsVisibility([], [label.itemID]); // hide label

// Fill/stroke editor
const { DialogFillEditor, DialogStrokeEditor } = require('/dialogui');
const fillEditor = DialogFillEditor.create();
const strokeEditor = DialogStrokeEditor.create();
```

23.19 Fills – options object pattern (SDK 3.2 breaking change)

```
// Old (removed): createSetBrushFill(colourRef, sel)
// New: use options object as first argument
```

```
const desc = FillDescriptor.create(SolidFill, { r: 255, g: 0, b: 0, alpha:
255 });
DocumentCommand.createSetBrushFill({ fillDescriptor: desc }, sel);
DocumentCommand.createSetPenFill({ fillDescriptor: desc }, sel);

// Hatch fill (SDK 3.2 new feature)
const hatchCmd = DocumentCommand.createSetHatchFillAttributes(sel, /*
builder */);
```

23.20 TableTextNode (SDK 3.2)

```
const { TableTextNodeDefinition } = require('/nodes');
const def = TableTextNodeDefinition.create({ x: 0, y: 0, width: 300,
height: 200 }, { width: 3, height: 3 });
doc.addNode(def);
```

23.21 GlyphAtts / ParagraphAtts (SDK 3.2)

```
const { GlyphAtts, GlyphAttStringType, GlyphAttDoubleType } = require('/
glyphatts');
const { ParagraphAtts, ParagraphAttBooleanType, ParagraphAttDoubleType,
ParagraphAttInt32Type } = require('/paragraphatts');

const ga = GlyphAtts.create();
ga.setString(GlyphAttStringType.StyleName, 'Heading');
ga.setDouble(GlyphAttDoubleType.Height, 24);

const pa = ParagraphAtts.create();
pa.setBoolean(ParagraphAttBooleanType.Caps, true);
```

23.22 Attribute runs on Story (SDK 3.2)

```
for (const run of story.glyphAttRuns) {
  console.log(run.begin, run.end, run.glyphAtts);
}
```

23.23 Fluent node-definition builders (SDK 3.2)

```
const def = PolyCurveNodeDefinition.createDefault()
  .setUserDescription('My Shape')
  .setTransform(xf);
```

23.24 Colour → FillDescriptor auto-convert (SDK 3.2)

```
// Fill/line methods accept Colour directly
def.addBrushFillDescriptor(rgba8(200, 80, 40, 255));
```

23.25 `node.spreadBaseBox` (SDK 3.2)

```
const box = node.spreadBaseBox; // equivalent to getSpreadBaseBox(true)
```

23.26 Spread / artboard document properties (SDK 3.2)

```
const { ArtboardDocumentProperties } = require('/documentproperties');
const props =
ArtboardDocumentProperties.create().setUseMargin(true).setMargin(margin);
doc.setArtboardDocumentProperties(artboard, props);
doc.setArtboardSizeWithAnchor(artboard, 400, 300, SpatialAnchor.Centre);
```

23.27 Critical rules

Rule	Why
<code>require('/module')</code> not <code>./module</code>	SDK module registry, not file system
<code>doc.title</code> not <code>doc.name</code>	<code>.name</code> is undefined
<code>rgba8()</code> not <code>Colour.fromRGBA()</code>	<code>.fromRGBA</code> doesn't exist
<code>doc.executeCommand(cmd, true)</code> for preview, <code>doc.clearPreviews()</code> after loop	Without <code>clearPreviews()</code> , cancelled previews stay on canvas permanently

Rule	Why
Previews don't stack — rebuild in one command each update	SDK replaces previous preview; layering commands doesn't accumulate
<code>result == DialogResult.Ok</code>	Use title-case Ok not OK loose equality <code>==</code> works correctly
<code>range.end - 1</code> for text selections	Excludes the hidden story terminator
<code>Selection.create(doc, node)</code> for text	<code>Selection.createEmpty()</code> causes silent failure
Read <code>doc.selection</code> BEFORE <code>setCurrentSpread</code>	Spread change clears selection
Compound commands for multi-step changes	One undo step, not N separate entries
Desktop only for file access	Other paths → <code>PERMISSION_DENIED</code>
No <code>toLocaleString()</code>	Not supported — use plain number or <code>String(n)</code>
Check <code>String(ri.format) === 'RGBA8'</code> before <code>PixelReaderWriter.RGBA8.create()</code>	Format mismatch throws <code>INVALID_ARGS</code> — format is an enum object, use <code>String()</code> to compare
<code>bitmap.format</code> returns a plain integer not a name	Use <code>RasterFormat.keys/values</code> to look up — see lesson 14 loading example
<code>res.response.statusCode == 200</code> not <code>res.statusCode</code>	<code>statusCode</code> is on <code>res.response</code> , not <code>res</code> directly; body is <code>res.response.content</code>
<code>create*Filter</code> not <code>createSet*FilterParameters</code> to apply a new filter	<code>createSet*</code> is a silent no-op if no filter exists yet
No <code>CompoundCommandBuilder</code> for effect commands	Silent no-op — execute each effect command individually
<code>setTimeout(delay, callback)</code> not <code>setTimeout(callback, delay)</code>	SDK order is opposite to Node/browser
<code>main()</code> call at bottom of async scripts	Async functions don't auto-run
<code>Transform.around()</code> replaces <code>Transform.about()</code> (SDK 3.2)	<code>about()</code> removed — use <code>around()</code>
<code>createSetRasterSelectionFromPolygon()</code> replaces <code>rasterSelectPolygon()</code> (SDK 3.2)	Old name removed
<code>createSetEditable(sel, bool)</code> — selection first (SDK 3.2)	Selection is now first argument
<code>createSetBrushFill({ fillDescriptor }, sel)</code> — options object (SDK 3.2)	Old signature removed

Rule	Why
<code>dialog.setIsEnabledBySelector()</code> / <code>setIsVisibleBySelector()</code> are silent no-ops (SDK 3.2)	Use <code>setItemsVisibility()</code> instead
<code>TableTextNodeDefinition.create(rect, {width, height})</code> not <code>create(rect, width, height)</code>	Grid dimensions are an object
Dialog values are read-only after build phase	Must set values before <code>runModal()</code>
<code>setCurrentSpread</code> clears selection	Read <code>doc.selection</code> first
<code>node.spreadBaseBox</code> equivalent to <code>getSpreadBaseBox(true)</code> (SDK 3.2)	Convenience property, defaults include <code>Clips</code> to <code>true</code>
<code>RasterSelectionOutlineAlignment</code> — Centre, Inside, Outside	Required for <code>createOutlineRasterSelection</code> (SDK 3.2)
Fill/line methods on definitions accept <code>Colour</code> directly (SDK 3.2)	A solid fill is created automatically
Node-definition setters are chainable (SDK 3.2)	<code>.setDescription()</code> , <code>.setTransform()</code> , etc. return this
<code>DialogControl.setIsEnabled()</code> now works correctly (SDK 3.2)	Previously a no-op — now updates <code>isEnabled</code> properly
Handlers fire on user interaction only (SDK 3.2)	Programmatic value changes no longer trigger <code>onValueChangedHandler</code>
Geometry structs have proper wrapper types (SDK 3.2)	<code>Rectangle</code> , <code>Point</code> , <code>Vector</code> , etc. are documented types, not anonymous objects